EJERCICIO 4

1.Enunciado del problema

En una zapateria, cree un algoritmo que lea tres lados (reales) de un triangulo y determine si forman un triangulo valido. Evalúe la expresion logica del teorema de la desigualdad triangular: (lado1 + lado2 > lado3 AND lado1 + lado3 > lado2 AND lado2 + lado3 > lado1). Si es valido, clasifiquelo usando expresiones logicas: equilatero (los tres iguales), isosceles (al menos dos iguales), escaleno (todos diferentes). Asigne el tipo como cadena a una variable. Muestre por pantalla el tipo de triangulo. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue al final un resumen de los datos ingresados.

2.ANÁLISIS DEL PROBLEMA

Análisis del problema: Validación y clasificación de un triángulo

1. Datos de entrada

Se ingresan por teclado tres números reales que representan los lados de un triángulo.

Variable	Descripción	Tipo de dato
lado1	Primer lado del triángulo.	float
lado2	Segundo lado del triángulo.	float
lado3	Tercer lado del triángulo.	float

2. Proceso

1. Leer los tres lados del triángulo: lado1, lado2 y lado3.

2. Validar si los lados forman un triángulo mediante la siguiente expresión lógica:

$$\begin{aligned}&\left(lado1 + lado2 > lado3\right)\\ \wedge &\left(lado1 + lado3 > lado2\right)\\ \wedge &\left(lado2 + lado3 > lado1\right)\end{aligned}$$

El resultado se almacena en la variable valido:

- Si las tres condiciones se cumplen, valido = 1.
- Si alguna condición no se cumple, valido = 0.

3. Clasificar el triángulo si es válido:

- Equilátero: cuando los tres lados son iguales.
- Isósceles: cuando al menos dos lados son iguales.

- Escaleno: cuando los tres lados son diferentes.

4. Si los lados no forman un triángulo válido, asignar el texto "No valido" a la variable tipo.

5. Calcular el valor mínimo numérico entre las variables lado1, lado2, lado3 y valido.

El procedimiento consiste en:

- Asignar inicialmente lado1 a minimo.
- Comparar lado2 con minimo y actualizarlo si es menor.
- Comparar lado3 con minimo y actualizarlo si es menor.
- Comparar valido con minimo y actualizarlo si es menor.

6. Mostrar los resultados y un resumen final con los tres lados, la validación, el tipo de triángulo y el valor mínimo numérico.

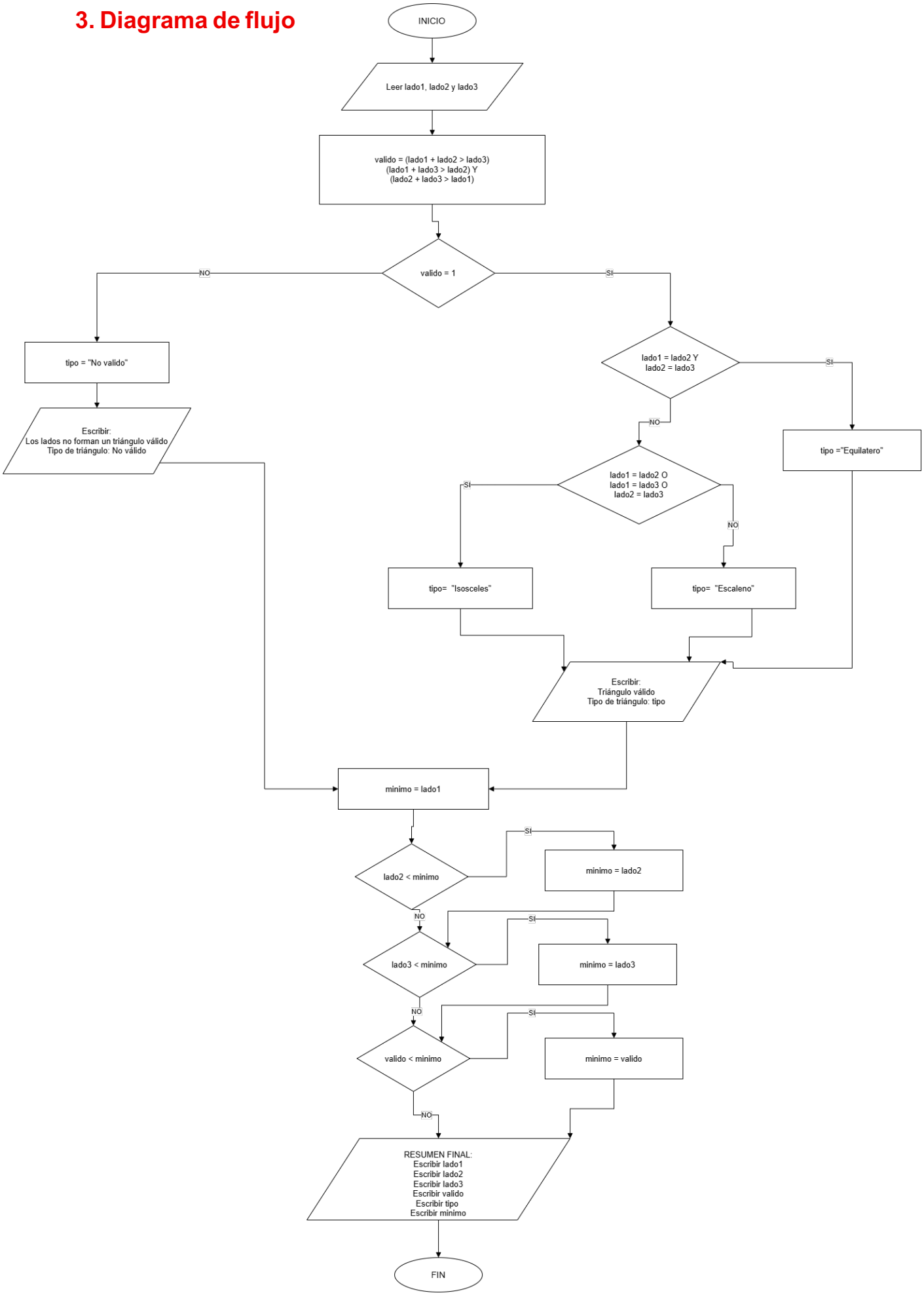
3. Datos de salida

Dato de salida	**Descripción**
valido	Indica si los lados forman un triángulo válido (1 = verdadero, 0 = falso).
tipo	Muestra si el triángulo es equilátero, isósceles, escaleno o no válido.
minimo	Muestra el menor valor numérico entre los tres lados y la variable valido.
Resumen final	Presenta los valores ingresados y los resultados obtenidos.

4. Variables a utilizar y tipo de dato

Variable	Tipo de dato	Descripción
lado1	float	Almacena el primer lado del triángulo.
lado2	float	Almacena el segundo lado del triángulo.
lado3	float	Almacena el tercer lado del triángulo.
valido	int	Almacena el resultado de la validación (1 o 0).
tipo	char[20]	Almacena la clasificación del triángulo como texto.
minimo	float	Almacena el menor valor numérico encontrado.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Clasificacion_Triangulos

    Definir lado1, lado2, lado3 Como Real
    Definir minimo Como Real
    Definir valido Como Entero
    Definir tipo Como Caracter

    Escribir "Ingrese el primer lado:"
    Leer lado1

    Escribir "Ingrese el segundo lado:"
    Leer lado2

    Escribir "Ingrese el tercer lado:"
    Leer lado3

    valido <- (lado1 + lado2 > lado3 Y lado1 + lado3 > lado2 Y lado2 + lado3 > lado1)

    Si valido Entonces

        Si lado1 = lado2 Y lado2 = lado3 Entonces
            tipo <- "Equilatero"

        Sino
            Si lado1 = lado2 O lado1 = lado3 O lado2 = lado3 Entonces
                tipo <- "Isosceles"
            Sino
                tipo <- "Escaleno"
            FinSi
        FinSi

        Escribir ""
        Escribir "Triangulo valido."
        Escribir "Tipo de triangulo: ", tipo

    Sino

        tipo <- "No valido"

        Escribir ""
        Escribir "Los lados no forman un triangulo valido."
        Escribir "Tipo de triangulo: ", tipo

    FinSi

    minimo <- lado1

    Si lado2 < minimo Entonces
        minimo <- lado2
    FinSi

    Si lado3 < minimo Entonces
        minimo <- lado3
    FinSi

    Si valido < minimo Entonces
        minimo <- valido
    FinSi

    Escribir "Valor minimo numerico = ", minimo

    Escribir ""
    Escribir "RESUMEN FINAL"
    Escribir "Primer lado: ", lado1
    Escribir "Segundo lado: ", lado2
    Escribir "Tercer lado: ", lado3
    Escribir "Triangulo valido: ", valido
    Escribir "Tipo de triangulo: ", tipo
    Escribir "Valor minimo numerico: ", minimo

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1: Triángulo equilátero

Datos de entrada:

- lado1 = 20
- lado2 = 20
- lado3 = 20

Tabla de prueba de escritorio

Paso	Variables / valores	Evaluación de condiciones	Salida en pantalla
1	lado1 = 20	Se solicita el primer lado.	Ingrese el primer lado: 20
2	lado2 = 20	Se solicita el segundo lado.	Ingrese el segundo lado: 20
3	lado3 = 20	Se solicita el tercer lado.	Ingrese el tercer lado: 20
4	valido = 1	(20+20>20) Y (20+20>20) Y (20+20>20) = Verdadero	Ninguna
5	valido = 1	Si valido = Verdadero	Triángulo válido.
6	tipo = Equilatero	(20=20) Y (20=20) = Verdadero	Tipo de triángulo: Equilatero
7	minimo = 20	Se asigna minimo = lado1.	Ninguna
8	minimo = 20	lado2 < minimo: 20 < 20 = Falso	Ninguna
9	minimo = 20	lado3 < minimo: 20 < 20 = Falso	Ninguna
10	minimo = 1	valido < minimo: 1 < 20 = Verdadero	Valor minimo numerico = 1.00
11	Todas las variables	Se muestran los valores finales.	RESUMEN FINAL

5.2. Prueba de escritorio N.º 2: Triángulo no válido

Datos de entrada:

- lado1 = 20
- lado2 = 40
- lado3 = 60

Tabla de prueba de escritorio

Paso	Variables / valores	Evaluación de condiciones	Salida en pantalla
1	lado1 = 20	Se solicita el primer lado.	Ingrese el primer lado: 20
2	lado2 = 40	Se solicita el segundo lado.	Ingrese el segundo lado: 40
3	lado3 = 60	Se solicita el tercer lado.	Ingrese el tercer lado: 60
4	valido = 0	(20+40>60) = Falso. La condición completa es Falsa.	Ninguna
5	valido = 0	Si valido = Falso.	Los lados no forman un triángulo válido.
6	tipo = No valido	Se ejecuta el bloque Sino.	Tipo de triángulo: No valido
7	minimo = 20	Se asigna minimo = lado1.	Ninguna
8	minimo = 20	lado2 < minimo: 40 < 20 = Falso	Ninguna
9	minimo = 20	lado3 < minimo: 60 < 20 = Falso	Ninguna
10	minimo = 0	valido < minimo: 0 < 20 = Verdadero	Valor minimo numerico = 0.00
11	Todas las variables	Se muestran los valores finales.	RESUMEN FINAL

6. Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{

    float lado1;
    float lado2;
    float lado3;
    int valido;
    char tipo[20];
    float minimo;

    printf("Ingrese el primer lado:");
    scanf("%f", &lado1);

    printf("Ingrese el segundo lado:");
    scanf("%f", &lado2);

    printf("Ingrese el tercer lado:");
    scanf("%f", &lado3);


    valido = (lado1 + lado2 > lado3 && lado1 + lado3 > lado2 && lado2 + lado3 > lado1);

    if (valido)
    {
        if (lado1 == lado2 && lado2 == lado3)
        {
            sprintf(tipo, "Equilatero");
        }
        else if (lado1 == lado2 ||
                 lado1 == lado3 ||
                 lado2 == lado3)
        {

            sprintf(tipo, "Isosceles");
        }
        else
        {
            sprintf(tipo, "Escaleno");
        }

        printf("\nTriangulo valido.\n");

        printf("Tipo de triangulo: %s\n", tipo);
    }
    else
    {
        sprintf(tipo, "No valido");

        printf("\nLos lados no forman un triangulo valido.\n");

        printf("Tipo de triangulo: %s\n", tipo);
    }

    minimo = lado1;

    if (lado2 < minimo)
        minimo = lado2;

    if (lado3 < minimo)
        minimo = lado3;

    if (valido < minimo)
        minimo = valido;

    printf("Valor minimo numerico = %.2f\n", minimo);

    printf("\nRESUMEN FINAL\n");
    printf("Primer lado: %.2f\n", lado1);
    printf("Segundo lado: %.2f\n", lado2);
    printf("Tercer lado: %.2f\n", lado3);
    printf("Triangulo valido: %d\n", valido);
    printf("Tipo de triangulo: %s\n", tipo);
    printf("Valor minimo numerico: %.2f\n", minimo);


}
```

7. Salida del programa

Prueba 1

```
Ingrese el primer lado:20
Ingrese el segundo lado:20
Ingrese el tercer lado:20

Triangulo valido.
Tipo de triangulo: Equilatero
Valor minimo numerico = 1.00

RESUMEN FINAL
Primer lado: 20.00
Segundo lado: 20.00
Tercer lado: 20.00
Triangulo valido: 1
Tipo de triangulo: Equilatero
Valor minimo numerico: 1.00
```

Prueba 2

```
Ingrese el primer lado:20
Ingrese el segundo lado:40
Ingrese el tercer lado:60

Los lados no forman un triangulo valido.
Tipo de triangulo: No valido
Valor minimo numerico = 0.00

RESUMEN FINAL
Primer lado: 20.00
Segundo lado: 40.00
Tercer lado: 60.00
Triangulo valido: 0
Tipo de triangulo: No valido
Valor minimo numerico: 0.00
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 1

1.Enunciado del problema

En una joyería, diseñe un algoritmo que calcule el salario neto de un trabajador. Lea el salario bruto (real), el número de cargas familiares (entero) y los años de antiguedad (entero). Use constantes simbólicas BONO_CARGA = 50.0, BONO_ANTIGUEDAD = 30.0, DESCUENTO_ISSS = 0.03, DESCUENTO_AFP = 0.0725, LIMITE_ANTIGUEDAD = 10. Calcule: bonos = cargas * BONO_CARGA + min(antiguedad, LIMITE_ANTIGUEDAD) * BONO_ANTIGUEDAD; descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP); salario_neto = salario_bruto + bonos - descuentos. Evalúe la expresion lógica (salario_neto > salario_bruto). Muestre por pantalla el desglose. Además, calcule el valor minimo de todas las variables utilizadas. Identifique explícitamente los datos de entrada y la información de salida.

2. Análisis del problema

Nombre del problema: Cálculo del salario neto de un trabajador de una joyería.

El algoritmo permite calcular el salario neto de un trabajador tomando en cuenta su salario bruto, las cargas familiares, los años de antigüedad, los bonos adicionales y los descuentos correspondientes al ISSS y AFP. Además, determina si el salario neto es mayor que el salario bruto y encuentra el valor mínimo entre las variables utilizadas.

1. Datos de entrada

Son los datos que el usuario debe ingresar para realizar los cálculos.

Dato de entrada	Descripción	Tipo de dato
salario_bruto	Sueldo inicial del trabajador.	float
cargas	Número de cargas familiares del trabajador.	int
antiguedad	Años que lleva trabajando en la empresa.	int

2. Proceso

Se realizan las siguientes operaciones matemáticas y comparaciones:

Paso 1. Calcular el bono por cargas familiares.

Cada carga familiar recibe un bono de $50.

$$BonoCarga = cargas \times 50$$

Paso 2. Calcular el bono por antigüedad.

Se otorgan $30 por cada año trabajado, con un límite máximo de 10 años.

$$BonoAntiguedad = min\left(antiguedad,10\right) \times 30$$

Paso 3. Calcular el total de bonos.

$$Bonos = BonoCarga + BonoAntiguedad$$

Paso 4. Calcular los descuentos.

Se aplica el 3 % de ISSS y el 7,25 % de AFP sobre el salario bruto.

$$Descuentos = salario_bruto \times \left(0.03 + 0.0725\right)$$

Paso 5. Calcular el salario neto.

$$SalarioNeto = salarioBruto + Bonos - Descuentos$$

Paso 6. Comparar el salario neto con el salario bruto.

- Si el salario neto es mayor que el salario bruto, mostrar que es mayor.
- Caso contrario, mostrar que no es mayor.

Paso 7. Encontrar el valor mínimo.

Se compara el valor de las siguientes variables para encontrar el menor:

- Salario bruto.
- Cargas familiares.
- Antigüedad.
- Bonos.
- Descuentos.
- Salario neto.

3. Datos de salida

Son los resultados que presenta el programa al finalizar los cálculos.

Dato de salida	Descripción
Salario neto	Cantidad final que recibe el trabajador después de sumar los bonos y restar los descuentos.
Salario bruto	Sueldo inicial del trabajador antes de aplicar descuentos y bonos.
Bonos	Cantidad total de dinero adicional por cargas familiares y antigüedad.
Descuentos	Total descontado por concepto de ISSS y AFP.
Comparación	Indica si el salario neto es mayor que el salario bruto.
Valor mínimo	Muestra el menor valor encontrado entre las variables comparadas.

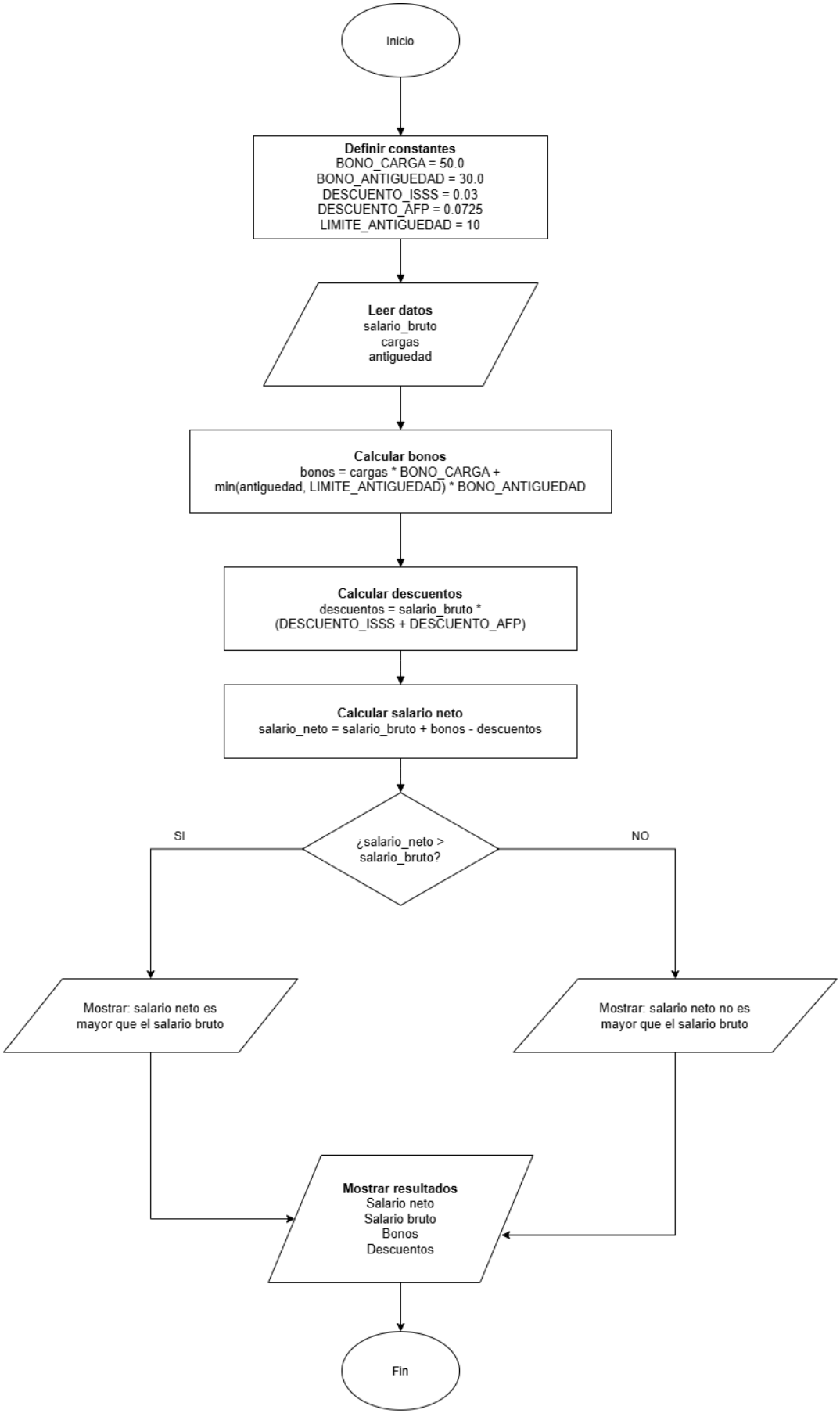
4. Variables a utilizar

Variable	Tipo de dato	Descripción
salario_bruto	float	Almacena el salario inicial del trabajador.
cargas	int	Almacena el número de cargas familiares.
antiguedad	int	Almacena los años de antigüedad.
bonos	float	Almacena el total de bonos calculados.
descuentos	float	Almacena el total de descuentos.
salario_neto	float	Almacena el salario final del trabajador.
minimo	float	Almacena el menor valor encontrado.

5. Constantes utilizadas

Constante	Valor	Descripción
BONO_CARGA	50.0	Bono por cada carga familiar.
BONO_ANTIGUEDAD	30.0	Bono por cada año de antigüedad.
DESCUENTO_ISSS	0.03	Descuento del 3 % por ISSS.
DESCUENTO_AFP	0.0725	Descuento del 7,25 % por AFP.
LIMITE_ANTIGUEDAD	10	Máximo de años considerados para el bono de antigüedad.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Salario_Neto_Joyeria

    // Definicion de constantes
    BONO_CARGA <- 50.0
    BONO_ANTIGUEDAD <- 30.0
    DESCUENTO_ISSS <- 0.03
    DESCUENTO_AFP <- 0.0725
    LIMITE_ANTIGUEDAD <- 10

    // Declaracion de variables
    Definir salario_bruto Como Real
    Definir cargas Como Entero
    Definir antiguedad Como Entero
    Definir bonos Como Real
    Definir descuentos Como Real
    Definir salario_neto Como Real
    Definir minimo Como Real

    // Datos de entrada
    Escribir "Ingrese el salario bruto:"
    Leer salario_bruto

    Escribir "Numero de cargas familiares:"
    Leer cargas

    Escribir "Anios de antiguedad:"
    Leer antiguedad

    // Calculo de bonos
    bonos <- cargas * BONO_CARGA + Min(antiguedad, LIMITE_ANTIGUEDAD) * BONO_ANTIGUEDAD

    // Calculo de descuentos
    descuentos <- salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP)

    // Calculo del salario neto
    salario_neto <- salario_bruto + bonos - descuentos

    // Evaluacion de la expresion logica
    Si salario_neto > salario_bruto Entonces
        Escribir "Salario neto es mayor que el salario bruto"
    SiNo
        Escribir "Salario neto no es mayor que el salario bruto"
    FinSi

    // Mostrar resultados
    Escribir "Salario neto es: $", salario_neto
    Escribir "Salario bruto es: $", salario_bruto
    Escribir "Cantidad del bono es: $", bonos
    Escribir "Descuento es: $", descuentos

    // Calculo del valor minimo
    minimo <- salario_bruto

    Si cargas < minimo Entonces
        minimo <- cargas
    FinSi

    Si antiguedad < minimo Entonces
        minimo <- antiguedad
    FinSi

    Si bonos < minimo Entonces
        minimo <- bonos
    FinSi

    Si descuentos < minimo Entonces
        minimo <- descuentos
    FinSi

    Si salario_neto < minimo Entonces
        minimo <- salario_neto
    FinSi

    Escribir "Valor minimo: ", minimo

FinAlgoritmo
```

5.1 Prueba de escritorio 1

Se utilizaron los siguientes datos de ejemplo para comprobar los cálculos del algoritmo.

Tabla 1. Datos de entrada

Dato	Valor
Salario bruto	$470.00
Cargas familiares	5
Antigüedad	25 años

Tabla 2. Cálculo de los bonos

Paso	Variable	Operación	Resultado
1	bonoCargas	5 × 50	$250.00
2	bonoAntiguedad	10 × 30	$300.00
3	bonos	250 + 300	$550.00

Tabla 3. Cálculo de los descuentos

Paso	Variable	Operación	Resultado
4	descuentoISSS	470 × 0.03	$14.10
5	descuentoAFP	470 × 0.0725	$34.075
6	descuentos	14.10 + 34.075	$48.175

Tabla 4. Cálculo del salario neto

Paso	Variable	Operación	Resultado
7	salarioNeto	470 + 550 − 48.175	$971.825
8	salarioNeto > salarioBruto	971.825 > 470	Verdadero (1)

Resultado final: el salario neto es de $971.82.

5.2 Prueba de escritorio 2

Se utilizaron otros datos de ejemplo para comprobar los cálculos del algoritmo del salario neto.

Tabla 1. Datos de entrada

Dato	Valor
Salario bruto	$600.00
Cargas familiares	3
Antigüedad	5 años

Tabla 2. Cálculo de los bonos

Paso	Variable	Operación	Resultado
1	bonoCargas	3 × 50	$150.00
2	bonoAntiguedad	5 × 30	$150.00
3	bonos	150 + 150	$300.00

Tabla 3. Cálculo de los descuentos

Paso	Variable	Operación	Resultado
4	descuentoISSS	600 × 0.03	$18.00
5	descuentoAFP	600 × 0.0725	$43.50
6	descuentos	18 + 43.50	$61.50

Tabla 4. Cálculo del salario neto

Paso	Variable	Operación	Resultado
7	salarioNeto	600 + 300 − 61.50	$838.50
8	salarioNeto > salarioBruto	838.50 > 600	Verdadero (1)

Resultado final: el salario neto es de $838.50

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define BONO_CARGA 50.0
#define BONO_ANTIGUEDAD 30.0
#define DESCUENTO_ISSS 0.03
#define DESCUENTO_AFP 0.0725
#define LIMITE_ANTIGUEDAD 10

int min(int a,int b){
    return(a<b) ? a : b;

}

int main() {

    float salario_bruto;
    int cargas;
    int antiguedad;
    float bonos;
    float descuentos;
    float salario_neto;
    float minimo;

    printf("Ingreso del salario bruto\n");
    scanf("%f", &salario_bruto);

    printf("Numero de cargas familiares\n");
    scanf("%d", &cargas);

    printf ("Anios de antiguedad\n");
    scanf("%d", &antiguedad);

    bonos = cargas * BONO_CARGA + min(antiguedad, LIMITE_ANTIGUEDAD) * BONO_ANTIGUEDAD;
    descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP);
     salario_neto = salario_bruto + bonos - descuentos;

     if(salario_neto > salario_bruto){
        printf ("Salario neto es mayor que el salario bruto\n");

     }else{
         printf ("Salario neto no es mayor que el salario bruto\n");
     }

     printf("salarito neto es: $%.2f\n",salario_neto);
     printf("salarito bruto es: $%.2f\n:",salario_bruto);
     printf("cantidad del bono es: $%.2f\n:",bonos);
     printf("descuento es: $%.2f\n:",descuentos);

     minimo = salario_bruto;

  if (cargas < minimo)
        minimo = cargas;

    if (antiguedad < minimo)
        minimo = antiguedad;

    if (bonos < minimo)
        minimo = bonos;

    if (descuentos < minimo)
        minimo = descuentos;

    if (salario_neto < minimo)
        minimo = salario_neto;

     printf("Valor minimo: %.2f\n", minimo);

    }
```

7. Salida del programa

Prueba 1

```
PS C:\msys64\EJEMPLOS C> ./ejercicio1.exe
Ingreso del salario bruto
470
Numero de cargas familiares
5
Anios de antiguedad
25
Salario neto es mayor que el salario bruto
salarito neto es: $971.83
salarito bruto es: $470.00
:cantidad del bono es: $550.00
:descuento es: $48.17
:Valor minimo: 5.00
```

Prueba 2

```
PS C:\msys64\EJEMPLOS C> ./ejercicio1.exe
Ingreso del salario bruto
600
Numero de cargas familiares
3
Anios de antiguedad
5
Salario neto es mayor que el salario bruto
salarito neto es: $838.50
salarito bruto es: $600.00
:cantidad del bono es: $300.00
:descuento es: $61.50
:Valor minimo: 3.00
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 2

1.Enunciado del problema

En una empresa de transporte, construya un algoritmo que simule un sistema de validacion de PIN. Use una constante simbolica PIN_CORRECTO = 1234 (entero de 4 digitos) y MAX_INTENTOS = 3. Lea un PIN ingresado (entero). Evalúe la expresion relacional (pin_ingresado == PIN_CORRECTO). Asigne el resultado a una variable booleana acceso_permitido. Use la expresion aritmetica (intentos + 1) para llevar un contador (inicializado con literal 0). Evalúe la expresion logica (intentos < MAX_INTENTOS AND NOT acceso_permitido). Muestre por pantalla el estado de la validacion. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de inicializar todas las variables antes de usarlas.

2. Análisis del problema

Nombre del problema: Sistema de validación de PIN en una empresa de transporte.

El algoritmo permite simular un sistema de seguridad mediante la validación de un PIN de cuatro dígitos. El usuario debe ingresar un PIN y el programa verifica si coincide con el PIN correcto establecido. Se dispone de un máximo de tres intentos. Si el usuario introduce el PIN correcto, se permite el acceso; de lo contrario, se deniega cuando se agotan los intentos. Finalmente, se calcula el valor mínimo de todas las variables y constantes utilizadas.

1. Datos de entrada

Dato de entrada	Descripción	Tipo de dato
pin_ingresado	Almacena el PIN de cuatro dígitos que introduce el usuario.	int

2. Proceso

Se realizan las siguientes operaciones matemáticas, relacionales y lógicas:

Paso 1. Declarar las constantes simbólicas.

- PIN_CORRECTO = 1234: establece el PIN válido.
- MAX_INTENTOS = 3: establece el número máximo de intentos permitidos.

Paso 2. Inicializar las variables.

Se asignan valores iniciales antes de utilizarlas.

- pin_ingresado = 0
- intentos = 0
- acceso_permitido = 0
- minimo = 0.0

Paso 3. Evaluar la expresión lógica.

Se verifica que los intentos sean menores que el máximo permitido y que el acceso todavía no haya sido autorizado.

(intentos < MAX_INTENTOS AND NOT acceso_permitido)

Paso 4. Solicitar el PIN al usuario.

Se solicita ingresar un PIN de cuatro dígitos.

Paso 5. Validar el PIN ingresado.

Se compara el PIN ingresado con el PIN correcto.

(pin_ingresado == PIN_CORRECTO)

El resultado se almacena en acceso_permitido.

- Si el PIN coincide, se permite el acceso.
- Si el PIN no coincide, se indica que es incorrecto.

Paso 6. Incrementar el contador de intentos.

Se aumenta el número de intentos realizados.

intentos + 1

Paso 7. Mostrar el estado de validación.

- Si el PIN es correcto, se muestra acceso permitido.
- Si es incorrecto, se informa al usuario.
- Si se agotan los tres intentos, se muestra acceso denegado.

Paso 8. Calcular el valor mínimo.

Se comparan los valores finales de todas las variables y constantes utilizadas, almacenando el menor valor numérico en la variable minimo.

3. Datos de salida

Dato de salida	Descripción
Estado del PIN	Indica si el PIN ingresado es correcto o incorrecto.
Acceso permitido	Muestra un mensaje cuando el usuario ingresa el PIN correcto.
Intentos realizados	Indica cuántos intentos ha utilizado el usuario.
Acceso denegado	Informa que se agotaron los tres intentos sin ingresar el PIN correcto.
Valor mínimo	Muestra el menor valor numérico encontrado entre todas las variables y constantes.

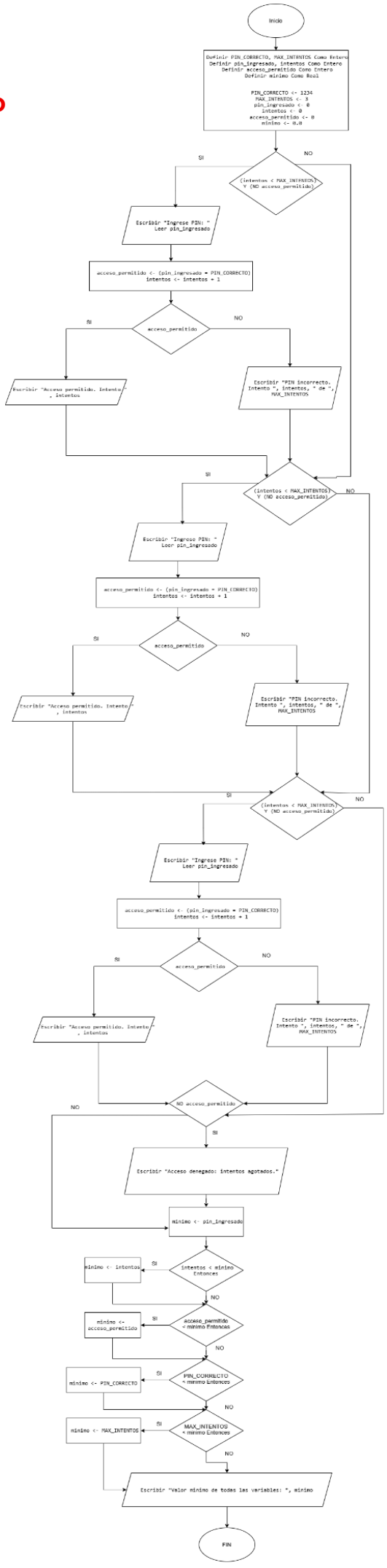
4. Variables a utilizar

Variable	Tipo de dato	Descripción
pin_ingresado	int	Almacena el PIN que introduce el usuario.
intentos	int	Cuenta el número de intentos realizados.
acceso_permitido	int	Almacena el resultado de la validación: 1 (verdadero) o 0 (falso).
minimo	double	Almacena el menor valor numérico encontrado entre las variables y constantes.

5. Constantes utilizadas

Constante	Tipo de dato	Valor	Descripción
PIN_CORRECTO	int	1234	Establece el PIN correcto de acceso.
MAX_INTENTOS	int	3	Establece el máximo de intentos permitidos.

3. Diagrama de flujo



4. Pseucodigo

```
Algoritmo Validacion_PIN

    Definir PIN_CORRECTO, MAX_INTENTOS Como Entero
    Definir pin_ingresado, intentos Como Entero
    Definir acceso_permitido Como Entero
    Definir minimo Como Real

    PIN_CORRECTO <- 1234
    MAX_INTENTOS <- 3
    pin_ingresado <- 0
    intentos <- 0
    acceso_permitido <- 0
    minimo <- 0.0

    Si (intentos < MAX_INTENTOS) Y (NO acceso_permitido) Entonces

        Escribir "Ingrese PIN: "
        Leer pin_ingresado

        acceso_permitido <- (pin_ingresado = PIN_CORRECTO)
        intentos <- intentos + 1

        Si acceso_permitido Entonces
            Escribir "Acceso permitido. Intento ", intentos
        SiNo
            Escribir "PIN incorrecto. Intento ", intentos, " de ", MAX_INTENTOS
        FinSi

    FinSi

    Si (intentos < MAX_INTENTOS) Y (NO acceso_permitido) Entonces

        Escribir "Ingrese PIN: "
        Leer pin_ingresado

        acceso_permitido <- (pin_ingresado = PIN_CORRECTO)
        intentos <- intentos + 1

        Si acceso_permitido Entonces
            Escribir "Acceso permitido. Intento ", intentos
        SiNo
            Escribir "PIN incorrecto. Intento ", intentos, " de ", MAX_INTENTOS
        FinSi

    FinSi

    Si (intentos < MAX_INTENTOS) Y (NO acceso_permitido) Entonces

        Escribir "Ingrese PIN: "
        Leer pin_ingresado

        acceso_permitido <- (pin_ingresado = PIN_CORRECTO)
        intentos <- intentos + 1

        Si acceso_permitido Entonces
            Escribir "Acceso permitido. Intento ", intentos
        SiNo
            Escribir "PIN incorrecto. Intento ", intentos, " de ", MAX_INTENTOS
        FinSi

    FinSi

    Si NO acceso_permitido Entonces
        Escribir "Acceso denegado: intentos agotados."
    FinSi

    minimo <- pin_ingresado

    Si intentos < minimo Entonces
        minimo <- intentos
    FinSi

    Si acceso_permitido < minimo Entonces
        minimo <- acceso_permitido
    FinSi

    Si PIN_CORRECTO < minimo Entonces
        minimo <- PIN_CORRECTO
    FinSi

    Si MAX_INTENTOS < minimo Entonces
        minimo <- MAX_INTENTOS
    FinSi

    Escribir "Valor minimo de todas las variables: ", minimo

FinAlgoritmo
```

5.1 Prueba de escritorio: Validación de PIN

Se utilizaron los siguientes datos de ejemplo para comprobar los cálculos del algoritmo.

1. Datos iniciales

Variable	Valor
PIN_CORRECTO	1234
MAX_INTENTOS	3
pin_ingresado	1234
intentos	0
acceso_permitido	0
validacion	0

2. Tabla de operaciones

Paso	Variable	Operación	Resultado
1	pin_ingresado	Ingresar PIN	1234
2	intentos	0 + 1	1
3	acceso_permitido	1234 == 1234	Verdadero
4	validacion	(1 < 3) AND NOT Verdadero	Falso

3. Resultado final

Variable	Resultado
pin_ingresado	1234
intentos	1
acceso_permitido	Verdadero
validacion	Falso

4. Mensaje en pantalla

Mensaje	Resultado
Validación del PIN	PIN correcto. Acceso permitido.
Intentos realizados	1
Acceso permitido	Verdadero
Estado de validación	Falso

5.2 Prueba de escritorio 2: Validación de PIN

Datos de ejemplo: PIN_CORRECTO = 1234 y MAX_INTENTOS = 3.

1. Datos iniciales

Variable	Valor
PIN_CORRECTO	1234
MAX_INTENTOS	3
pin_ingresado	4321, 0000, 2222
intentos	0
acceso_permitido	0
validacion	0

2. Tabla de operaciones

Paso	Variable	Operación	Resultado
1	pin_ingresado	Ingresar primer PIN	4321
2	intentos	0 + 1	1
3	acceso_permitido	4321 == 1234	Falso
4	validacion	(1 < 3) AND NOT Falso	Verdadero
5	pin_ingresado	Ingresar segundo PIN	0000
6	intentos	1 + 1	2
7	acceso_permitido	0000 == 1234	Falso
8	validacion	(2 < 3) AND NOT Falso	Verdadero
9	pin_ingresado	Ingresar tercer PIN	2222
10	intentos	2 + 1	3
11	acceso_permitido	2222 == 1234	Falso
12	validacion	(3 < 3) AND NOT Falso	Falso

3. Resultado final

Variable	Resultado
pin_ingresado	4321, 0000, 2222
intentos	3
acceso_permitido	Falso
validacion	Falso

4. Mensaje en pantalla

Mensaje	Resultado
Validación del PIN	PIN incorrecto. Acceso no permitido.
Intentos realizados	3
Acceso permitido	Falso
Estado de validación	Falso

6. Codificación en lenguaje C

```c
#include <stdio.h>
int main()
{
    int PIN_CORRECTO = 1234;
    int MAX_INTENTOS = 3;
    int pin_ingresado = 0;
    int intentos = 0;
    int acceso_permitido = 0;
    double minimo = 0.0;

    if ((intentos < MAX_INTENTOS) && (!acceso_permitido))
    {
        printf("Ingrese PIN: ");
        scanf("%d", &pin_ingresado);
        acceso_permitido = (pin_ingresado == PIN_CORRECTO);
        intentos = intentos + 1;
        if (acceso_permitido)
        {
            printf("Acceso permitido. Intento %d\n", intentos);
        }
        else
        {
            printf("PIN incorrecto. Intento %d de %d\n", intentos, MAX_INTENTOS);
        }
    }
    if ((intentos < MAX_INTENTOS) && (!acceso_permitido))
    {
        printf("Ingrese PIN: ");
        scanf("%d", &pin_ingresado);
        acceso_permitido = (pin_ingresado == PIN_CORRECTO);
        intentos = intentos + 1;
        if (acceso_permitido)
        {
            printf("Acceso permitido. Intento %d\n", intentos);
        }
        else
        {
            printf("PIN incorrecto. Intento %d de %d\n", intentos, MAX_INTENTOS);
        }
    }
    if ((intentos < MAX_INTENTOS) && (!acceso_permitido))
    {
        printf("Ingrese PIN: ");
        scanf("%d", &pin_ingresado);
        acceso_permitido = (pin_ingresado == PIN_CORRECTO);
        intentos = intentos + 1;
        if (acceso_permitido)
        {
            printf("Acceso permitido. Intento %d\n", intentos);
        }
        else
        {
            printf("PIN incorrecto. Intento %d de %d\n", intentos, MAX_INTENTOS);
        }
    }
    if (!acceso_permitido)
    {
        printf("Acceso denegado: intentos agotados.\n");
    }

    minimo = pin_ingresado;

    if (intentos < minimo)
    {
        minimo = intentos;
    }

    if (acceso_permitido < minimo)
    {
        minimo = acceso_permitido;
    }

    if (PIN_CORRECTO < minimo)
    {
        minimo = PIN_CORRECTO;
    }

    if (MAX_INTENTOS < minimo)
    {
        minimo = MAX_INTENTOS;
    }
    printf("Valor minimo de todas las variables: %.2f\n", minimo);
}
```

7. Salida del programa

Prueba 1

```
PS C:\msys64\EJEMPLOS C> ./ejercicio2.exe
Ingrese PIN: 1234
Acceso permitido. Intento 1
Valor minimo de todas las variables: 1.00
```

Prueba 2

```
Ingrese PIN: 4321
PIN incorrecto. Intento 1 de 3
Ingrese PIN: 0000
PIN incorrecto. Intento 2 de 3
Ingrese PIN: 2222
PIN incorrecto. Intento 3 de 3
Acceso denegado: intentos agotados.
Valor minimo de todas las variables: 0.00
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 3

1.Enunciado del problema

En un cine, diseñe un algoritmo que lea una fecha (dia, mes, año como enteros) y calcule el dia juliano (numero de dia del año, 1-366). Use constantes simbolicas para los dias por mes: DIA_ENE = 31, DIA_FEB = 28, DIA_FEB_BISIESTO = 29, DIA_MAR = 31, DIA_ABR = 30, etc. Evalúe la expresion logica para año bisiesto: (año % 4 == 0 AND (año % 100 != 0 OR año % 400 == 0)). Calcule el dia juliano sumando los dias de los meses anteriores mas el dia actual. Asigne el resultado a una variable entera. Muestre por pantalla la fecha y el dia juliano. Ademas, calcule el valor minimo de todas las variables utilizadas. Incluya una variable auxiliar para llevar un contador de operaciones.

2.Analisis del problema

1. Datos de entrada

- dia: día de la fecha ingresada por teclado (tipo entero).
- mes: mes de la fecha ingresada por teclado (tipo entero).
- anio: año de la fecha ingresada por teclado (tipo entero).

2. Proceso

1. Leer el día, mes y año.
2. Declarar las constantes simbólicas utilizando const int para representar los días de cada mes:
    - DIA_ENE = 31
    - DIA_FEB = 28
    - DIA_FEB_BISIESTO = 29
    - DIA_MAR = 31
    - DIA_ABR = 30
    - DIA_MAY = 31
    - DIA_JUN = 30
    - DIA_JUL = 31
    - DIA_AGO = 31
    - DIA_SEP = 30
    - DIA_OCT = 31

- DIA_NOV = 30

3. Evaluar si el año es bisiesto mediante la expresión lógica:

$$\left(anio\ \%\ 4 = = 0\right) \wedge \left(anio\ \%\ 100 \neq 0 \vee anio\ \%\ 400 = = 0\right)$$

Asignar el resultado a es_bisiesto.

4. Inicializar dia_juliano con el día ingresado:

$$dia_juliano = dia$$

5. Sumar los días de los meses anteriores al mes ingresado mediante condiciones if. Para febrero, utilizar 29 días si el año es bisiesto o 28 días si no lo es.
6. Incrementar contador_operaciones cada vez que se realiza una operación contabilizada en el código.
7. Mostrar la fecha ingresada, el día juliano y la cantidad de operaciones realizadas.
8. Inicializar valor_minimo con el valor de dia.
9. Comparar valor_minimo con las variables mes, anio, dia_juliano y contador_operaciones, asignando el menor valor encontrado.
10. Convertir el valor booleano de es_bisiesto a un valor numérico mediante:

$$valor_bisiesto_numerico = es_bisiesto?1:0$$

11. Comparar valor_bisiesto_numerico con valor_minimo para determinar el menor valor.
12. Mostrar los valores finales de las variables y el valor mínimo encontrado.

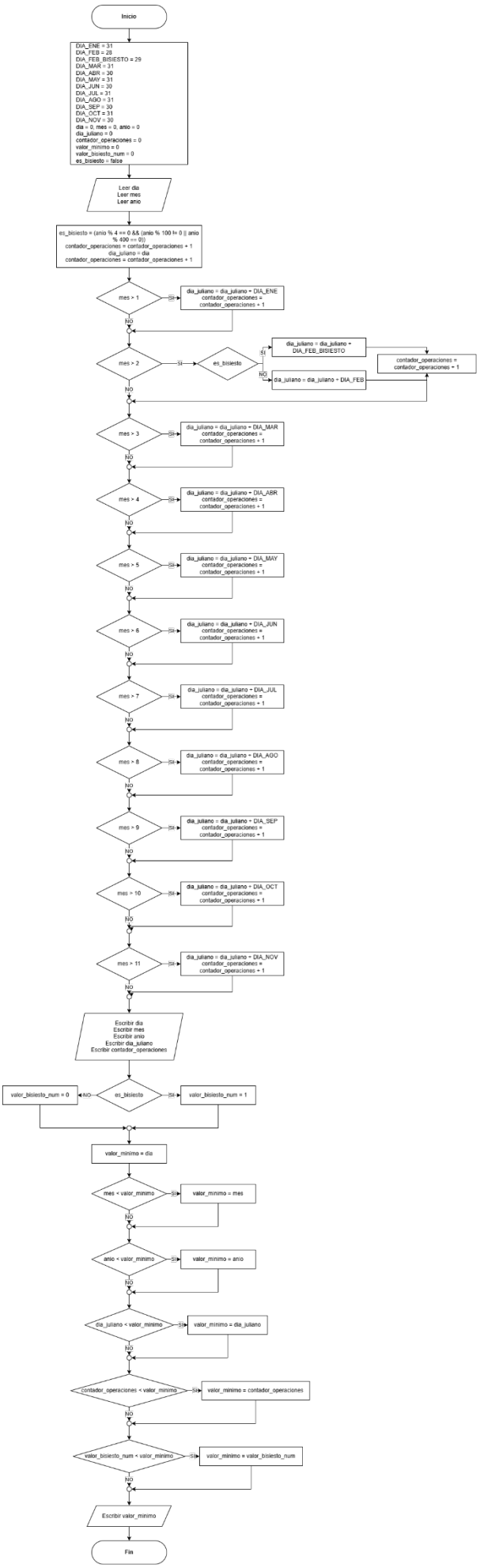
3. Datos de salida

- Fecha ingresada (día/mes/año).
- Día juliano correspondiente (número de día del año).
- Cantidad de operaciones realizadas.
- Valores finales de las variables utilizadas.
- Estado del año bisiesto (verdadero o falso).
- Valor mínimo de las variables numéricas evaluadas.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
dia	int	Almacena el día ingresado por teclado.
mes	int	Almacena el mes ingresado por teclado.
anio	int	Almacena el año ingresado por teclado.
es_bisiesto	bool	Almacena el resultado de la expresión lógica que determina si el año es bisiesto.
dia_juliano	int	Almacena el número del día dentro del año.
contador_operaciones	int	Registra la cantidad de operaciones realizadas.
valor_minimo	int	Almacena el menor valor encontrado entre las variables numéricas evaluadas.
valor_bisiesto_numerico	int	Almacena el valor numérico de es_bisiesto (1 verdadero, 0 falso).
DIA_ENE	const int	Representa los 31 días de enero.
DIA_FEB	const int	Representa los 28 días de febrero.
DIA_FEB_BISIESTO	const int	Representa los 29 días de febrero en año bisiesto.
DIA_MAR	const int	Representa los 31 días de marzo.
DIA_ABR	const int	Representa los 30 días de abril.
DIA_MAY	const int	Representa los 31 días de mayo.
DIA_JUN	const int	Representa los 30 días de junio.
DIA_JUL	const int	Representa los 31 días de julio.
DIA_AGO	const int	Representa los 31 días de agosto.
DIA_SEP	const int	Representa los 30 días de septiembre.
DIA_OCT	const int	Representa los 31 días de octubre.
DIA_NOV	const int	Representa los 30 días de noviembre.

3.Diagrama de flujo



4. Pseudocódigo

```
Algoritmo CalculoDiaJuliano

Definir DIA_ENE, DIA_FEB, DIA_FEB_BISIESTO, DIA_MAR, DIA_ABR, DIA_MAY Como Entero

Definir DIA_JUN, DIA_JUL, DIA_AGO, DIA_SEP, DIA_OCT, DIA_NOV Como Entero


DIA_ENE <- 31

DIA_FEB <- 28

DIA_FEB_BISIESTO <- 29

DIA_MAR <- 31

DIA_ABR <- 30

DIA_MAY <- 31

DIA_JUN <- 30

DIA_JUL <- 31

DIA_AGO <- 31

DIA_SEP <- 30

DIA_OCT <- 31

DIA_NOV <- 30

Definir dia, mes, anio, dia_juliano, contador_operaciones, valor_minimo, valor_bisiesto_num Como Entero

Definir es_bisiesto Como Logico

dia <- 0

mes <- 0

anio <- 0

dia_juliano <- 0

contador_operaciones <- 0

valor_minimo <- 0

valor_bisiesto_num <- 0
```

```
es_bisiesto <- Falso
Escribir "Ingrese dia (1-31):"
Leer dia
Escribir "Ingrese mes (1-12):"
Leer mes
Escribir "Ingrese anio:"
Leer anio
es_bisiesto <- (anio % 4 == 0 Y (anio % 100 <> 0 O anio % 400 == 0))
contador_operaciones <- contador_operaciones + 1
dia_juliano <- dia
contador_operaciones <- contador_operaciones + 1
Si mes > 1 Entonces
dia_juliano <- dia_juliano + DIA_ENE
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 2 Entonces
Si es_bisiesto Entonces
dia_juliano <- dia_juliano + DIA_FEB_BISIESTO
Sino
dia_juliano <- dia_juliano + DIA_FEB
FinSi
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 3 Entonces
dia_juliano <- dia_juliano + DIA_MAR
contador_operaciones <- contador_operaciones + 1
FinSi
```

```
Si mes > 4 Entonces
dia_juliano <- dia_juliano + DIA_ABR
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 5 Entonces
dia_juliano <- dia_juliano + DIA_MAY
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 6 Entonces
dia_juliano <- dia_juliano + DIA_JUN
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 7 Entonces
dia_juliano <- dia_juliano + DIA_JUL
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 8 Entonces
dia_juliano <- dia_juliano + DIA_AGO
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 9 Entonces
dia_juliano <- dia_juliano + DIA_SEP
contador_operaciones <- contador_operaciones + 1
FinSi
Si mes > 10 Entonces
dia_juliano <- dia_juliano + DIA_OCT
contador_operaciones <- contador_operaciones + 1
FinSi
```

```
Si mes > 11 Entonces
dia_juliano <- dia_juliano + DIA_NOV
contador_operaciones <- contador_operaciones + 1
FinSi
Escribir "Fecha ingresada: ", dia, "/", mes, "/", anio
Escribir "Dia juliano (1-366): ", dia_juliano
Escribir "Operaciones realizadas: ", contador_operaciones
Si es_bisiesto Entonces
valor_bisiesto_num <- 1
Sino
valor_bisiesto_num <- 0
FinSi
valor_minimo <- dia
Si mes < valor_minimo Entonces
valor_minimo <- mes
FinSi
Si anio < valor_minimo Entonces
valor_minimo <- anio
FinSi
Si dia_juliano < valor_minimo Entonces
valor_minimo <- dia_juliano
FinSi
Si contador_operaciones < valor_minimo Entonces
valor_minimo <- contador_operaciones
FinSi
Si valor_bisiesto_num < valor_minimo Entonces
valor_minimo <- valor_bisiesto_num
```

```
FinSi
Escribir "Valores finales de las variables:"
Escribir "- dia: ", dia
Escribir "- mes: ", mes
Escribir "- anio: ", anio
Escribir "- dia_juliano: ", dia_juliano
Escribir "- contador_operaciones: ", contador_operaciones
Escribir "- es_bisiesto (numerico): ", valor_bisiesto_num
Escribir "Valor minimo entre todas las variables: ", valor_minimo
FinAlgoritmo
```

5.1. Primera prueba de escritorio

Datos de entrada:

- dia = 15
- mes = 3
- anio = 2024

Desarrollo paso a paso:

1. Se determina si el año es bisiesto:

$$2024$$

Resultado: es_bisiesto = true.

2. Se inicializa el día juliano:

$$dia_juliano = 15$$

3. Como el mes es marzo, se suman los días de enero y febrero (29 días porque es año bisiesto):

$$dia_juliano = 15 + 31 + 29 = 75$$

4. Se calcula el contador de operaciones:

$$contador_operaciones = 4$$

5. Se busca el valor mínimo entre las variables:

$$minimo = 1$$

Tabla de prueba de escritorio:

N.º de paso	Variables y valores	Evaluación de condiciones	Salida en pantalla
1	dia=15, mes=3, anio=2024	No aplica	Ingrese dia (1-31): 15
2	mes=3	No aplica	Ingrese mes (1-12): 3
3	anio=2024	No aplica	Ingrese anio: 2024
4	es_bisiesto=true	Año bisiesto →Verdadero	—
5	dia_juliano=15, contador=2	No aplica	—

6	dia_juliano=46, contador=3	mes > 1 →Verdadero	—
7	dia_juliano=75, contador=4	mes > 2 →Verdadero; año bisiesto	—
8	dia_juliano=75	mes > 3 →Falso	—
9	dia_juliano=75	Condiciones mes > 4 hasta mes > 11 →Falsas	—
10	valor_minimo=1	Se compara el día, mes, año, día juliano, contador y valor numérico de bisiesto	—
11	dia_juliano=75, contador=4	No aplica	Fecha ingresada: 15/03/2024
12	valor_minimo=1	No aplica	Valor minimo de todas las variables utilizadas: 1

Operaciones realizadas:

- dia_juliano = 15
- dia_juliano = 15 + 31 = 46
- dia_juliano = 46 + 29 = 75
- contador_operaciones = 4
- valor_bisiesto_numerico = 1
- valor_minimo = 1

5.2. Segunda prueba de escritorio

Datos de entrada:

- dia = 25
- mes = 4
- anio = 2009

Desarrollo paso a paso:

1. Se determina si el año es bisiesto:

$$(2009$$

Resultado: es_bisiesto = false.

2. Se inicializa el día juliano:

$$dia_juliano = 25$$

3. Como el mes es abril, se suman los días de enero, febrero y marzo:

$$dia_juliano = 25 + 31 + 28 + 31 = 115$$

4. Se calcula el contador de operaciones:

$$contador_operaciones = 5$$

$$Sebuscaelvalormínimoentrelasvariables:\[minimo = 0\]$$

Tabla de prueba de escritorio:

N.º de paso	Variables y valores	Evaluación de condiciones	Salida en pantalla
1	dia=25, mes=4, anio=2009	No aplica	Ingrese dia (1-31): 25
2	mes=4	No aplica	Ingrese mes (1-12): 4
3	anio=2009	No aplica	Ingrese anio: 2009
4	es_bisiesto=false	Año bisiesto →Falso	—
5	dia_juliano=25, contador=2	No aplica	—
6	dia_juliano=56, contador=3	mes > 1 →Verdadero	—

7	dia_juliano=84, contador=4	mes > 2 →Verdadero; año no bisiesto	—
8	dia_juliano=115, contador=5	mes > 3 →Verdadero	—
9	dia_juliano=115	mes > 4 → Falso; condiciones restantes también falsas	—
10	valor_minimo=0	Se compara el día, mes, año, día juliano, contador y valor numérico de bisiesto	—
11	dia_juliano=115, contador=5	No aplica	Fecha ingresada: 25/04/2009
12	valor_minimo=0	No aplica	Valor minimo de todas las variables utilizadas: 0

Operaciones realizadas:

- dia_juliano = 25
- dia_juliano = 25 + 31 = 56
- dia_juliano = 56 + 28 = 84
- dia_juliano = 84 + 31 = 115
- contador_operaciones = 5
- valor_bisiesto_numerico = 0
- valor_minimo = 0

6. Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    const int DIA_ENE = 31;
    const int DIA_FEB = 28;
    const int DIA_FEB_BISIESTO = 29;
    const int DIA_MAR = 31;
    const int DIA_ABR = 30;
    const int DIA_MAY = 31;
    const int DIA_JUN = 30;
    const int DIA_JUL = 31;
    const int DIA_AGO = 31;
    const int DIA_SEP = 30;
    const int DIA_OCT = 31;
    const int DIA_NOV = 30;

    int dia = 0;
    int mes = 0;
    int anio = 0;
    bool es_bisiesto = false;
    int dia_juliano = 0;
    int contador_operaciones = 0;
    int valor_minimo = 0;

    printf("Ingrese dia (1-31): ");
    scanf("%d", &dia);

    printf("Ingrese mes (1-12): ");
    scanf("%d", &mes);

    printf("Ingrese anio: ");
    scanf("%d", &anio);

    es_bisiesto = (anio % 4 == 0 && (anio % 100 != 0 || anio % 400 == 0));
    contador_operaciones = contador_operaciones + 1;

    dia_juliano = dia;
    contador_operaciones = contador_operaciones + 1;

    if (mes > 1) {
        dia_juliano = dia_juliano + DIA_ENE;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 2) {
        if (es_bisiesto) {
            dia_juliano = dia_juliano + DIA_FEB_BISIESTO;
        } else {
            dia_juliano = dia_juliano + DIA_FEB;
        }
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 3) {
        dia_juliano = dia_juliano + DIA_MAR;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 4) {
        dia_juliano = dia_juliano + DIA_ABR;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 5) {
        dia_juliano = dia_juliano + DIA_MAY;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 6) {
        dia_juliano = dia_juliano + DIA_JUN;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 7) {
        dia_juliano = dia_juliano + DIA_JUL;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 8) {
        dia_juliano = dia_juliano + DIA_AGO;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 9) {
        dia_juliano = dia_juliano + DIA_SEP;
        contador_operaciones = contador_operaciones + 1;
```

```c
    }
    if (mes > 10) {
        dia_juliano = dia_juliano + DIA_OCT;
        contador_operaciones = contador_operaciones + 1;
    }
    if (mes > 11) {
        dia_juliano = dia_juliano + DIA_NOV;
        contador_operaciones = contador_operaciones + 1;
    }


    printf("Fecha ingresada: %02d/%02d/%04d\n", dia, mes, anio);
    printf("Dia juliano (1-366): %d\n", dia_juliano);
    printf("Operaciones realizadas: %d\n", contador_operaciones);


    valor_minimo = dia;

    if (mes < valor_minimo) {
        valor_minimo = mes;
    }
    if (anio < valor_minimo) {
        valor_minimo = anio;
    }
    if (dia_juliano < valor_minimo) {
        valor_minimo = dia_juliano;
    }
    if (contador_operaciones < valor_minimo) {
        valor_minimo = contador_operaciones;
    }

    int valor_bisiesto_numerico = es_bisiesto ? 1 : 0;
    if (valor_bisiesto_numerico < valor_minimo) {
        valor_minimo = valor_bisiesto_numerico;
    }

    printf("\nValores finales de las variables:\n");
    printf("- dia = %d\n", dia);
    printf("- mes = %d\n", mes);
    printf("- anio = %d\n", anio);
    printf("- dia_juliano = %d\n", dia_juliano);
    printf("- contador_operaciones = %d\n", contador_operaciones);
    printf("- es_bisiesto = %d (%s)\n", valor_bisiesto_numerico, es_bisiesto ? "true" : "false");
    printf("\nValor minimo de todas las variables utilizadas: %d\n", valor_minimo);

}

```

7. Salida del programa

Prueba 1

```
Ingrese dia (1-31): 15
Ingrese mes (1-12): 3
Ingrese anio: 24
Fecha ingresada: 15/03/0024
Dia juliano (1-366): 75
Operaciones realizadas: 4

Valores finales de las variables:
- dia = 15
- mes = 3
- anio = 24
- dia_juliano = 75
- contador_operaciones = 4
- es_bisiesto = 1 (true)

Valor minimo de todas las variables utilizadas: 1
```

Prueba 2

```
Ingrese dia (1-31): 25
Ingrese mes (1-12): 4
Ingrese anio: 2009
Fecha ingresada: 25/04/2009
Dia juliano (1-366): 115
Operaciones realizadas: 5

Valores finales de las variables:
- dia = 25
- mes = 4
- anio = 2009
- dia_juliano = 115
- contador_operaciones = 5
- es_bisiesto = 0 (false)

Valor minimo de todas las variables utilizadas: 0
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 5

1.Enunciado del problema

En una jugueteria, diseñe un algoritmo que lea la cantidad de articulos (entero) y el precio unitario (real). Use constantes simbolicas para descuentos por volumen: DESC_10 = 0.05, DESC_50 = 0.10, DESC_100 = 0.15, UMBRAL_10 = 10, UMBRAL_50 = 50, UMBRAL_100 = 100, IVA = 0.15. Use expresiones logicas relacionales para seleccionar el descuento. Calcule subtotal, descuento, total con IVA. Asigne los resultados a variables reales. Muestre por pantalla una factura detallada. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de que las expresiones logicas esten bien parentizadas.

2.Analisis del problema

1. Datos de entrada

- cantidad: cantidad de artículos que se van a comprar (tipo entero).
- precio_unitario: precio de cada artículo (tipo real).

2. Proceso

1. Leer la cantidad de artículos y el precio unitario.
2. Declarar las constantes simbólicas:
    - DESC_10 = 0.05 (5 % de descuento).
    - DESC_50 = 0.10 (10 % de descuento).
    - DESC_100 = 0.15 (15 % de descuento).
    - UMBRAL_10 = 10.
    - UMBRAL_50 = 50.
    - UMBRAL_100 = 100.
    - IVA = 0.15 (15 % de impuesto).
3. Validar que la cantidad y el precio unitario sean mayores que cero mediante la expresión lógica:

$$\left(cantidad \leq 0\right) \vee \left(precio_unitario \leq 0\right)$$

Si se cumple, mostrar un mensaje de error. De lo contrario, continuar con los cálculos.

4. Calcular el subtotal: $subtotal = cantidad \times precio_unitario$

5. Seleccionar el descuento según la cantidad de artículos:
    - Si (cantidad >= UMBRAL_10) && (cantidad < UMBRAL_50), aplicar DESC_10.
    - Si (cantidad >= UMBRAL_50) && (cantidad < UMBRAL_100), aplicar DESC_50.
    - Si (cantidad >= UMBRAL_100), aplicar DESC_100.
    - Si no cumple ninguna condición, asignar un descuento de 0.0.
6. Calcular el valor monetario del descuento:

$$valor_descuento = subtotal \times descuento_aplicado$$

7. Calcular el subtotal después de aplicar el descuento:

$$subtotal_descuento = subtotal - valor_descuento$$

8. Calcular el valor del IVA:

$$valor_iva = subtotal_descuento \times IVA$$

9. Calcular el total final:

$$total = subtotal_descuento + valor_iva$$

10. Mostrar la factura detallada con la cantidad, precio unitario, subtotal, descuento aplicado, valor del descuento, subtotal con descuento, IVA y total a pagar.
11. Inicializar minimo con el valor de cantidad.
12. Comparar minimo con las variables precio_unitario, subtotal, descuento_aplicado, valor_descuento, subtotal_descuento, valor_iva y total, asignando el menor valor encontrado.
13. Mostrar el valor mínimo numérico obtenido.

3. Datos de salida

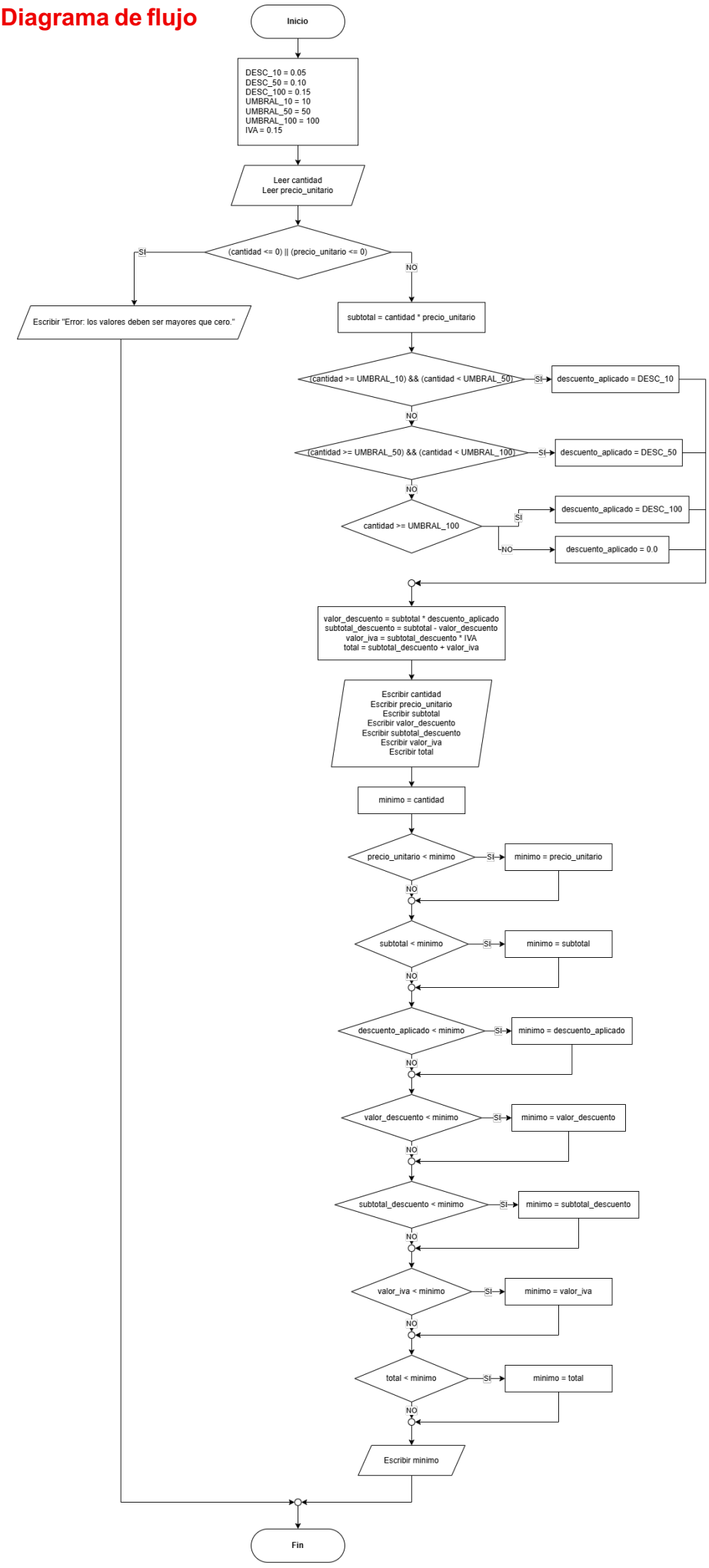
- Mensaje de error si la cantidad o el precio unitario son menores o iguales a cero.
- Cantidad de artículos.
- Precio unitario.
- Subtotal de la compra.
- Porcentaje de descuento aplicado.
- Valor monetario del descuento.

- Subtotal con descuento.
- Valor del IVA.
- Total final a pagar.
- Valor mínimo numérico encontrado.

4. Variables a utilizar

Variable	**Tipo de dato**	**Descripción**
cantidad	int	Almacena la cantidad de artículos comprados.
precio_unitario	float	Almacena el precio de cada artículo.
subtotal	float	Almacena el resultado de multiplicar la cantidad por el precio unitario.
descuento_aplicado	float	Almacena el porcentaje de descuento seleccionado.
valor_descuento	float	Almacena el valor monetario del descuento.
subtotal_descuento	float	Almacena el subtotal después de restar el descuento.
valor_iva	float	Almacena el valor correspondiente al IVA.
total	float	Almacena el total final que debe pagar el cliente.
minimo	float	Almacena el menor valor numérico encontrado entre las variables comparadas.
DESC_10	const	Representa el descuento del 5 %.
DESC_50	const	Representa el descuento del 10 %.
DESC_100	const	Representa el descuento del 15 %.
UMBRAL_10	const	Representa el límite de 10 artículos.
UMBRAL_50	const	Representa el límite de 50 artículos.
UMBRAL_100	const	Representa el límite de 100 artículos.
IVA	const	Representa el impuesto del 15 %.

3.Diagrama de flujo



4.Pseucodigo

```
Algoritmo FacturacionConDescuento
    Definir DESC_10, DESC_50, DESC_100, IVA Como Real
    Definir UMBRAL_10, UMBRAL_50, UMBRAL_100, cantidad Como Entero
    Definir precio_unitario, subtotal, descuento_aplicado, valor_descuento Como Real
    Definir subtotal_descuento, valor_iva, total, minimo Como Real

    DESC_10 <- 0.05
    DESC_50 <- 0.10
    DESC_100 <- 0.15
    UMBRAL_10 <- 10
    UMBRAL_50 <- 50
    UMBRAL_100 <- 100
    IVA <- 0.15

    Escribir "Ingrese la cantidad de articulos: "
    Leer cantidad

    Escribir "Ingrese el precio unitario: "
    Leer precio_unitario

    Si (cantidad <= 0) O (precio_unitario <= 0) Entonces
        Escribir "Error: los valores deben ser mayores que cero."
    Sino
        subtotal <- cantidad * precio_unitario

        Si (cantidad >= UMBRAL_10) Y (cantidad < UMBRAL_50) Entonces
            descuento_aplicado <- DESC_10
        Sino
            Si (cantidad >= UMBRAL_50) Y (cantidad < UMBRAL_100) Entonces
                descuento_aplicado <- DESC_50
            Sino
                Si cantidad >= UMBRAL_100 Entonces
                    descuento_aplicado <- DESC_100
                Sino
                    descuento_aplicado <- 0.0
                FinSi
            FinSi
        FinSi

        valor_descuento <- subtotal * descuento_aplicado
        subtotal_descuento <- subtotal - valor_descuento
        valor_iva <- subtotal_descuento * IVA
        total <- subtotal_descuento + valor_iva

        Escribir "========== FACTURA =========="
        Escribir "Cantidad de articulos: ", cantidad
        Escribir "Precio unitario:       $", precio_unitario
        Escribir "Subtotal:              $", subtotal
        Escribir "Descuento aplicado:    ", descuento_aplicado * 100, "%"
        Escribir "Valor del descuento:   $", valor_descuento
        Escribir "Subtotal con descuento:$", subtotal_descuento
        Escribir "IVA (15%):             $", valor_iva
        Escribir "-----------------------------"
        Escribir "TOTAL A PAGAR:         $", total
        Escribir "============================="

        minimo <- cantidad

        Si precio_unitario < minimo Entonces
            minimo <- precio_unitario
        FinSi

        Si subtotal < minimo Entonces
            minimo <- subtotal
        FinSi

        Si descuento_aplicado < minimo Entonces
            minimo <- descuento_aplicado
        FinSi

        Si valor_descuento < minimo Entonces
            minimo <- valor_descuento
        FinSi

        Si subtotal_descuento < minimo Entonces
            minimo <- subtotal_descuento
        FinSi

        Si valor_iva < minimo Entonces
            minimo <- valor_iva
        FinSi

        Si total < minimo Entonces
            minimo <- total
        FinSi

        Escribir "Valor minimo numerico: ", minimo
    FinSi
FinAlgoritmo
```

5.1 Prueba de escritorio

Datos de entrada: cantidad = 20, precio_unitario = 10

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cantidad=20, precio=10	Validación: Verdadero (datos válidos)	Ingreso de datos
2	subtotal=200	—	200
3	descuento=0.05	20 >= 10 && 20 < 50 → Verdadero	0.05
4	valor_descuento=10, subtotal_descuento=190	—	10 190
5	valor_iva=28.50, total=218.50	—	218.50
6	minimo=0.05	Comparación de valores →mínimo 0.05	Factura y valor mínimo: 0.05

2. Segunda prueba

Datos de entrada: cantidad = 100, precio_unitario = 5

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cantidad=100, precio=5	Validación: Verdadero (datos válidos)	Ingreso de datos
2	subtotal=500	—	500
3	descuento=0.15	100 >= 100 → Verdadero	0.15
4	valor_descuento=75, subtotal_descuento=425	—	75 425
5	valor_iva=63.75, total=488.75	—	488.75
6	minimo=0.15	Comparación de valores →mínimo 0.15	Factura y valor mínimo: 0.15

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define DESC_10 0.05
#define DESC_50 0.10
#define DESC_100 0.15
#define UMBRAL_10 10
#define UMBRAL_50 50
#define UMBRAL_100 100
#define IVA 0.15

int main()
{
    int cantidad;
    float precio_unitario;
    float subtotal;
    float descuento_aplicado;
    float valor_descuento;
    float subtotal_descuento;
    float valor_iva;
    float total;
    float minimo;

    printf("Ingrese la cantidad de articulos: ");
    scanf("%d", &cantidad);

    printf("Ingrese el precio unitario: ");
    scanf("%f", &precio_unitario);

    // Validacion
    if ((cantidad <= 0) || (precio_unitario <= 0))
    {
        printf("Error: los valores deben ser mayores que cero.\n");
    }
    else
    {
        subtotal = cantidad * precio_unitario;

        if ((cantidad >= UMBRAL_10) && (cantidad < UMBRAL_50))
        {
            descuento_aplicado = DESC_10;
        }
        else if ((cantidad >= UMBRAL_50) && (cantidad < UMBRAL_100))
        {
            descuento_aplicado = DESC_50;
        }
        else if (cantidad >= UMBRAL_100)
        {
            descuento_aplicado = DESC_100;
        }
        else
        {
            descuento_aplicado = 0.0;
        }

        valor_descuento = subtotal * descuento_aplicado;

        subtotal_descuento = subtotal - valor_descuento;

        valor_iva = subtotal_descuento * IVA;

        total = subtotal_descuento + valor_iva;

        printf("\n========== FACTURA ==========\n");
        printf("Cantidad de articulos: %d\n", cantidad);
        printf("Precio unitario:       $%.2f\n", precio_unitario);
        printf("Subtotal:              $%.2f\n", subtotal);
        printf("Descuento aplicado:    %.0f%%\n", descuento_aplicado * 100);
        printf("Valor del descuento:   $%.2f\n", valor_descuento);
        printf("Subtotal con descuento:$%.2f\n", subtotal_descuento);
        printf("IVA (15%%):             $%.2f\n", valor_iva);
        printf("-----------------------------\n");
        printf("TOTAL A PAGAR:         $%.2f\n", total);
        printf("=============================\n");

        minimo = cantidad;

        if (precio_unitario < minimo)
            minimo = precio_unitario;

        if (subtotal < minimo)
            minimo = subtotal;

        if (descuento_aplicado < minimo)
            minimo = descuento_aplicado;

        if (valor_descuento < minimo)
            minimo = valor_descuento;

        if (subtotal_descuento < minimo)
            minimo = subtotal_descuento;

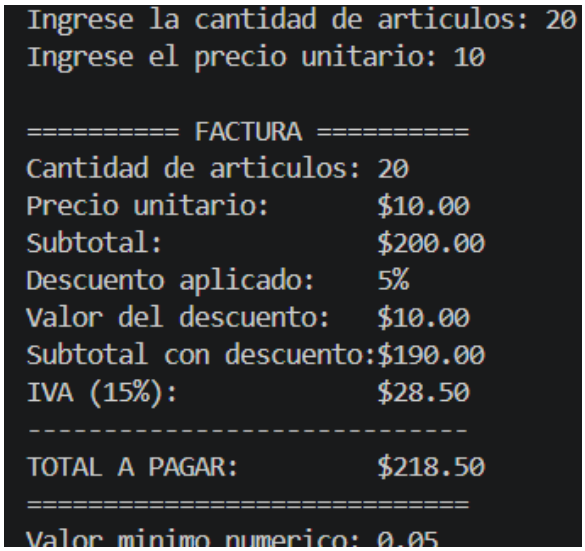
        if (valor_iva < minimo)
            minimo = valor_iva;

        if (total < minimo)
            minimo = total;

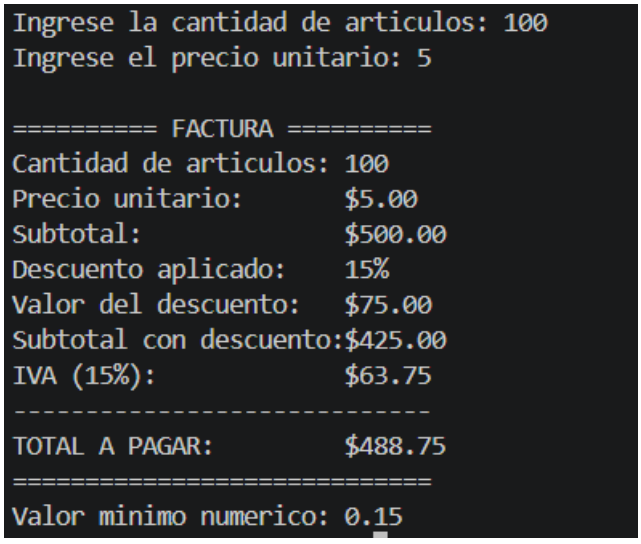
        printf("Valor minimo numerico: %.2f\n", minimo);
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 6

1.Enunciado del problema

En un cine, construya un algoritmo que lea un volumen en galones (real). Use una constante simbolica GALON_LITRO = 3.78541. Calcule la equivalencia en litros con la expresion aritmetica (galones * GALON_LITRO) y asignela a litros. Evalúe la expresion relacional (litros > 0). Muestre por pantalla 'X galones equivalen a Y litros'. Ademas del cálculo principal, calcule el producto del resultado por PI. Agregue un mensaje indicando el tipo de operación realizada.

2.ANÁLISIS DEL PROBLEMA

1. Datos de entrada

Son los datos que el usuario debe ingresar por teclado para realizar el proceso.

Dato	Tipo de dato	Descripción
galones	Real (float)	Volumen ingresado por el usuario en galones.

2. Proceso

1. Declarar la constante simbólica GALON_LITRO = 3.78541.
2. Declarar la constante PI = 3.14159.
3. Leer el volumen en galones ingresado por el usuario.
4. Calcular la equivalencia en litros mediante la expresión aritmética:

$$litros = galones \times GALON_LITRO$$

5. Evaluar la expresión relacional:

$$litros > 0$$

6. Si el resultado es verdadero, calcular el producto de los litros por PI:

$$producto = litros \times PI$$

7. Mostrar la equivalencia de galones a litros.
8. Mostrar el resultado del producto por PI.
9. Mostrar un mensaje indicando el tipo de operación realizada: multiplicación.
10. Si la condición es falsa, mostrar un mensaje indicando que el volumen no es válido.

3. Datos de salida

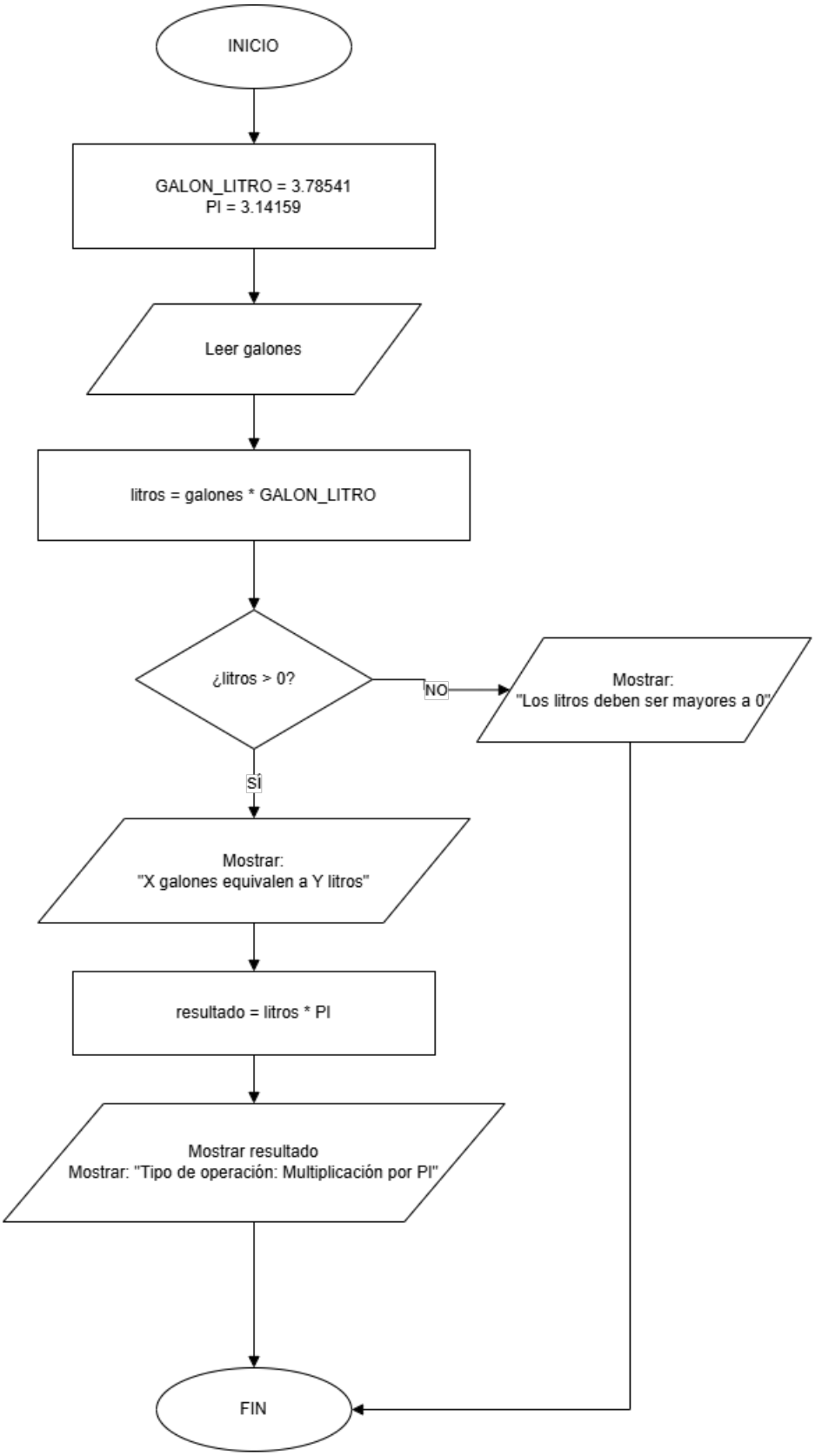
Son los resultados y mensajes que se mostrarán en pantalla.

Dato de salida	Descripción
litros	Equivalencia del volumen ingresado en litros.
validacion	Resultado de comprobar si los litros son mayores que cero (verdadero o falso).
producto	Resultado de multiplicar los litros por PI.
Tipo de operación	Mensaje que indica que la operación realizada fue una multiplicación.
Mensaje de validación	Indica si el volumen ingresado es válido o no.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
galones	float	Almacena el volumen ingresado en galones.
litros	float	Almacena la equivalencia del volumen en litros.
producto	float	Almacena el resultado de multiplicar los litros por PI.
validacion	int	Almacena el resultado de la expresión relacional (1 = verdadero, 0 = falso).
GALON_LITRO	Constante real	Factor de conversión equivalente a 3.78541.
PI	Constante real	Valor aproximado de PI, equivalente a 3.14159.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Galones_Litros

    Definir GALON_LITRO, PI Como Real
    Definir litros, galones, resultado Como Real

    GALON_LITRO <- 3.78541
    PI <- 3.14159

    litros <- 0
    galones <- 0
    resultado <- 0

    Escribir "Ingrese el valor del volumen en galones:"
    Leer galones

    litros <- galones * GALON_LITRO

    Si litros > 0 Entonces

        Escribir galones, " galones equivalen a ", litros, " litros"

        resultado <- litros * PI

        Escribir "El resultado de los litros por PI es: ", resultado

        Escribir "Tipo de operacion realizada: Multiplicacion por PI."

    SiNo

        Escribir "Los litros deben ser mayores a 0"

    FinSi

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1: Volumen válido

Dato de entrada: 5 galones.

Cálculos:

- Conversión de galones a litros:

$$litros = 5 \times 3.78541 = 18.92705$$

- Evaluación de la condición:

$$litros > 0 \Rightarrow 18.92705 > 0 = Verdadero$$

- Multiplicación por PI:

$$resultado = 18.92705 \times 3.14159$$

$$resultado \approx 59.4603$$

Tabla de prueba de escritorio:

Paso	galones	litros	PI	resultado	Condición litros > 0	Salida en pantalla
1	0.00	0.00	3.14159	0.00	—	Ingrese el valor del volumen en galones:
2	5.00	0.00	3.14159	0.00	—	Se ingresa 5
3	5.00	18.9271	3.14159	0.00	—	Se calcula la conversión
4	5.00	18.9271	3.14159	0.00	Verdadero	Entra al if
5	5.00	18.9271	3.14159	59.4603	Verdadero	Se calcula litros por PI
6	5.00	18.9271	3.14159	59.4603	—	Muestra los resultados y tipo de operación

5.2. Prueba de escritorio N.º 2: Volumen no válido

Dato de entrada: -3 galones.

Cálculos:

- Conversión de galones a litros:

$$litros = -3 \times 3.78541 = -11.35623$$

- Evaluación de la condición:

$$litros > 0 \Rightarrow -11.35623 > 0 = Falso$$

Como la condición es falsa, el programa ejecuta el bloque else. Por lo tanto, no realiza la multiplicación por PI y la variable resultado conserva su valor inicial de 0.

Tabla de prueba de escritorio:

Paso	**galones**	**litros**	**PI**	**resultado**	**Condición litros > 0**	**Salida en pantalla**
1	0.00	0.00	3.14159	0.00	—	Ingrese el valor del volumen en galones:
2	-3.00	0.00	3.14159	0.00	—	Se ingresa -3
3	-3.00	-11.3562	3.14159	0.00	—	Se calcula la conversión
4	-3.00	-11.3562	3.14159	0.00	Falso	Entra al else
5	-3.00	-11.3562	3.14159	0.00	Falso	Muestra el mensaje de volumen no válido

6. Codificación en lenguaje C

```c

#include <stdio.h>
#include <math.h>

int main()
{
    float GALON_LITRO = 3.78541;
    float PI = 3.14159;
    float litros = 0.0, galones = 0.0, resultado = 0.0;

    printf("Ingrese el valor del volumen en galones: ");
    scanf("%f", &galones);

    litros = galones * GALON_LITRO;

    if (litros > 0)
    {
        printf("%.2f galones equivalen a %.2f litros\n", galones, litros);

        resultado = litros * PI;

        printf("El resultado de los litros por PI es: %.2f\n", resultado);

        printf("Tipo de operacion realizada: Multiplicacion por PI.\n");
    }
    else
    {
        printf("Los litros deben ser mayores a 0\n");
    }
}
```

7. Salida del programa

Prueba 1

```
Ingrese el valor del volumen en galones: 5
5.00 galones equivalen a 18.93 litros
El resultado de los litros por PI es: 59.46
Tipo de operacion realizada: Multiplicacion por PI.
```

Prueba 2

```
Ingrese el valor del volumen en galones: -3
Los litros deben ser mayores a 0
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 7

1.Enunciado del problema

En una empresa de transporte, diseñe un algoritmo que calcule el area de un circulo. Declare una constante simbolica PI = 3.14159 y una variable real radio. Lea el radio por teclado. Calcule el area con la expresion aritmetica (PI * radio * radio) y asignela a la variable area. Muestre por pantalla el resultado con un mensaje descriptivo. Justifique por que se usa constante simbolica y no un literal para PI. Ademas del calculo principal, calcule el resto de una division adicional. Incluya un mensaje de confirmacion al finalizar la operacion.

2.Analisis del problema

1. Datos de entrada

- radio: valor real que representa el radio del círculo, ingresado por teclado.
- numero1: primer número entero ingresado por teclado para realizar la división.
- numero2: segundo número entero ingresado por teclado para realizar la división.

2. Proceso

1. Declarar la constante simbólica PI = 3.14159.
2. Leer el valor del radio del círculo.
3. Calcular el área del círculo mediante la siguiente expresión aritmética:

$$area = PI \times radio \times radio$$

4. Mostrar por pantalla el área calculada con dos decimales.
5. Leer el primer número entero (numero1).
6. Leer el segundo número entero (numero2).
7. Calcular el resto de la división entre los dos números mediante el operador módulo (%):

$$resto = numero1 \ \% \ numero2$$

8. Mostrar por pantalla el resto de la división.
9. Mostrar un mensaje de confirmación indicando que la operación finalizó correctamente.

   Justificación del uso de la constante simbólica PI:

   Se utiliza una constante simbólica porque permite representar el valor de PI (3.14159) mediante un identificador, facilita la comprensión del código y evita escribir repetidamente el valor numérico. Además, su valor permanece fijo durante la ejecución del programa.

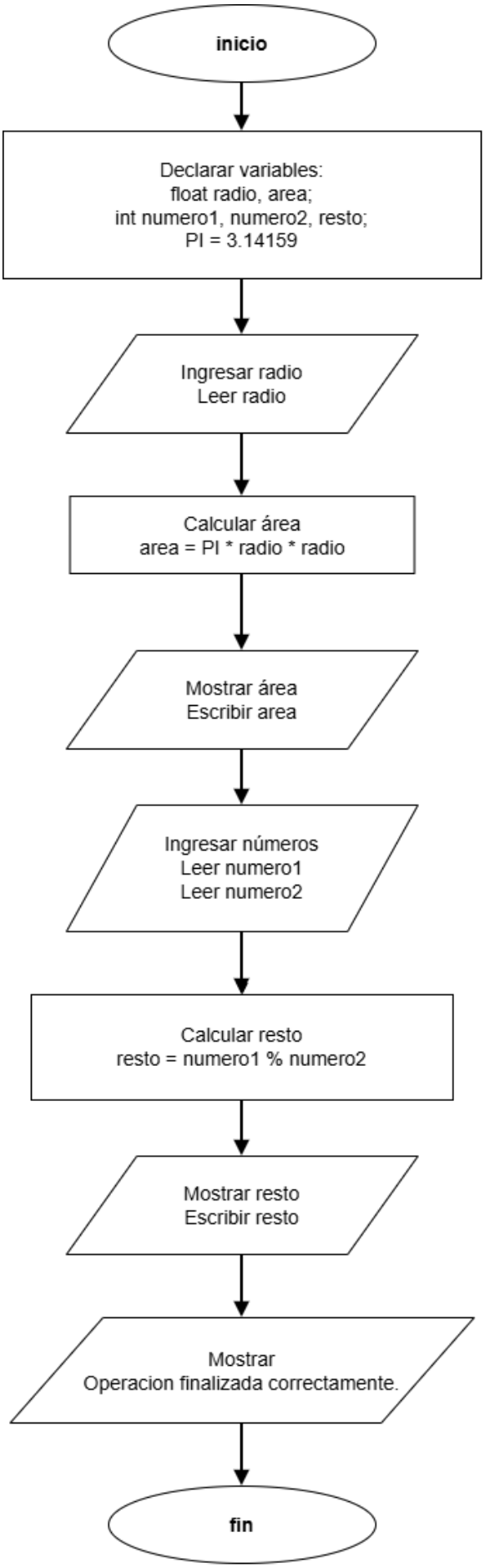
3. Datos de salida

- Área del círculo calculada, mostrada con dos decimales.
- Resto obtenido de la división entre los dos números enteros.
- Mensaje de confirmación: "Operación finalizada correctamente".

4. Variables a utilizar

Variable	Tipo de dato	Descripción
PI	Constante real (#define)	Almacena el valor fijo de 3.14159.
radio	Real (float)	Almacena el radio del círculo.
area	Real (float)	Almacena el resultado del área del círculo.
numero1	Entero (int)	Almacena el primer número de la división.
numero2	Entero (int)	Almacena el segundo número de la división.
resto	Entero (int)	Almacena el resto de la división.

3.Diagrama de flujo



4.Pseucodigo

```
Area_Circulo_y_Resto

    Definir radio, area Como Real
    Definir numero1, numero2, resto Como Entero

    Constante PI = 3.14159

    Escribir "Ingrese el radio del circulo: "
    Leer radio

    area = PI * radio * radio

    Escribir "El area del circulo es: ", area

    Escribir "Ingrese el primer numero: "
    Leer numero1

    Escribir "Ingrese el segundo numero: "
    Leer numero2

    resto = numero1 MOD numero2

    Escribir "El resto de la division es: ", resto

    Escribir "Operacion finalizada correctamente."

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1

Datos de entrada válidos

Datos de entrada:

- Radio = 5
- Número 1 = 10
- Número 2 = 3

Cálculos:

1. Cálculo del área del círculo:

$$area = 3.14159 \times 5 \times 5$$

$$area = 78.54$$

2. Cálculo del residuo:

$$resto = 10 \,\%\, 3 = 1$$

Tabla de prueba de escritorio:

Paso	radio	area	numero1	numero2	resto	Evaluación	Salida en pantalla
1	—	—	—	—	—	—	Ingrese el radio del círculo:
2	5	—	—	—	—	—	Se ingresa 5
3	5	78.54	—	—	—	—	El área del círculo es: 78.54
4	5	78.54	—	—	—	—	Ingrese el primer número:
5	5	78.54	10	—	—	—	Se ingresa 10
6	5	78.54	10	—	—	—	Ingrese el segundo número:
7	5	78.54	10	3	—	—	Se ingresa 3
8	5	78.54	10	3	1	—	El resto de la división es: 1
9	5	78.54	10	3	1	—	Operación finalizada correctamente.

5.2. Prueba de escritorio N.º 2

Segundo conjunto de datos

Datos de entrada:

- Radio = 3
- Número 1 = 20
- Número 2 = 4

Cálculos:

1. Cálculo del área del círculo:

$$area = 3.14159 \times 3 \times 3$$

$$area = 28.27$$

2. Cálculo del residuo:

$$resto = 20 \ \% \ 4 = 0$$

Tabla de prueba de escritorio:

Paso	radio	area	numero1	numero2	resto	Evaluación	Salida en pantalla
1	—	—	—	—	—	—	Ingrese el radio del círculo:
2	3	—	—	—	—	—	Se ingresa 3
3	3	28.27	—	—	—	—	El área del círculo es: 28.27
4	3	28.27	—	—	—	—	Ingrese el primer número:
5	3	28.27	20	—	—	—	Se ingresa 20
6	3	28.27	20	—	—	—	Ingrese el segundo número:
7	3	28.27	20	4	—	—	Se ingresa 4
8	3	28.27	20	4	0	—	El resto de la división es: 0
9	3	28.27	20	4	0	—	Operación finalizada correctamente.

6. Codificación en lenguaje C

```c

#include <stdio.h>

#define PI 3.14159

int main()
{
    float radio;
    float area;
    int numero1;
    int numero2;
    int resto;

    printf("Ingrese el radio del circulo: ");
    scanf("%f", &radio);

    area = PI * radio * radio;

    printf("El area del circulo es: %.2f\n", area);

    printf("\nIngrese el primer numero: ");
    scanf("%d", &numero1);

    printf("Ingrese el segundo numero: ");
    scanf("%d", &numero2);

    resto = numero1 % numero2;

    printf("El resto de la division es: %d\n", resto);

    printf("\nOperacion finalizada correctamente.\n");
}

```

7. Salida del programa

Prueba 1

```
Ingrese el radio del circulo: 5
El area del circulo es: 78.54

Ingrese el primer numero: 10
Ingrese el segundo numero: 3
El resto de la division es: 1

Operacion finalizada correctamente.
```

Prueba 2

```
Ingrese el radio del circulo: 3
El area del circulo es: 28.27

Ingrese el primer numero: 20
Ingrese el segundo numero: 4
El resto de la division es: 0
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 8

1.Enunciado del problema

En una zapateria, cree un algoritmo que lea la base (real) y la altura (real) de un triangulo. Use la constante literal 2 en el denominador. Calcule el area con la expresion aritmetica ((base * altura) / 2) y asignela a area. Evalúe la expresion logica (base > 0 AND altura > 0). Muestre por pantalla el area calculada. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue validaciones para que no haya resultados negativos.

2.Analisis del problema

1. Datos de entrada

- yardas: distancia expresada en yardas, ingresada por teclado (tipo real).

2. Proceso

1. Leer la distancia en yardas.
2. Calcular la equivalencia en pies utilizando la constante literal 3:

$$\mathbf{pies} = \mathbf{yardas} * 3$$

3. Evaluar la expresión relacional:

$$\mathbf{pies} > 0$$

4. Calcular el cociente del resultado entre el factor 1.5:

$$\mathbf{cociente} = \mathbf{pies} \,/\, 1.5$$

5. Mostrar la conversión de yardas a pies.
6. Presentar las conclusiones del algoritmo.

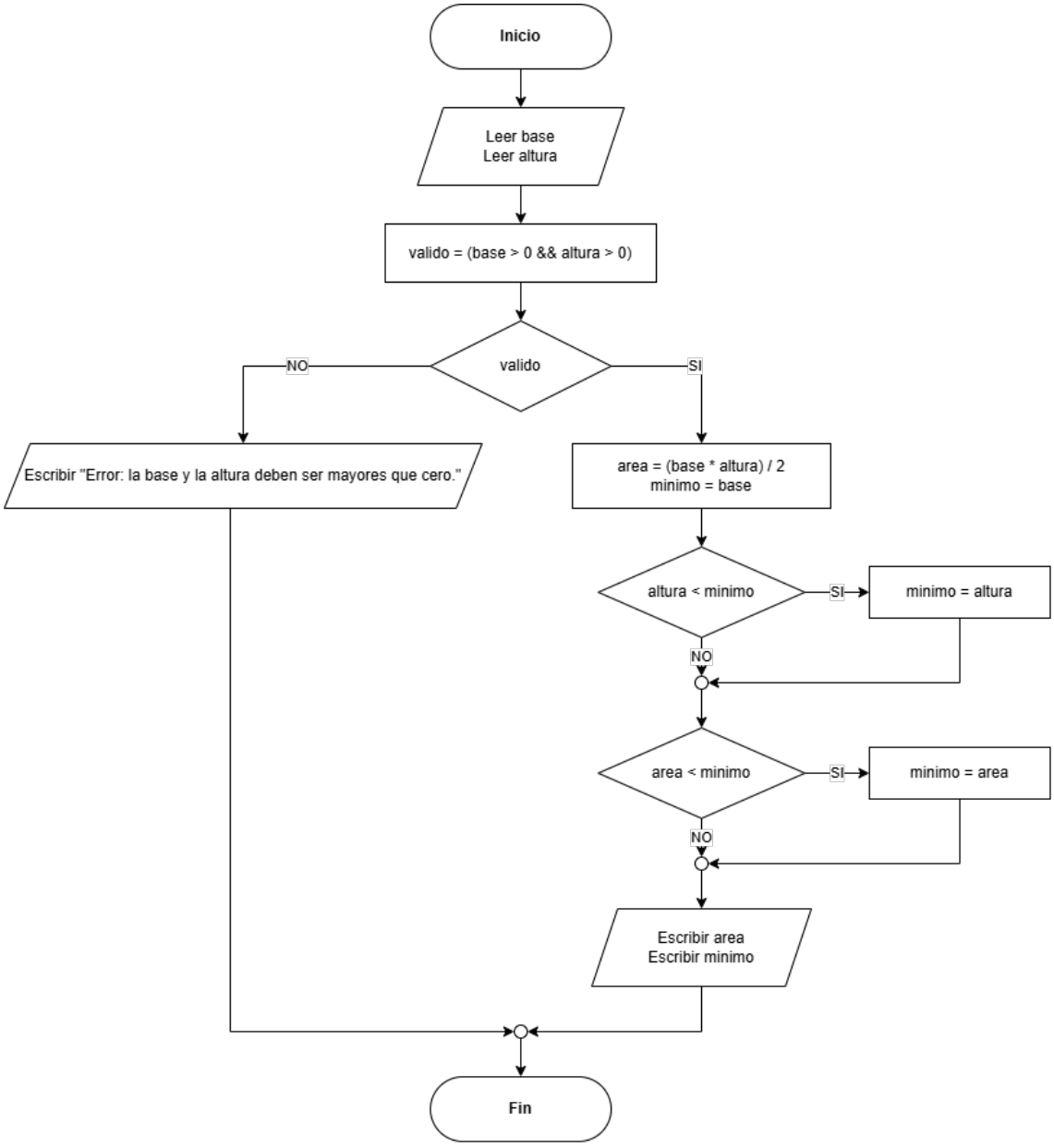
3. Datos de salida

- Equivalencia de la distancia ingresada en yardas a pies.
- Resultado de la expresión relacional pies > 0 (verdadero o falso).
- Cociente obtenido al dividir los pies para 1.5.
- Conclusiones sobre la conversión realizada.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
yardas	float	Almacena la distancia ingresada en yardas.
pies	float	Almacena la equivalencia de yardas a pies.
cociente	float	Almacena el resultado de dividir los pies para 1.5.
valido	int	Almacena el resultado de la expresión relacional pies > 0 (1 verdadero, 0 falso).

3.Diagrama de flujo



4.Pseucodigo

```
Algoritmo Area_Triangulo_Valor_Minimo

    Definir base, altura, area, minimo Como Real
    Definir valido Como Logico

    Escribir "Ingrese la base del triangulo: "
    Leer base

    Escribir "Ingrese la altura del triangulo: "
    Leer altura

    valido <- (base > 0 Y altura > 0)

    Si valido Entonces

        area <- (base * altura) / 2

        minimo <- base

        Si altura < minimo Entonces
            minimo <- altura
        FinSi

        Si area < minimo Entonces
            minimo <- area
        FinSi

        Escribir "El area del triangulo es: ", area

        Escribir "El valor minimo numerico es: ", minimo

    SiNo

        Escribir "Error: la base y la altura deben ser mayores que cero."

    FinSi

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1: Datos válidos

Condición verdadera

Datos de entrada:

- Base = 10
- Altura = 6

Cálculos:

1. Validación de los datos:

$$valido = \big(10 > 0 \;\&\&\; 6 > 0\big) = 1$$

2. Cálculo del área:

$area = \frac{10 \times 6}{2} = 30$

3. Determinación del valor mínimo:

- minimo = base = 10
- ¿Altura menor que mínimo?$(6 < 10)$→Verdadero. Entonces minimo = 6.
- ¿Área menor que mínimo? (30 < 6) →Falso.

Resultado final: minimo = 6.

Tabla de prueba de escritorio:

Paso	base	altura	area	minimo	valido	Evaluación de condiciones	Salida en pantalla
1	—	—	—	—	—	—	Ingrese la base del triángulo:
2	10	—	—	—	—	—	Se ingresa 10
3	10	6	—	—	—	—	Ingrese la altura del triángulo: Se ingresa 6
4	10	6	—	—	1	(base > 0 && altura > 0) = Verdadero	—
5	10	6	—	—	1	if (valido) = Verdadero	—
6	10	6	30	—	1	—	—

7	10	6	30	10	1	Asignación minimo = base	—
8	10	6	30	6	1	altura < minimo = Verdadero	—
9	10	6	30	6	1	area < minimo = Falso	—
10	10	6	30	6	1	—	El área del triángulo es: 30.00
11	10	6	30	6	1	—	El valor mínimo numérico es: 6.00

5.2. Prueba de escritorio N.º 2: Datos no válidos

Condición falsa

Datos de entrada:

- Base = -4
- Altura = 8

Cálculos:

1. Validación de los datos:

$$valido = \left(-4 > 0 \;\&\&\; 8 > 0 \right) = 0$$

La condición es falsa porque la base es negativa.

2. Como valido = 0, el programa no ingresa al bloque if.

Las variables area y minimo no reciben ningún valor, por lo que permanecen sin inicializar.

Tabla de prueba de escritorio:

Paso	**base**	**altura**	**area**	**minimo**	**valido**	**Evaluación de condiciones**	**Salida en pantalla**
1	—	—	—	—	—	—	Ingrese la base del triángulo:
2	-4	—	—	—	—	—	Se ingresa -4
3	-4	8	—	—	—	—	Ingrese la altura del triángulo: Se ingresa 8
4	-4	8	—	—	0	(base > 0 && altura > 0) = Falso	—
5	-4	8	—	—	0	if (valido) = Falso	No entra al bloque if

6. Codificación en lenguaje C

```c

#include <stdio.h>

int main()
{
    float base = 0.0;
    float altura = 0.0;
    float area = 0.0;
    float minimo = 0.0;
    int valido = 0;

    printf("Ingrese la base del triangulo: ");
    scanf("%f", &base);

    printf("Ingrese la altura del triangulo: ");
    scanf("%f", &altura);

    valido = (base > 0 && altura > 0);

    if (valido)
    {
        area = (base * altura) / 2;

        minimo = base;

        if (altura < minimo)
        {
            minimo = altura;
        }

        if (area < minimo)
        {
            minimo = area;
        }

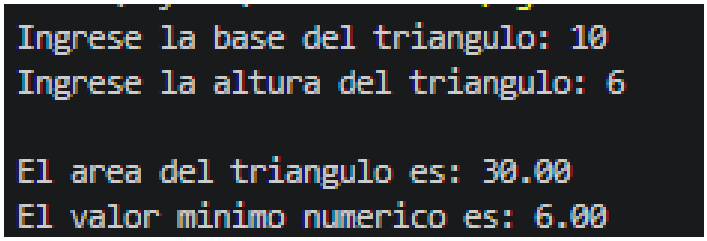
        printf("\nEl area del triangulo es: %.2f\n", area);

        printf("El valor minimo numerico es: %.2f\n", minimo);
    }
    else
    {
        printf("\nError: la base y la altura deben ser mayores que cero.\n");
    }
}

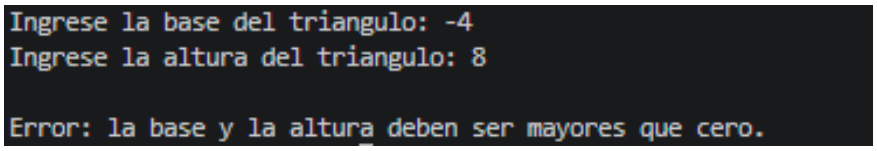
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 9

1.Enunciado del problema

En una farmacia, construya un algoritmo que lea una distancia en metros (real). Use una constante simbolica METRO_PIES = 3.28084. Calcule la equivalencia con la expresion aritmetica (metros * METRO_PIES) y asignela a pies. Evalúe la expresion relacional (pies > 0). Muestre por pantalla 'X metros equivalen a Y pies'. Ademas del calculo principal, determine el doble del resultado. Incluya un mensaje de confirmacion al finalizar la operacion.

2.Análisis del problema

1. Datos de entrada

- metros: distancia expresada en metros, ingresada por teclado (tipo real).

2. Proceso

1. Leer la distancia en metros.
2. Definir la constante simbólica METRO_PIES = 3.28084.
3. Calcular la equivalencia de metros a pies:

$$pies = metros * METRO_PIES$$

4. Evaluar la expresión relacional:

$$pies > 0$$

5. Calcular el doble del resultado:

$$doble = pies * 2$$

6. Mostrar por pantalla el mensaje: «X metros equivalen a Y pies».
7. Mostrar el resultado de la expresión relacional y el doble de los pies.
8. Mostrar un mensaje de confirmación al finalizar la operación.

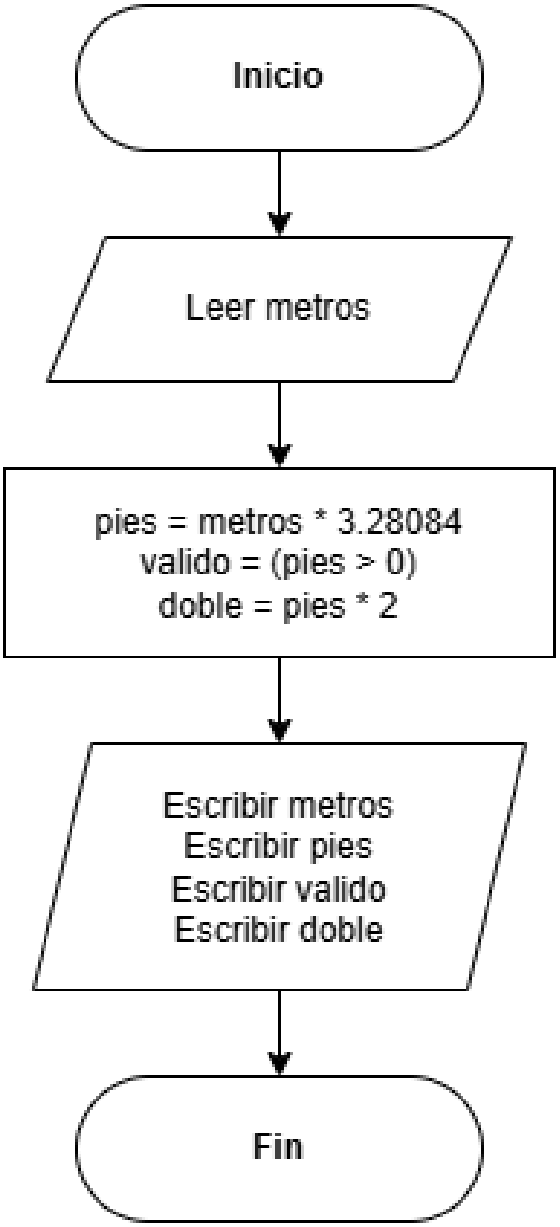
3. Datos de salida

- Equivalencia de metros a pies.
- Resultado de la expresión relacional pies > 0 (verdadero o falso).
- Doble del resultado obtenido.
- Mensaje de confirmación de la operación finalizada correctamente.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
metros	float	Almacena la distancia ingresada en metros.
pies	float	Almacena la equivalencia de metros a pies.
doble	float	Almacena el doble del resultado en pies.
valido	int	Almacena el resultado de la expresión relacional pies > 0 (1 verdadero, 0 falso).
METRO_PIES	Constante simbólica (#define)	Representa el factor de conversión de metros a pies: 3.28084.

3.Diagrama de flujo



4.Pseucodigo

```
Algoritmo Conversion_Metros_a_Pies

    Definir metros, pies, doble Como Real
    Definir valido Como Logico

    Escribir "Ingrese la distancia en metros: "
    Leer metros

    pies <- metros * 3.28084

    valido <- (pies > 0)

    doble <- pies * 2

    Escribir metros, " metros equivalen a ", pies, " pies"

    Escribir "El resultado es positivo: ", valido

    Escribir "El doble del resultado es: ", doble, " pies"

    Escribir "Operacion finalizada correctamente."

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1: Distancia positiva

Datos de entrada:

- Metros = 10

Cálculos:

1. Conversión de metros a pies:

$$pies = 10 \times 3.28084 = 32.8084$$

2. Evaluación de la condición:

$$valido = \left(32.8084 > 0\right) = 1$$

3. Cálculo del doble:

$$doble = 32.8084 \times 2 = 65.6168$$

Tabla de prueba de escritorio:

Paso	metros	pies	valido	doble	Evaluación de condiciones	Salida en pantalla
1	—	—	—	—	—	Ingrese la distancia en metros:
2	10	—	—	—	—	Se ingresa 10
3	10	32.8084	—	—	—	Se realiza la conversión
4	10	32.8084	1	—	pies > 0 = Verdadero	—
5	10	32.8084	1	65.6168	—	Se calcula el doble
6	10	32.8084	1	65.6168	—	Muestra los resultados

5.2. Prueba de escritorio N.º 2: Distancia positiva

Datos de entrada:

- Metros = 15

Cálculos:

1. Conversión de metros a pies:

$$pies = 15 \times 3.28084 = 49.2126$$

2. Evaluación de la condición:

$$valido = \left(49.2126 > 0\right) = 1$$

3. Cálculo del doble:

$$doble = 49.2126 \times 2 = 98.4252$$

Tabla de prueba de escritorio:

Paso	metros	pies	valido	doble	Evaluación de condiciones	Salida en pantalla
1	—	—	—	—	—	Ingrese la distancia en metros:
2	15	—	—	—	—	Se ingresa 15
3	15	49.2126	—	—	—	Se realiza la conversión
4	15	49.2126	1	—	pies > 0 = Verdadero	—
5	15	49.2126	1	98.4252	—	Se calcula el doble
6	15	49.2126	1	98.4252	—	Muestra los resultados

6. Codificación en lenguaje C

```c

#include <stdio.h>

#define METRO_PIES 3.28084

int main()
{
    float metros;
    float pies;
    float doble;
    int valido;

    printf("Ingrese la distancia en metros: ");
    scanf("%f", &metros);

    pies = metros * METRO_PIES;

    valido = (pies > 0);

    doble = pies * 2;

    printf("\n%.2f metros equivalen a %.2f pies\n", metros, pies);

    printf("El resultado es positivo: %d\n", valido);

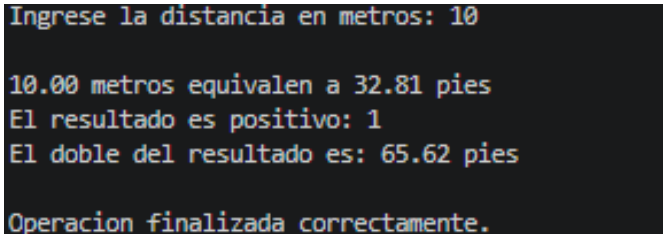
    printf("El doble del resultado es: %.2f pies\n", doble);

    printf("\nOperacion finalizada correctamente.\n");
}

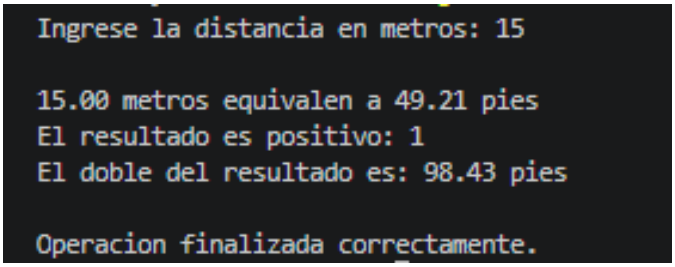
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 10

1.Enunciado del problema

En una empresa de telecomunicaciones, construya un algoritmo que lea dos numeros enteros b y w por teclado. Evalúe las expresiones relacionales (b == w) y (b != w). Asigne los resultados booleanos a son_iguales y son_diferentes. Muestre por pantalla si los numeros son iguales o diferentes. Justifique el tipo de dato de cada variable. Adicionalmente, calcule la mitad del resultado obtenido. Incluya una seccion que explique brevemente el proposito del algoritmo.

2.Análisis del problema

1. Datos de entrada

- b: primer número entero ingresado por teclado.
- w: segundo número entero ingresado por teclado.

2. Proceso

1. Leer los dos números enteros b y w.
2. Evaluar la expresión relacional para determinar si son iguales:

$$son_iguales = \left(b = = w\right)$$

3. Evaluar la expresión relacional para determinar si son diferentes:

$$son_diferentes = \left(b! = w\right)$$

4. Calcular la mitad del resultado obtenido:

$$mitad = son_iguales \ / \ 2.0$$

5. Mostrar por pantalla si los números son iguales o diferentes.
6. Mostrar el resultado de la mitad calculada.
7. Explicar brevemente el propósito del algoritmo.

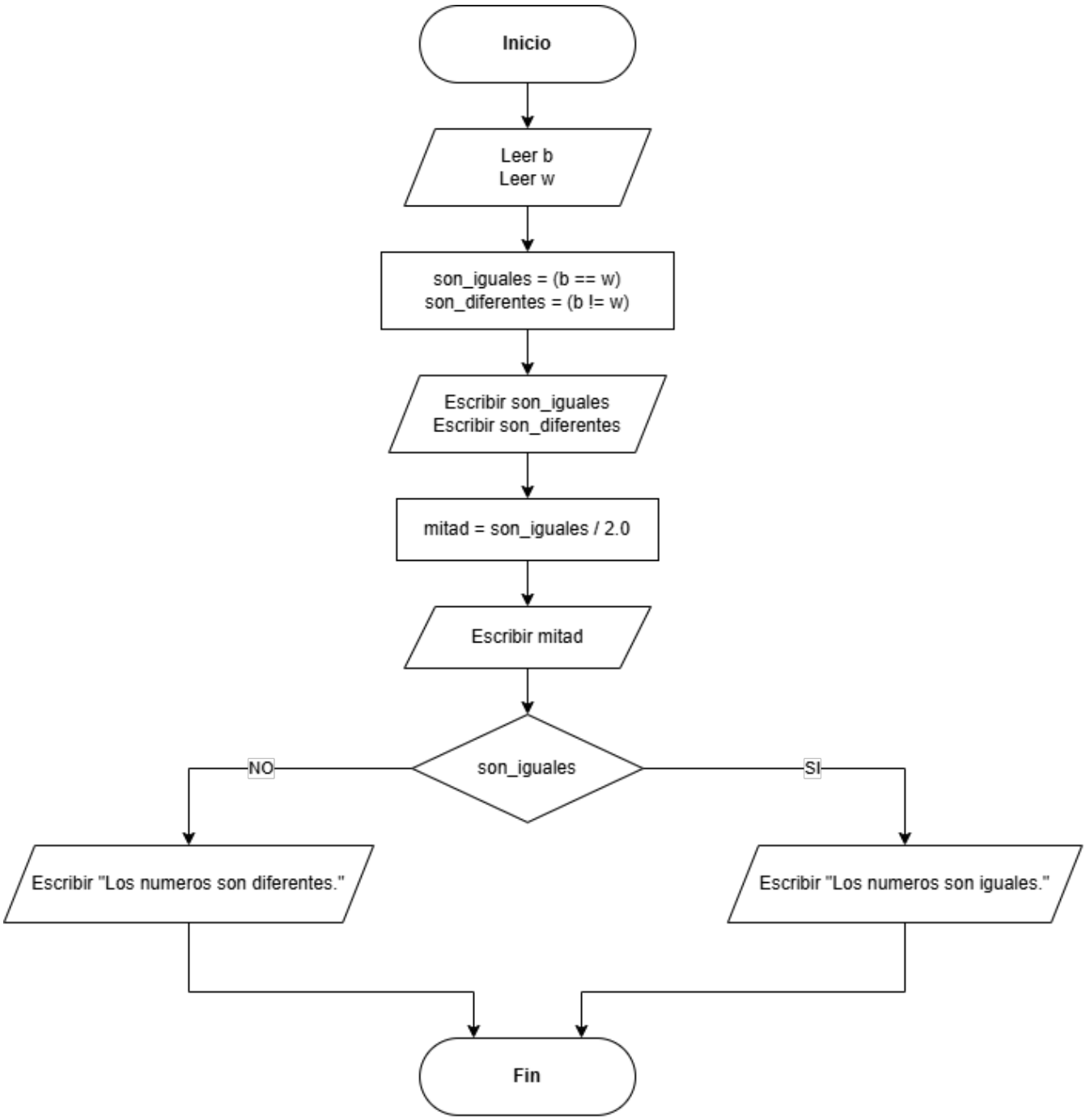
3. Datos de salida

- Resultado de la comparación de igualdad (son_iguales).
- Resultado de la comparación de diferencia (son_diferentes).
- Mensaje indicando si los números son iguales o diferentes.
- Mitad del resultado de la variable son_iguales.
- Propósito del algoritmo.

4. Variables a utilizar

Variable	**Tipo de dato**	**Descripción**
b	int	Almacena el primer número entero.
w	int	Almacena el segundo número entero.
son_iguales	int	Almacena el resultado de la comparación b == w (1 verdadero, 0 falso).
son_diferentes	int	Almacena el resultado de la comparación b != w (1 verdadero, 0 falso).
mitad	float	Almacena la mitad del resultado de son_iguales.

3.Diagrama de flujo



4.Pseucodigo

```
Algoritmo Comparacion_Numeros

    Definir b, w Como Entero
    Definir son_iguales, son_diferentes Como Logico
    Definir mitad Como Real

    Escribir "Ingrese el primer numero: "
    Leer b

    Escribir "Ingrese el segundo numero: "
    Leer w

    son_iguales <- (b = w)
    son_diferentes <- (b <> w)

    Escribir "¿Los numeros son iguales?: ", son_iguales
    Escribir "¿Los numeros son diferentes?: ", son_diferentes

    mitad <- son_iguales / 2.0

    Escribir "La mitad del resultado obtenido es: ", mitad

    Si son_iguales Entonces
        Escribir "Los numeros son iguales."
    SiNo
        Escribir "Los numeros son diferentes."
    FinSi

    Escribir "Proceso con exito."

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1: Números iguales

Condición verdadera

Datos de entrada:

- b = 8
- w = 8

Cálculos:

1. Comparación de igualdad:

$$son_iguales = \left(8 = = 8\right) = 1$$

2. Comparación de diferencia:

$$son_diferentes = \left(8! = 8\right) = 0$$

3. Cálculo de la mitad:

$$mitad = \frac{1}{2.0} = 0.50$$

4. Evaluación de la condición:

$$son_iguales = = 1 \Rightarrow Verdadero$$

Tabla de prueba de escritorio:

Paso	b	w	son_iguales	son_diferentes	mitad	Evaluación de condiciones	Salida en pantalla
1	—	—	—	—	—	—	Ingrese el primer número:
2	8	—	—	—	—	—	Se ingresa 8

3	8	8	—	—	—	—	Ingrese el segundo número: Se ingresa 8
4	8	8	1	—	—	b == w = Verdadero	—
5	8	8	1	0	—	b != w = Falso	—
6	8	8	1	0	0.50	—	—
7	8	8	1	0	0.50	—	Los números son iguales: 1
8	8	8	1	0	0.50	—	Los números son diferentes: 0
9	8	8	1	0	0.50	—	La mitad del resultado obtenido es: 0.50
10	8	8	1	0	0.50	son_iguales == 1 = Verdadero	Los números son iguales. Proceso con éxito.

5.2. Prueba de escritorio N.º 2: Números diferentes

Condición falsa

Datos de entrada:

- b = 10
- w = 5

Cálculos:

1. Comparación de igualdad:

$$son_iguales = \left(10 = = 5\right) = 0$$

2. Comparación de diferencia:

$$son_diferentes = \left(10! = 5\right) = 1$$

3. Cálculo de la mitad:

$$mitad = \frac{0}{2.0} = 0.00$$

4. Evaluación de la condición:

$$son_iguales = = 1 \Rightarrow Falso$$

Tabla de prueba de escritorio:

Paso	b	w	son_iguales	son_diferentes	mitad	Evaluación de condiciones	Salida en pantalla
1	—	—	—	—	—	—	Ingrese el primer número:
2	10	—	—	—	—	—	Se ingresa 10
3	10	5	—	—	—	—	Ingrese el segundo número: Se ingresa 5
4	10	5	0	—	—	b == w = Falso	—
5	10	5	0	1	—	b != w = Verdadero	—
6	10	5	0	1	0.00	—	—
7	10	5	0	1	0.00	—	Los números son iguales: 0
8	10	5	0	1	0.00	—	Los números son diferentes: 1

9	10	5	0	1	0.00	—	La mitad obtenido es: 0.00
10	10	5	0	1	0.00	son_iguales == 1 = Falso	Los números son diferentes. Proceso con éxito.

6. Codificación en lenguaje C

```c

#include <stdio.h>

int main()
{
    int b;
    int w;
    int son_iguales;
    int son_diferentes;
    float mitad;

    printf("Ingrese el primer numero: ");
    scanf("%d", &b);

    printf("Ingrese el segundo numero: ");
    scanf("%d", &w);

    son_iguales = (b == w);
    son_diferentes = (b != w);

    printf("\n¿Los numeros son iguales?: %d", son_iguales);
    printf("\n¿Los numeros son diferentes?: %d", son_diferentes);

    mitad = son_iguales / 2.0;

    printf("\nLa mitad del resultado obtenido es: %.2f", mitad);

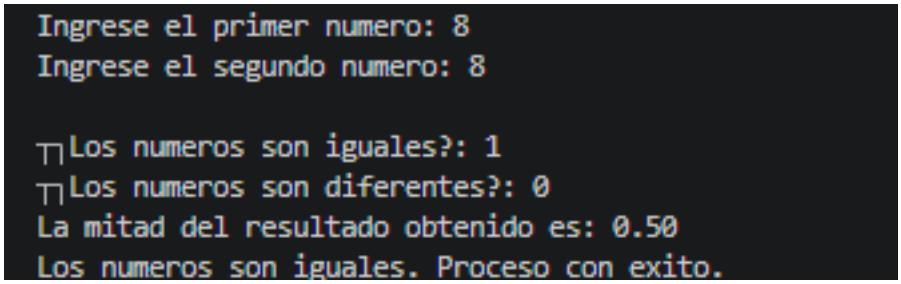
    if (son_iguales == 1)
    {
        printf("\nLos numeros son iguales.");
    }
    else
    {
        printf("\nLos numeros son diferentes.");
    }

    printf(" Proceso con exito.\n");
}

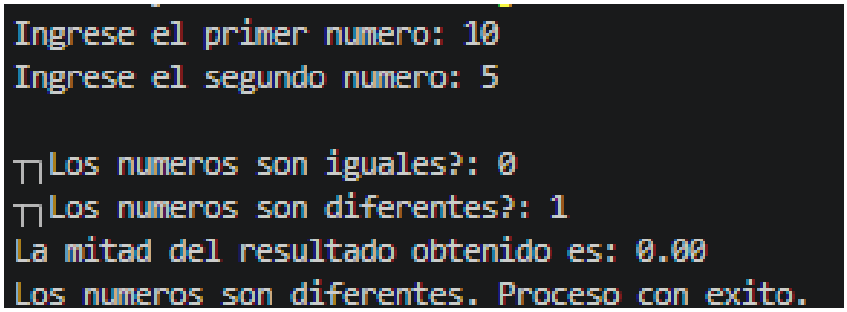
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERICICIO 11

1.Enunciado del problema

En una empresa de telecomunicaciones, construya un algoritmo que lea un peso en onzas (real). Use una constante simbolica ONZA_GRAMO = 28.3495. Calcule la equivalencia en gramos con la expresion aritmetica (onzas * ONZA_GRAMO) y asignela a gramos. Evalúe la expresion relacional (gramos > 0). Muestre por pantalla 'X onzas equivalen a Y gramos'. Adicionalmente, calcule la mitad del resultado obtenido. Incluya una seccion que explique brevemente el proposito del algoritmo.

2.Análisis del problema

1. Datos de entrada

onzas: peso expresado en onzas, ingresado por teclado (tipo real).

2. Proceso

1. Leer el peso en onzas.
2. Definir la constante simbólica ONZA_GRAMO = 28.3495.
3. Calcular la equivalencia de onzas a gramos:

$$gramos = onzas * ONZA_GRAMO$$

4. Evaluar la expresión relacional:

$$gramos > 0$$

5. Calcular la mitad del resultado obtenido:

$$mitad = gramos \ / \ 2$$

6. Mostrar por pantalla el mensaje: «X onzas equivalen a Y gramos».
7. Mostrar el resultado de la expresión relacional y la mitad del resultado en gramos.

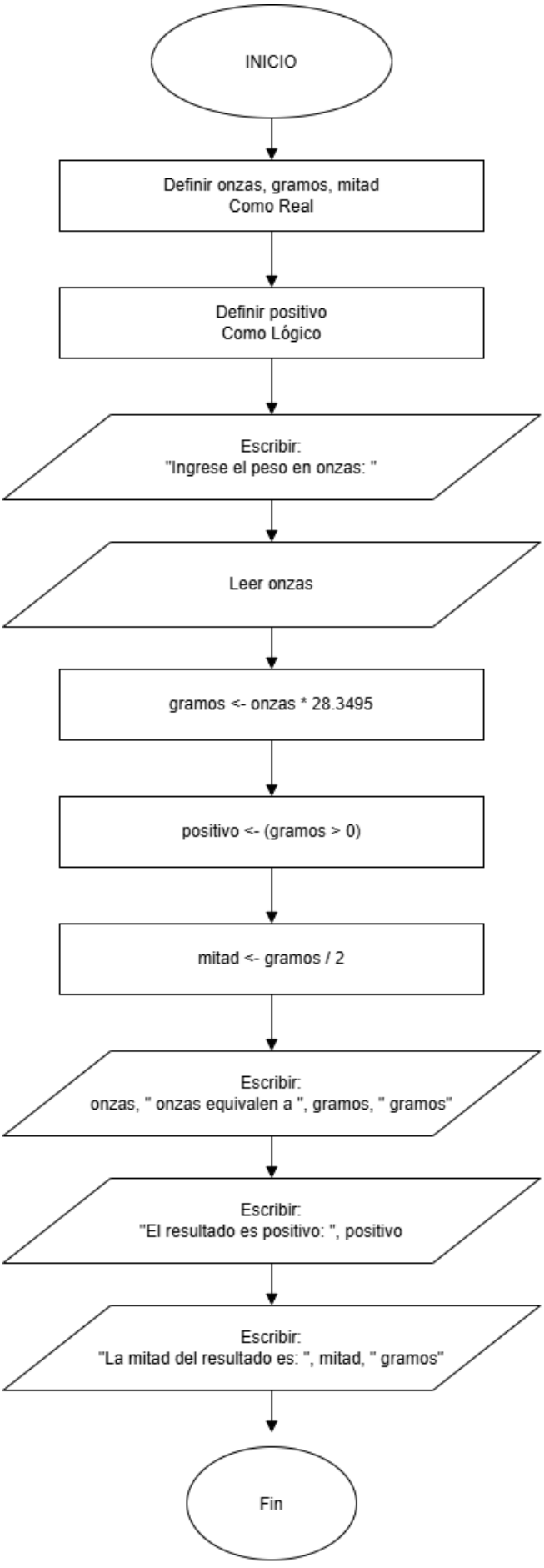
3. Datos de salida

- Equivalencia del peso ingresado de onzas a gramos.
- Resultado de la expresión relacional gramos > 0 (verdadero o falso).
- Mitad del resultado obtenido en gramos.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
onzas	float	Almacena el peso ingresado en onzas.
gramos	float	Almacena la equivalencia de onzas a gramos.
positivo	int	Almacena el resultado de la expresión relacional gramos > 0 (1 verdadero, 0 falso).
mitad	float	Almacena la mitad del resultado obtenido en gramos.
ONZA_GRAMO	Constante simbólica (#define)	Representa el factor de conversión de onzas a gramos: 28.3495.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Onzas_a_Gramos

    Definir onzas, gramos, mitad Como Real
    Definir positivo Como Logico

    Escribir "Ingrese el peso en onzas: "
    Leer onzas

    gramos <- onzas * 28.3495

    positivo <- (gramos > 0)

    mitad <- gramos / 2

    Escribir onzas, " onzas equivalen a ", gramos, " gramos"

    Escribir "El resultado es positivo: ", positivo

    Escribir "La mitad del resultado es: ", mitad, " gramos"

FinAlgoritmo
```

1. Prueba de escritorio N.º 1: Peso positivo

Datos de entrada:

- Onzas = 10

Cálculos:

1. Conversión de onzas a gramos:

$$gramos = 10 \times 28.3495 = 283.495$$

2. Evaluación de la condición:

$$positivo = \left(283.495 > 0\right) = 1$$

3. Cálculo de la mitad:

$$mitad = \frac{283.495}{2} = 141.7475$$

Tabla de prueba de escritorio:

Paso	onzas	gramos	positivo	mitad	Evaluación de condiciones	Salida en pantalla
1	—	—	—	—	—	Ingrese el peso en onzas:
2	10	—	—	—	—	Se ingresa 10
3	10	283.4950	—	—	—	Se realiza la conversión
4	10	283.4950	1	—	gramos > 0 = Verdadero	—
5	10	283.4950	1	141.7475	—	Se calcula la mitad
6	10	283.4950	1	141.7475	—	Muestra los resultados

2. Prueba de escritorio N.º 2: Peso positivo

Segundo conjunto de datos

Datos de entrada:

- Onzas = 5

Cálculos:

1. Conversión de onzas a gramos:

$$gramos = 5 \times 28.3495 = 141.7475$$

2. Evaluación de la condición:

$$positivo = \left(141.7475 > 0\right) = 1$$

3. Cálculo de la mitad:

$$mitad = \frac{141.7475}{2} = 70.87375$$

Tabla de prueba de escritorio:

Paso	onzas	gramos	positivo	mitad	Evaluación de condiciones	Salida en pantalla
1	—	—	—	—	—	Ingrese el peso en onzas:
2	5	—	—	—	—	Se ingresa 5
3	5	141.7475	—	—	—	Se realiza la conversión
4	5	141.7475	1	—	gramos > 0 = Verdadero	—
5	5	141.7475	1	70.87375	—	Se calcula la mitad
6	5	141.7475	1	70.87375	—	Muestra los resultados

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define ONZA_GRAMO 28.3495

int main()
{
    float onzas;
    float gramos;
    float mitad;
    int positivo;

    printf("Ingrese el peso en onzas: ");
    scanf("%f", &onzas);

    gramos = onzas * ONZA_GRAMO;

    positivo = (gramos > 0);

    mitad = gramos / 2;

    printf("\n%.2f onzas equivalen a %.2f gramos\n", onzas, gramos);

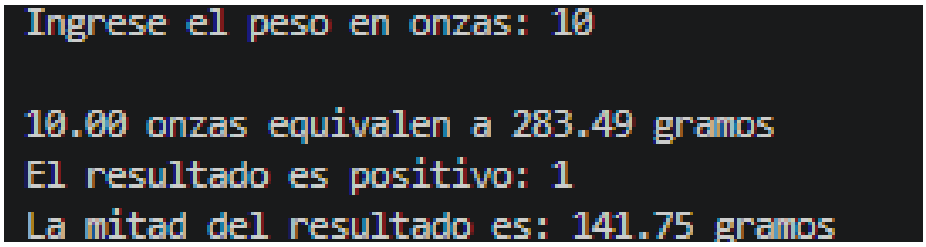
    printf("El resultado es positivo: %d\n", positivo);

    printf("La mitad del resultado es: %.2f gramos\n", mitad);

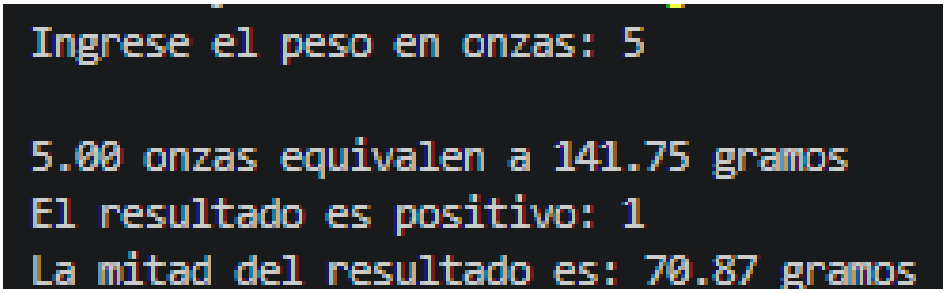

}
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 12

1.Enunciado del problema

En un gimnasio, construya un algoritmo que lea un volumen en litros (real). Use la constante literal 1000. Calcule la equivalencia en mililitros con la expresion aritmetica (litros * 1000) y asignela a ml. Evalúe la expresion relacional (ml >= 0). Muestre por pantalla 'X litros equivalen a Y ml'. Adicionalmente, calcule el porcentaje del resultado respecto a un valor de referencia. Justifique el tipo de dato simple elegido para cada variable.

2.Análisis del problema

1. Datos de entrada

- litros: volumen expresado en litros, ingresado por teclado (tipo real).

2. Proceso

1. Leer el volumen en litros.
2. Utilizar la constante literal 1000 para realizar la conversión.
3. Calcular la equivalencia de litros a mililitros:

$$ml = litros * 1000$$

4. Evaluar la expresión relacional:

$$ml >= 0$$

5. Definir un valor de referencia de 5000.0.
6. Calcular el porcentaje del resultado respecto al valor de referencia:

$$porcentaje = \frac{ml}{referencia} * 100$$

7. Mostrar por pantalla la equivalencia de litros a mililitros.
8. Mostrar el resultado de la expresión relacional y el porcentaje calculado.

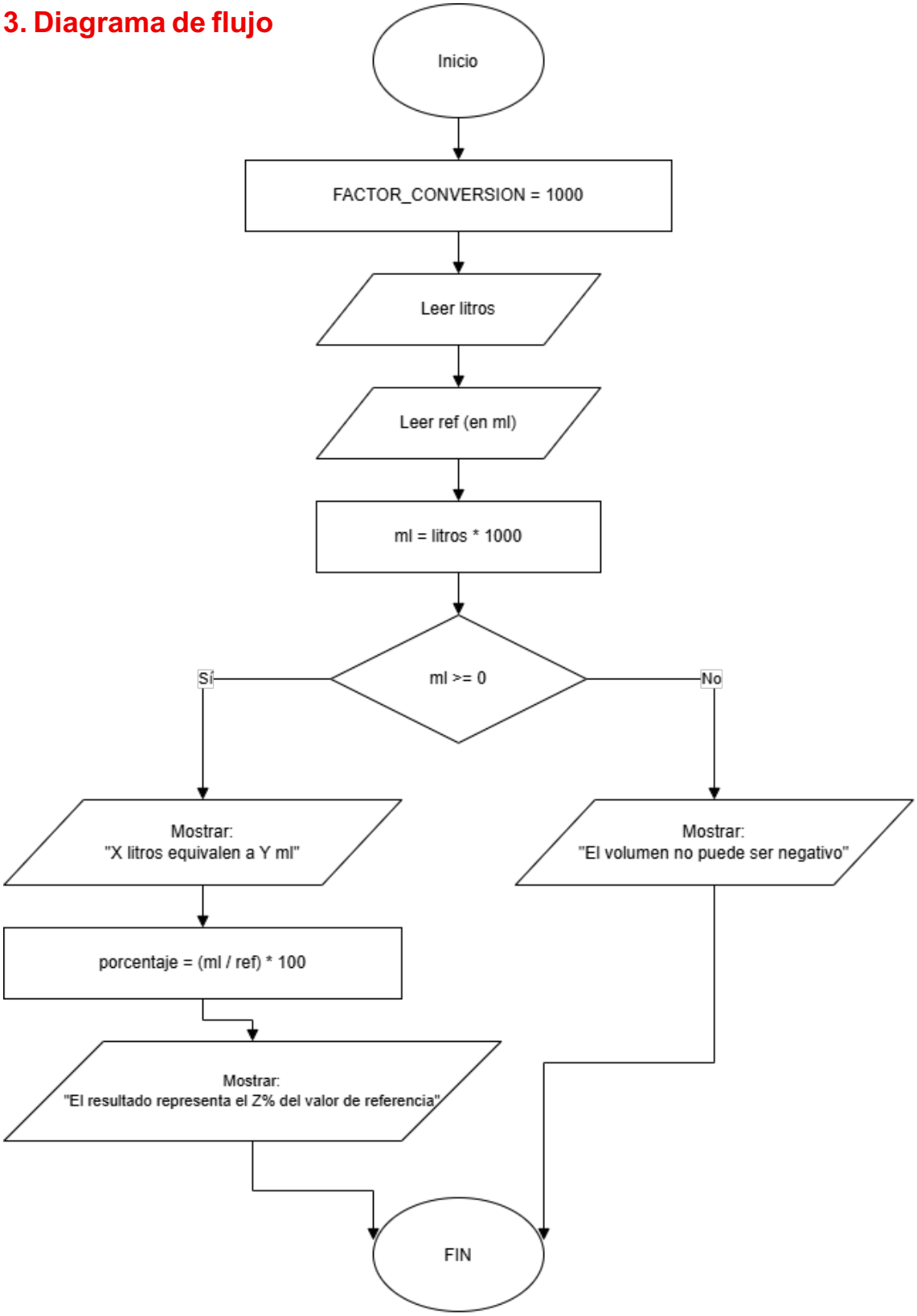
3. Datos de salida

- Equivalencia del volumen ingresado de litros a mililitros.
- Resultado de la expresión relacional ml >= 0 (1 verdadero, 0 falso).
- Porcentaje del resultado respecto al valor de referencia.

4. Variables a utilizar

Variable	Tipo de dato	Justificación
litros	float	Permite almacenar el volumen ingresado con valores decimales.
ml	float	Almacena la conversión a mililitros y permite representar decimales.
referencia	float	Almacena el valor de referencia de 5000.0.
valido	int	Almacena el resultado de la expresión relacional ml >= 0 (1 verdadero, 0 falso).
porcentaje	float	Almacena el porcentaje calculado, que puede contener valores decimales.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Litros_a_Mililitros

    Definir litros, ml, porcentaje, referencia Como Real
    Definir valido Como Logico

    referencia <- 5000.0

    Escribir "Ingrese el volumen en litros: "
    Leer litros

    ml <- litros * 1000

    valido <- (ml >= 0)

    porcentaje <- (ml / referencia) * 100

    Escribir litros, " litros equivalen a ", ml, " ml"

    Escribir "El resultado es mayor o igual a cero: ", valido

    Escribir "El porcentaje respecto al valor de referencia es: ", porcentaje, "%"

FinAlgoritmo
```

5.1. Prueba de escritorio

Datos de entrada:

litros = 5

N.º de paso	litros	ml	referencia	valido	porcentaje	Evaluación de condiciones	Salida en pantalla
1	5	—	5000	—	—	No aplica	Ingrese el volumen en litros: 5
2	5	5000	5000	—	—	No aplica	—
3	5	5000	5000	1	—	ml >= 0 → Verdadero	—
4	5	5000	5000	1	100	No aplica	—
5	5	5000	5000	1	100	No aplica	5.00 litros equivalen a 5000.00 ml
6	5	5000	5000	1	100	No aplica	El resultado es mayor o igual a cero: 1
7	5	5000	5000	1	100	No aplica	El porcentaje respecto al valor de referencia es: 100.00%

Operaciones realizadas:

- ml = 5 * 1000 = 5000
- valido = (5000 >= 0) = 1
- porcentaje = (5000 / 5000) * 100 = 100

5.2. Prueba de escritorio

Datos de entrada:

- litros = 2.5

N.º de paso	litros	ml	referencia	valido	porcentaje	Evaluación de condiciones	Salida en pantalla
1	2.5	—	5000	—	—	No aplica	Ingrese el volumen en litros: 2.5
2	2.5	2500	5000	—	—	No aplica	—
3	2.5	2500	5000	1	—	ml >= 0 → Verdadero	—
4	2.5	2500	5000	1	50	No aplica	—
5	2.5	2500	5000	1	50	No aplica	2.50 litros equivalen a 2500.00 ml
6	2.5	2500	5000	1	50	No aplica	El resultado es mayor o igual a cero: 1
7	2.5	2500	5000	1	50	No aplica	El porcentaje respecto al valor de referencia es: 50.00%

Operaciones realizadas:

- ml = 2.5 * 1000 = 2500
- valido = (2500 >= 0) = 1
- porcentaje = (2500 / 5000) * 100 = 50

6. Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{
    float litros;
    float ml;
    float porcentaje;
    float referencia = 5000.0;
    int valido;

    printf("Ingrese el volumen en litros: ");
    scanf("%f", &litros);

    ml = litros * 1000;

    valido = (ml >= 0);

    porcentaje = (ml / referencia) * 100;

    printf("\n%.2f litros equivalen a %.2f ml\n", litros, ml);

    printf("El resultado es mayor o igual a cero: %d\n", valido);

    printf("El porcentaje respecto al valor de referencia es: %.2f%%\n", porcentaje);

    return 0;
}
```

7. Salida del programa

Prueba 1

```
Ingrese el volumen en litros: 5

5.00 litros equivalen a 5000.00 ml
El resultado es mayor o igual a cero: 1
El porcentaje respecto al valor de referencia es: 100.00%
```

Prueba 2

```
Ingrese el volumen en litros: 2.5

2.50 litros equivalen a 2500.00 ml
El resultado es mayor o igual a cero: 1
El porcentaje respecto al valor de referencia es: 50.00%
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 13

1.Enunciado del problema

En una panaderia, construya un algoritmo que lea la edad (entero) de una persona. Use una constante simbolica MAYORIA_EDAD = 18. Evalúe la expresion relacional (edad >= MAYORIA_EDAD). Asigne el resultado booleano a es_mayor. Muestre por pantalla si la persona es mayor o menor de edad. Justifique el tipo de dato de cada variable. Ademas, calcule el triple del resultado si este es positivo. Asegurese de que las expresiones logicas esten bien parentizadas.

2.Analisis del problema

Análisis del problema

1. Datos de entrada

- edad: edad de la persona ingresada por teclado (tipo entero).

2. Proceso

1. Leer la edad de la persona.
2. Declarar la constante simbólica:
    - MAYORIA_EDAD = 18 (representa la edad mínima para ser mayor de edad).
3. Evaluar la expresión relacional para determinar si la persona es mayor de edad:

$$\left(edad \geq MAYORIA_EDAD\right)$$

4. Asignar el resultado booleano de la expresión a la variable es_mayor.
5. Mostrar por pantalla si la persona es mayor o menor de edad:
    - Si es_mayor es verdadero, mostrar que es mayor de edad.
    - Si es_mayor es falso, mostrar que es menor de edad.
6. Evaluar si el resultado es positivo mediante la condición:

$$\left(es_mayor > 0\right)$$

7. Calcular el triple del resultado:

$$triple = es_mayor \times 3$$

8. Mostrar el valor del triple calculado.

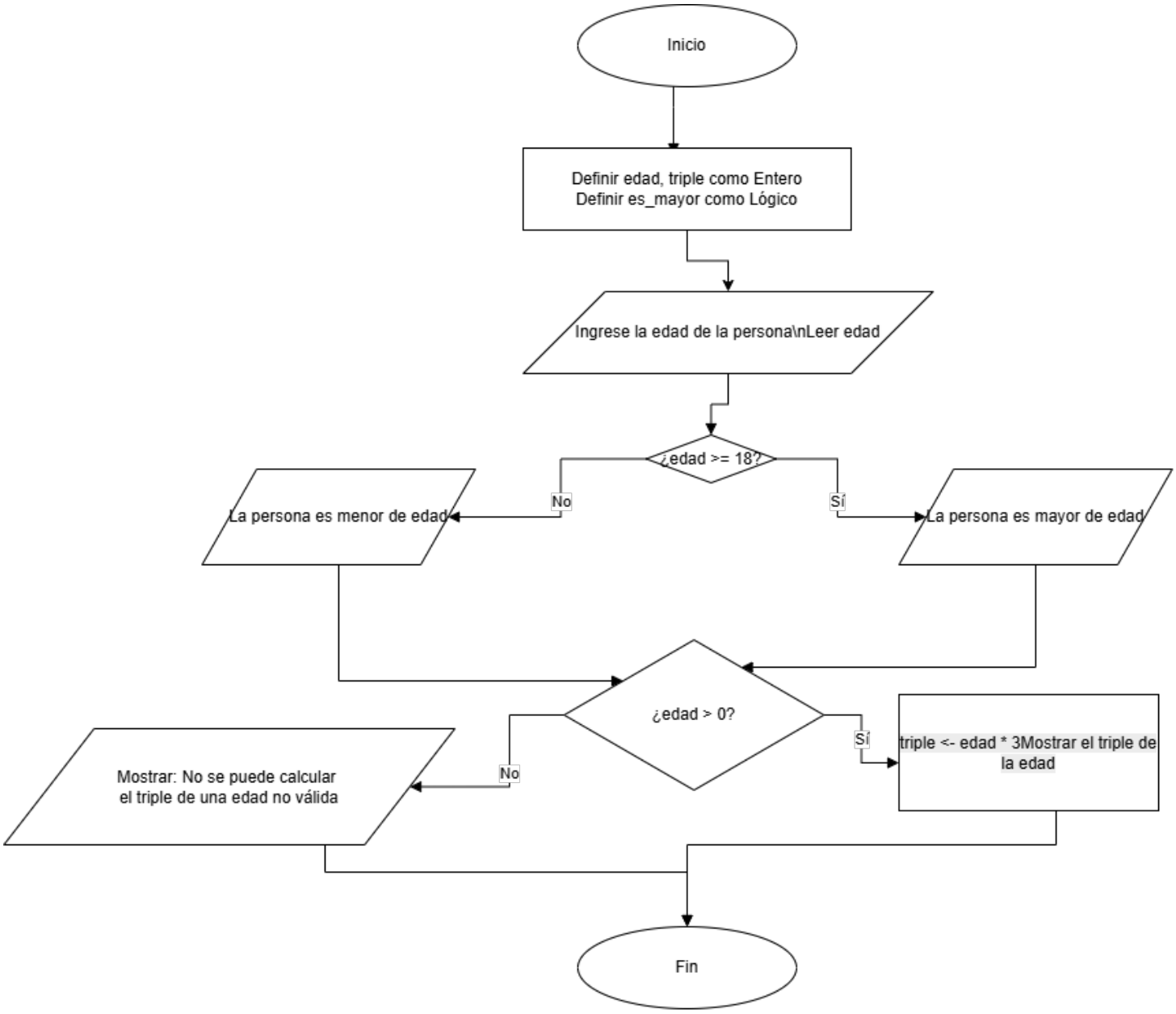
3. Datos de salida

- Mensaje indicando si la persona es mayor o menor de edad.
- Resultado booleano de es_mayor (1 si es verdadero y 0 si es falso).
- Triple del resultado booleano.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
edad	int	Almacena la edad ingresada por teclado.
es_mayor	bool	Almacena el resultado de comparar la edad con la mayoría de edad.
triple	int	Almacena el triple del resultado booleano.
MAYORIA_EDAD	const int	Representa la edad mínima para ser mayor de edad (18 años).

3.Diagrama de flujo



4.Pseucodigo

```
Algoritmo Mayoría_Edad_Triple

    Definir edad, triple Como Entero
    Definir es_mayor Como Logico

    Escribir "Ingrese la edad de la persona:"
    Leer edad

    es_mayor <- edad >= 18

    Si es_mayor Entonces
        Escribir "La persona es mayor de edad."
    SiNo
        Escribir "La persona es menor de edad."
    FinSi

    Si edad > 0 Entonces
        triple <- edad * 3
        Escribir "El triple de la edad es: ", triple
    SiNo
        Escribir "No se puede calcular el triple de una edad no valida."
    FinSi

FinAlgoritmo
```

5.1. Prueba de escritorio

Datos de entrada: edad = 20

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	edad = 20	No aplica	Ingrese la edad de la persona: 20
2	es_mayor = 1	edad >= 18 →Verdadero	—
3	es_mayor = 1	if (es_mayor) →Verdadero	La persona es mayor de edad.
4	triple = 60	edad > 0 →Verdadero	El triple de la edad es: 60

Operación realizada:

-triple = 20 * 3 = 60

5.2. Prueba de escritorio

Datos de entrada: edad = 15

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	edad = 15	No aplica	Ingrese la edad de la persona: 15
2	es_mayor = 0	edad >= 18 →Falso	—
3	es_mayor = 0	if (es_mayor) →Falso, ejecuta else	La persona es menor de edad.
4	triple = 45	edad > 0 →Verdadero	El triple de la edad es: 45

Operación realizada:

- triple = 15 * 3 = 45

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define MAYORIA_EDAD 18

int main() {

    int edad;
    int es_mayor;
    int triple;

    printf("Ingrese la edad de la persona: ");
    scanf("%d", &edad);

    es_mayor = (edad >= MAYORIA_EDAD);

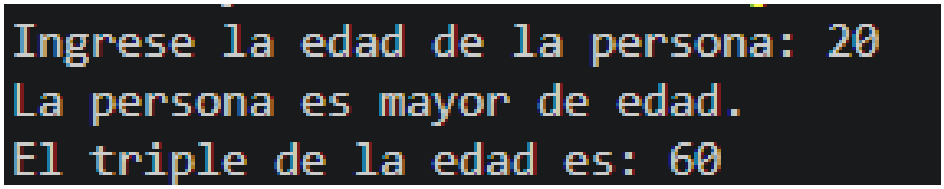
    if (es_mayor)
    {
        printf("La persona es mayor de edad.\n");
    }
    else
    {
        printf("La persona es menor de edad.\n");
    }

    if ((edad > 0))
    {
        triple = (edad * 3);
        printf("El triple de la edad es: %d\n", triple);
    }

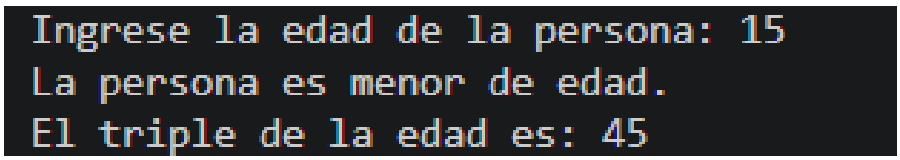
}
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 14

1.Enunciado del problema

En un hospital, diseñe un algoritmo que lea un monto en dolares (real). Use una constante simbolica TASA_CAMBIO = 4000.0. Calcule el equivalente en pesos con la expresion aritmetica (dolares * TASA_CAMBIO) y asignelo a pesos. Evalúe la expresion relacional (pesos > 0). Muestre por pantalla 'X dolares equivalen a Y pesos'. Ademas del calculo principal, determine si el resultado es positivo o negativo. Asegurese de inicializar todas las variables antes de usarlas.

2.Análisis del problema

1. Datos de entrada

- dolares: monto de dinero ingresado por teclado (tipo real).

2. Proceso

1. Declarar e inicializar las variables antes de utilizarlas.
2. Declarar la constante simbólica:
    - TASA_CAMBIO = 4000.0 (representa el valor de cambio de un dólar a pesos).
3. Leer el monto en dólares.
4. Calcular el equivalente en pesos mediante la expresión aritmética:

$$pesos = dolares \times TASA_CAMBIO$$

5. Evaluar la expresión relacional para determinar si el resultado es positivo:

$$\left(pesos > 0\right)$$

6. Asignar el resultado de la evaluación a la variable es_positivo.
7. Determinar si el resultado es positivo o negativo:
    - Si pesos > 0, mostrar que el resultado es positivo.
    - Si pesos < 0, mostrar que el resultado es negativo.
    - Si pesos == 0, mostrar que el resultado es cero.
8. Mostrar por pantalla la equivalencia entre dólares y pesos.

3. Datos de salida

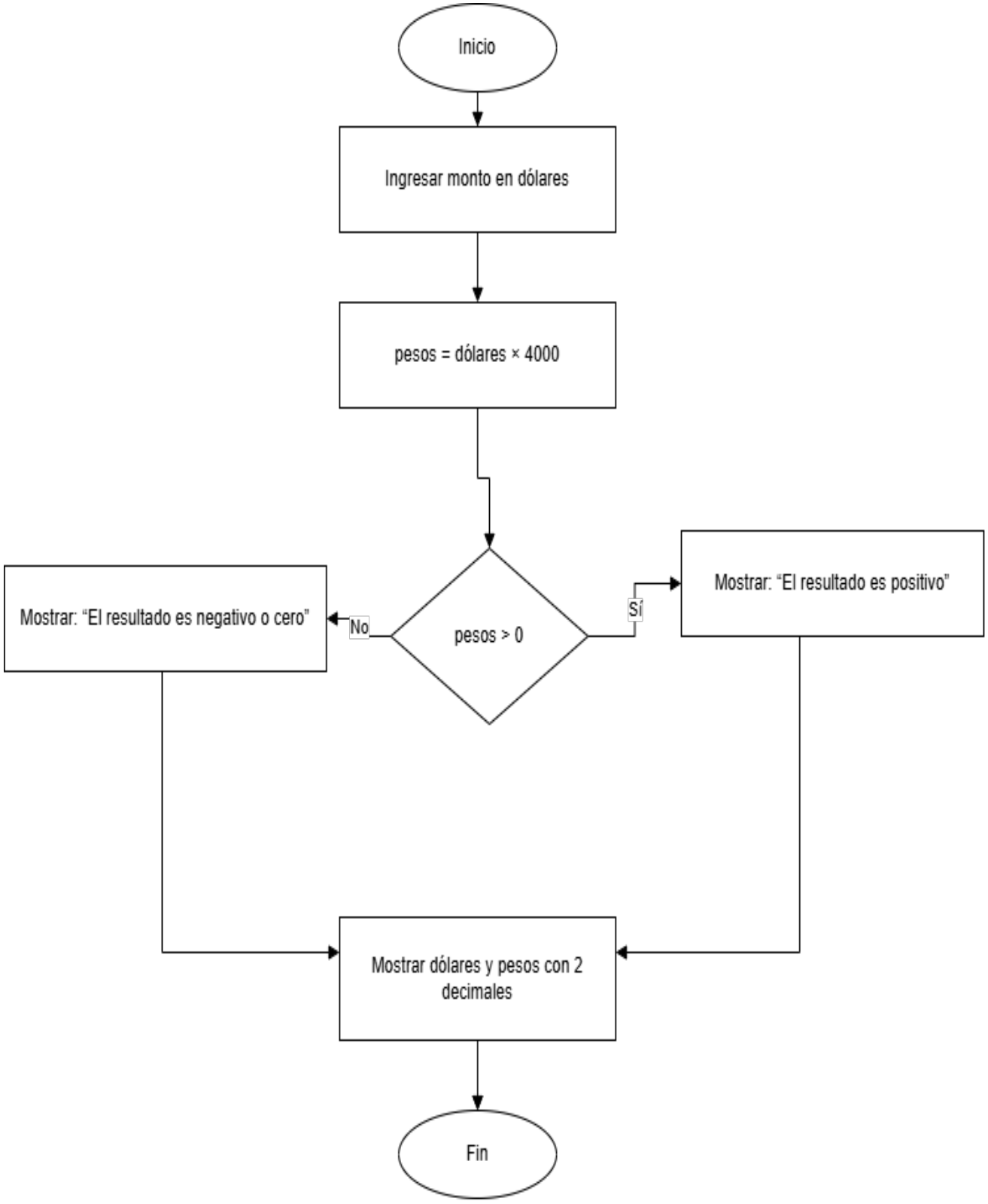
- Monto ingresado en dólares.
- Equivalente calculado en pesos.

- Resultado de la expresión relacional (1 si es positivo y 0 si no lo es).
- Mensaje indicando si el resultado es positivo, negativo o cero.

4. Variables a utilizar

Variable	**Tipo de dato**	**Descripción**
dolares	float	Almacena el monto ingresado en dólares.
pesos	float	Almacena el equivalente del monto en pesos.
es_positivo	int	Almacena el resultado de la expresión relacional (1 si es verdadero y 0 si es falso).
TASA_CAMBIO	Constante simbólica	Representa la tasa de cambio de 4000.0.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Dolares_Pesos

    Definir dolares, pesos Como Real
    Definir es_positivo Como Logico

    Escribir "Ingrese el monto en dolares:"
    Leer dolares

    pesos <- dolares * 4000

    es_positivo <- pesos > 0

    Escribir dolares, " dolares equivalen a ", pesos, " pesos"

    Si es_positivo Entonces
        Escribir "El resultado es positivo."
    SiNo
        Escribir "El resultado es negativo o cero."
    FinSi

FinAlgoritmo
```

5.1. Prueba de escritorio

Datos de entrada: dolares = 10

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	dolares = 10	No aplica	Ingrese el monto en dólares: 10
2	pesos = 40000	No aplica	—
3	es_positivo = 1	pesos > 0 →Verdadero	—
4	es_positivo = 1	if (es_positivo) → Verdadero	El resultado es positivo.

Operación realizada:

- pesos = 10 * 4000 = 40000

5.2. Prueba de escritorio

Datos de entrada: dolares = 15

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	dolares = 15	No aplica	Ingrese el monto en dólares: 15
2	pesos = 60000	No aplica	—
3	es_positivo = 1	pesos > 0 →Verdadero	—
4	es_positivo = 1	if (es_positivo) → Verdadero	El resultado es positivo.

Operación realizada:

- pesos = 15 * 4000 = 60000

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define TASA_CAMBIO 4000.0

int main() {
    float dolares = 0.0;
    float pesos = 0.0;
    int es_positivo = 0;

    printf("Ingrese el monto en dolares: ");
    scanf("%f", &dolares);

    pesos = (dolares * TASA_CAMBIO);

    es_positivo = (pesos > 0);

    printf("\n%.2f dolares equivalen a %.2f pesos\n", dolares, pesos);

    if (es_positivo)
    {
        printf("El resultado es positivo.\n");
    }
    else
    {
        printf("El resultado es negativo o cero.\n");
    }
}
```

7. Salida del programa

Prueba 1

```
Ingrese el monto en dolares: 10

10.00 dolares equivalen a 40000.00 pesos
El resultado es positivo.
```

Prueba 2

```
Ingrese el monto en dolares: 15

15.00 dolares equivalen a 60000.00 pesos
El resultado es positivo.
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 15

1.Enunciado del problema

En un taller mecanico, diseñe un algoritmo que lea una medida en centimetros (real). Use una constante simbolica FACTOR = 0.393701. Calcule la equivalencia en pulgadas con la expresion aritmetica (cm * FACTOR) y asignela a pulgadas. Evalúe la expresion relacional (pulgadas > 0). Muestre por pantalla 'X cm equivalen a Y pulgadas'. Ademas del calculo principal, calcule el resultado elevado al cubo. Asegurese de que las expresiones logicas esten bien parentizadas.

2.Analisis del problema

1. Datos de entrada

cm: medida en centímetros ingresada por teclado (tipo real).

2. Proceso

1. Leer la medida en centímetros.
2. Declarar la constante simbólica:
    - FACTOR = 0.393701 (representa el factor de conversión de centímetros a pulgadas).
3. Calcular la equivalencia en pulgadas mediante la expresión aritmética:

$$pulgadas = cm \times FACTOR$$

4. Evaluar la expresión relacional para determinar si el resultado es positivo:

$$\left(pulgadas > 0\right)$$

5. Asignar el resultado de la expresión relacional a la variable es_positivo.
6. Calcular el resultado elevado al cubo:

$$cubo = pulgadas \times pulgadas \times pulgadas$$

7. Mostrar por pantalla la equivalencia entre centímetros y pulgadas.
8. Mostrar el resultado de la expresión relacional y el valor del resultado elevado al cubo.

3. Datos de salida

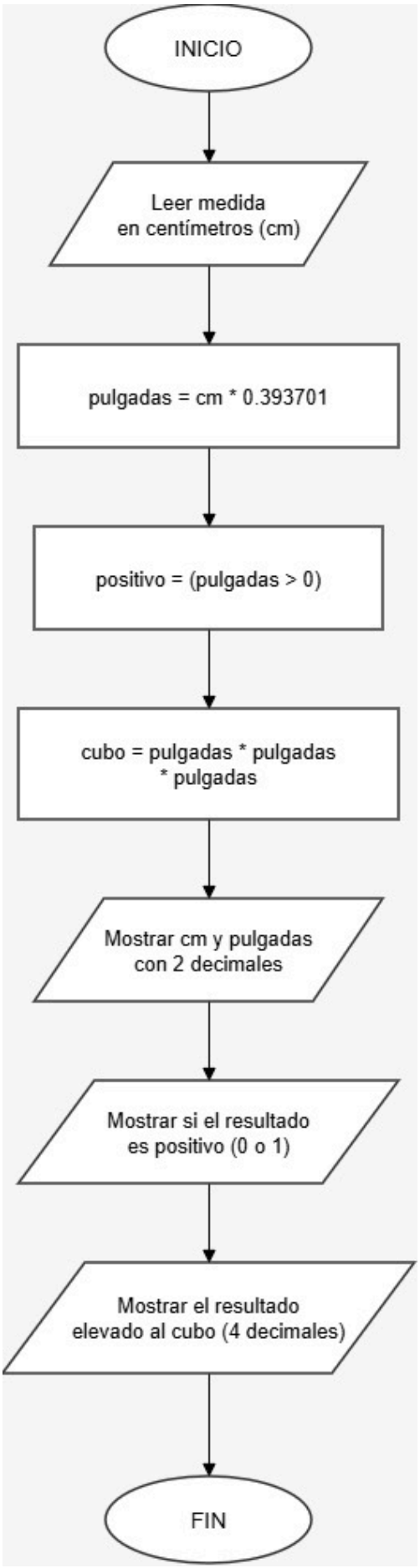
- Medida ingresada en centímetros.
- Equivalencia calculada en pulgadas.
- Resultado de la expresión relacional (1 si es verdadero y 0 si es falso).

- Resultado de las pulgadas elevadas al cubo.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
cm	float	Almacena la medida ingresada en centímetros.
pulgadas	float	Almacena la equivalencia de centímetros a pulgadas.
es_positivo	int	Almacena el resultado de la expresión relacional (1 verdadero y 0 falso).
cubo	float	Almacena el resultado de elevar las pulgadas al cubo.
FACTOR	Constante simbólica	Representa el factor de conversión de centímetros a pulgadas (0.393701).

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Centimetros_Pulgadas

    Definir cm, pulgadas, cubo Como Real
    Definir positivo Como Entero
    Definir FACTOR Como Real

    FACTOR <- 0.393701

    cm <- 0
    pulgadas <- 0
    cubo <- 0
    positivo <- 0

    Escribir "Ingrese la medida en centimetros: "
    Leer cm

    pulgadas <- cm * FACTOR

    positivo <- (pulgadas > 0)

    cubo <- pulgadas * pulgadas * pulgadas

    Escribir cm, " cm equivalen a ", pulgadas, " pulgadas"

    Escribir "El resultado es positivo: ", positivo

    Escribir "El resultado elevado al cubo es: ", cubo

FinAlgoritmo
```

5.1. Prueba de escritorio

Datos de entrada: cm = 10

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cm = 10	No aplica	Ingrese la medida en centímetros: 10
2	pulgadas = 3.93701	No aplica	—
3	positivo = 1	pulgadas > 0 → Verdadero	—
4	cubo = 61.0233	No aplica	—
5	cm = 10, pulgadas = 3.94	No aplica	10.00 cm equivalen a 3.94 pulgadas
6	positivo = 1	No aplica	El resultado es positivo: 1
7	cubo = 61.0233	No aplica	El resultado elevado al cubo es: 61.0233

Operaciones realizadas:

- pulgadas = 10 * 0.393701 = 3.93701
- positivo = (3.93701 > 0) = 1
- cubo = 3.93701 * 3.93701 * 3.93701 = 61.0233

5.2. Prueba de escritorio

Datos de entrada: cm = 15

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cm = 15	No aplica	Ingrese la medida en centímetros: 15
2	pulgadas = 5.905515	No aplica	—
3	positivo = 1	pulgadas > 0 → Verdadero	—
4	cubo = 206.0292	No aplica	—
5	cm = 15, pulgadas = 5.91	No aplica	15.00 cm equivalen a 5.91 pulgadas
6	positivo = 1	No aplica	El resultado es positivo: 1
7	cubo = 206.0292	No aplica	El resultado elevado al cubo es: 206.0292

Operaciones realizadas:

- pulgadas = 15 * 0.393701 = 5.905515
- positivo = (5.905515 > 0) = 1
- cubo = 5.905515 * 5.905515 * 5.905515 = 206.0292

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define FACTOR 0.393701

int main(){
    float cm = 0.0;
    float pulgadas = 0.0;
    float cubo = 0.0;
    int positivo = 0;

    printf("Ingrese la medida en centimetros: ");
    scanf("%f", &cm);

    pulgadas = (cm * FACTOR);

    positivo = (pulgadas > 0);

    cubo = (pulgadas * pulgadas * pulgadas);

    printf("\n%.2f cm equivalen a %.2f pulgadas\n", cm, pulgadas);

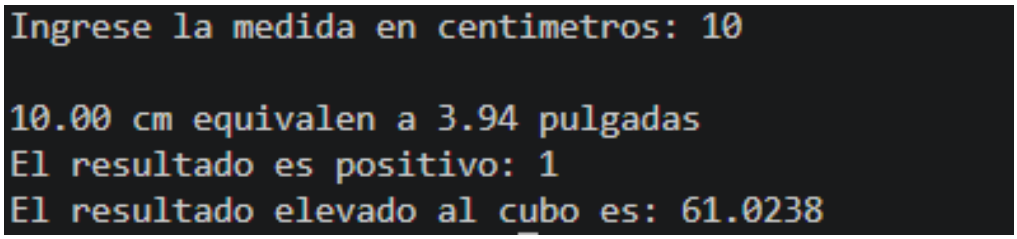
    printf("El resultado es positivo: %d\n", positivo);

    printf("El resultado elevado al cubo es: %.4f\n", cubo);

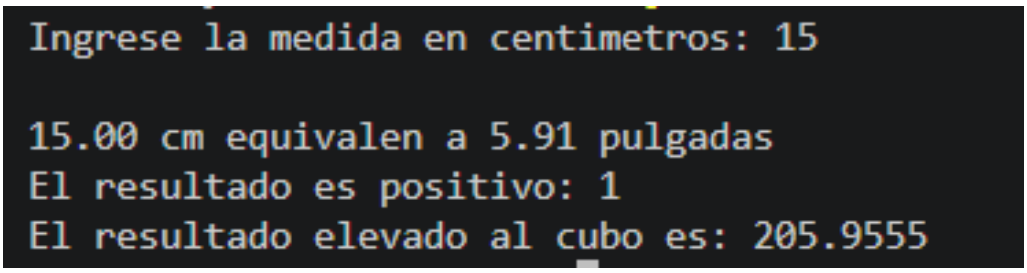
}
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 16

1.Enunciado del problema

En una floreria, construya un algoritmo que lea una altura en pies (real). Use una constante simbolica PIE_METRO = 0.3048. Calcule la equivalencia en metros con la expresion aritmetica (pies * PIE_METRO) y asignela a metros. Evalúe la expresion relacional (metros > 0). Muestre por pantalla 'X pies equivalen a Y metros'. Ademas del calculo principal, determine el valor maximo de las variables utilizadas. Documento claramente cuáles son las constantes simbolicas utilizadas.

2.Analisis del problema

1. Datos de entrada

- pies: altura expresada en pies (real).

2. Proceso

- Definir la constante simbólica:
  - PIE_METRO = 0.3048.
- Convertir pies a metros:
  - metros = pies * PIE_METRO.
- Evaluar la expresión relacional:
  - positivo = (metros > 0).
- Determinar el valor máximo de las variables utilizadas.

3. Datos de salida

- Mostrar la equivalencia de pies a metros.
- Mostrar si la medida en metros es mayor que cero.
- Mostrar el valor máximo de las variables utilizadas.

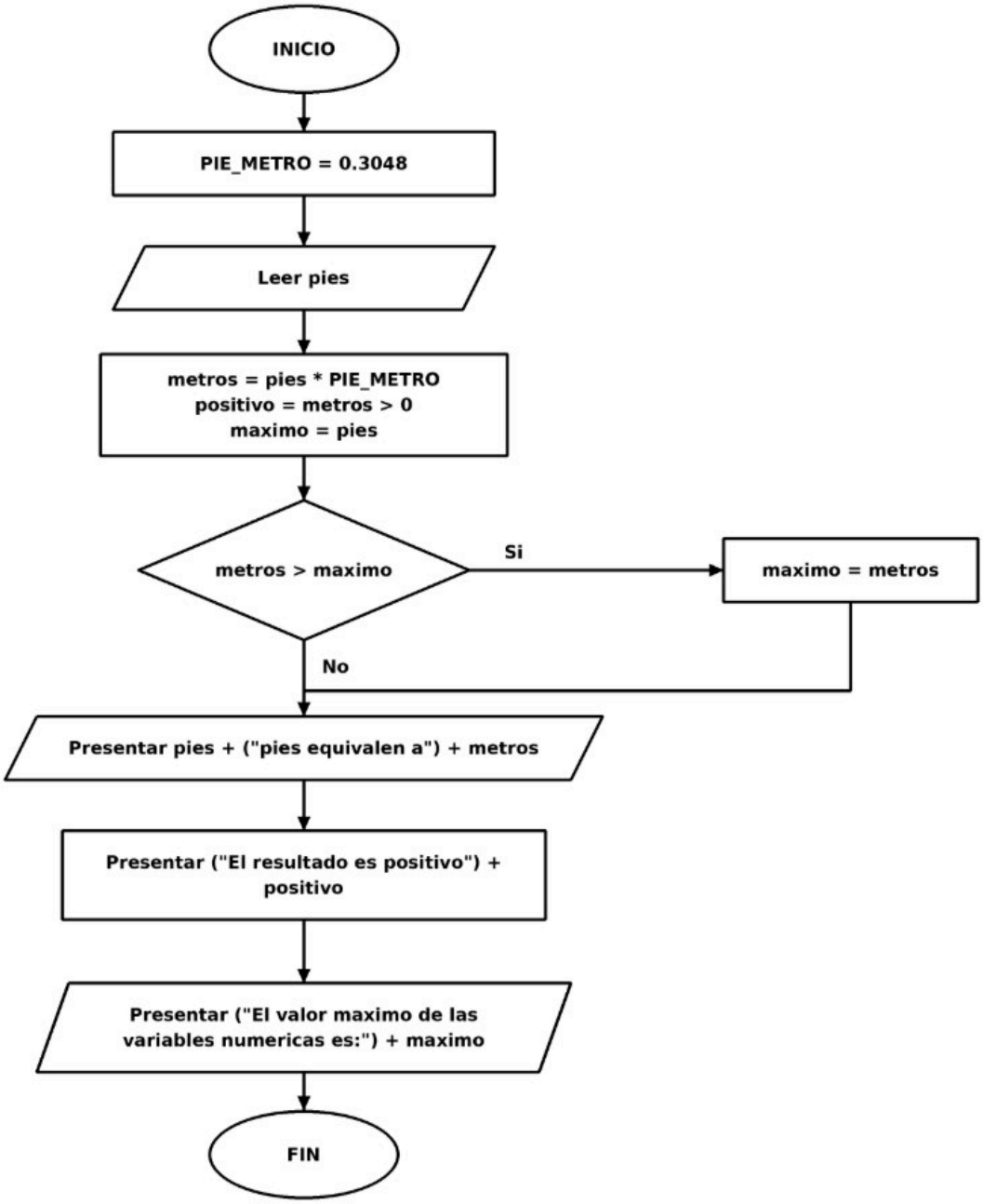
4. Variables a utilizar

Variable	Tipo de dato	Descripción
pies	float	Altura ingresada en pies.
metros	float	Equivalencia de pies a metros.
positivo	int	Resultado de la expresión relacional (1 verdadero, 0 falso).
maximo	float	Valor máximo determinado.

Constante simbólica:

Constante	Valor	Descripción
PIE_METRO	0.3048	Factor de conversión de pies a metros.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Pies_Metros

    Definir pies, metros, maximo Como Real
    Definir positivo Como Entero

    Definir PIE_METRO Como Real

    PIE_METRO <- 0.3048

    pies <- 0.0
    metros <- 0.0
    maximo <- 0.0
    positivo <- 0

    Escribir "Ingrese la altura en pies: "
    Leer pies

    metros <- pies * PIE_METRO

    positivo <- (metros > 0)

    maximo <- pies

    Si metros > maximo Entonces
        maximo <- metros
    FinSi

    Escribir ""
    Escribir pies, " pies equivalen a ", metros, " metros"

    Escribir "El resultado es positivo: ", positivo

    Escribir "El valor maximo de las variables numericas es: ", maximo

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1

Datos de entrada: pies = 10

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	pies = 10	No aplica	Ingrese la altura en pies: 10
2	metros = 3.048	No aplica	—
3	positivo = 1	metros > 0 → Verdadero	—
4	maximo = 10	No aplica	—
5	maximo = 10	metros > maximo → Falso	—
6	pies = 10, metros = 3.048	No aplica	10.00 pies equivalen a 3.05 metros
7	positivo = 1	No aplica	El resultado es positivo: 1
8	maximo = 10	No aplica	El valor máximo de las variables numéricas es: 10.00

Operaciones realizadas:

- metros = 10 * 0.3048 = 3.048
- positivo = (3.048 > 0) = 1
- maximo = 10
- 3.048 > 10 →Falso, se mantiene maximo = 10.

5.2. Prueba de escritorio N.º 2

Datos de entrada: pies = 5

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	pies = 5	No aplica	Ingrese la altura en pies: 5
2	metros = 1.524	No aplica	—
3	positivo = 1	metros > 0 → Verdadero	—
4	maximo = 5	No aplica	—
5	maximo = 5	metros > maximo → Falso	—
6	pies = 5, metros = 1.524	No aplica	5.00 pies equivalen a 1.52 metros
7	positivo = 1	No aplica	El resultado es positivo: 1
8	maximo = 5	No aplica	El valor máximo de las variables numéricas es: 5.00

Operaciones realizadas:

- metros = 5 * 0.3048 = 1.524
- positivo = (1.524 > 0) = 1
- maximo = 5
- 1.524 > 5 →Falso, se mantiene maximo = 5.

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define PIE_METRO 0.3048

int main()
{
    float pies = 0.0;
    float metros = 0.0;
    float maximo = 0.0;
    int positivo = 0;

    printf("Ingrese la altura en pies: ");
    scanf("%f", &pies);

    metros = (pies * PIE_METRO);

    positivo = (metros > 0);

    maximo = pies;

    if (metros > maximo)
    {
        maximo = metros;
    }

    printf("\n%.2f pies equivalen a %.2f metros\n", pies, metros);

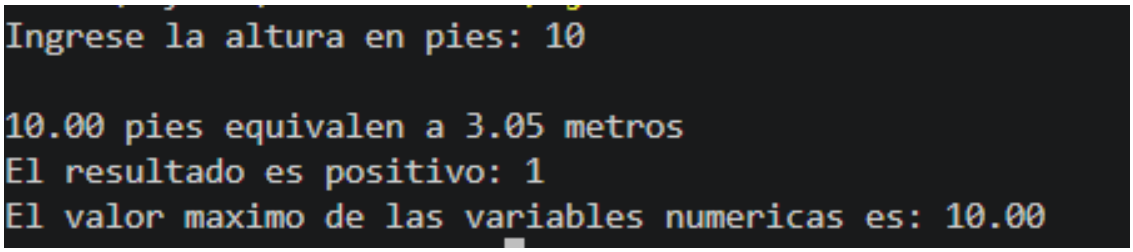
    printf("El resultado es positivo: %d\n", positivo);

    printf("El valor maximo de las variables numericas es: %.2f\n", maximo);

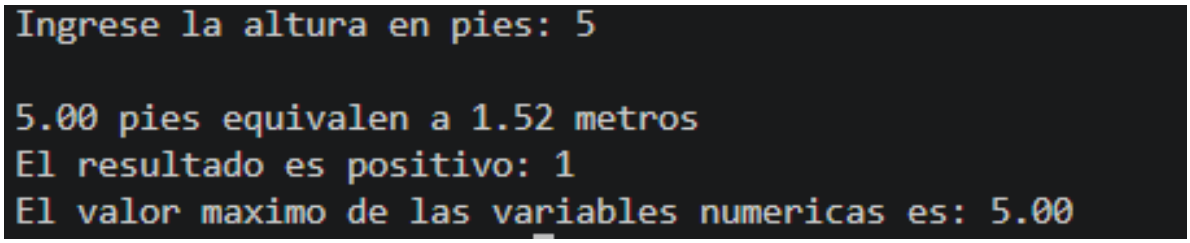
    return 0;
}
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERICICIO 17

1.Enunciado del problema

En un hospital, construya un algoritmo que lea un angulo en grados sexagesimales (real). Use la constante simbolica PI = 3.14159. Calcule la equivalencia en radianes con la expresion aritmetica (grados * PI / 180) y asignela a radianes. Evalúe la expresion logica (grados >= 0 AND grados <= 360). Muestre por pantalla 'X grados equivalen a Y radianes'. Ademas del calculo principal, calcule el resultado elevado al cubo. Incluya un encabezado descriptivo al iniciar la ejecucion.

2.Analisis del problema

1. Datos de entrada

- grados: ángulo ingresado por teclado, expresado en grados sexagesimales (real).

2. Proceso

1. Definir la constante simbólica:
    - PI = 3.14159.
2. Mostrar un encabezado descriptivo al iniciar la ejecución del algoritmo.
3. Convertir los grados sexagesimales a radianes mediante la expresión aritmética:

$$radianes = \frac{grados \times PI}{180}$$

4. Evaluar la expresión lógica para verificar que el ángulo esté dentro del rango permitido:

$$valido = \left(grados \geq 0 \; AND \; grados \leq 360\right)$$

5. Calcular el resultado elevado al cubo:

$$cubo = radianes \times radianes \times radianes$$

3. Datos de salida

- Encabezado descriptivo del algoritmo.
- Equivalencia del ángulo en grados a radianes.
- Resultado de la evaluación lógica (verdadero o falso).
- Resultado de los radianes elevados al cubo.

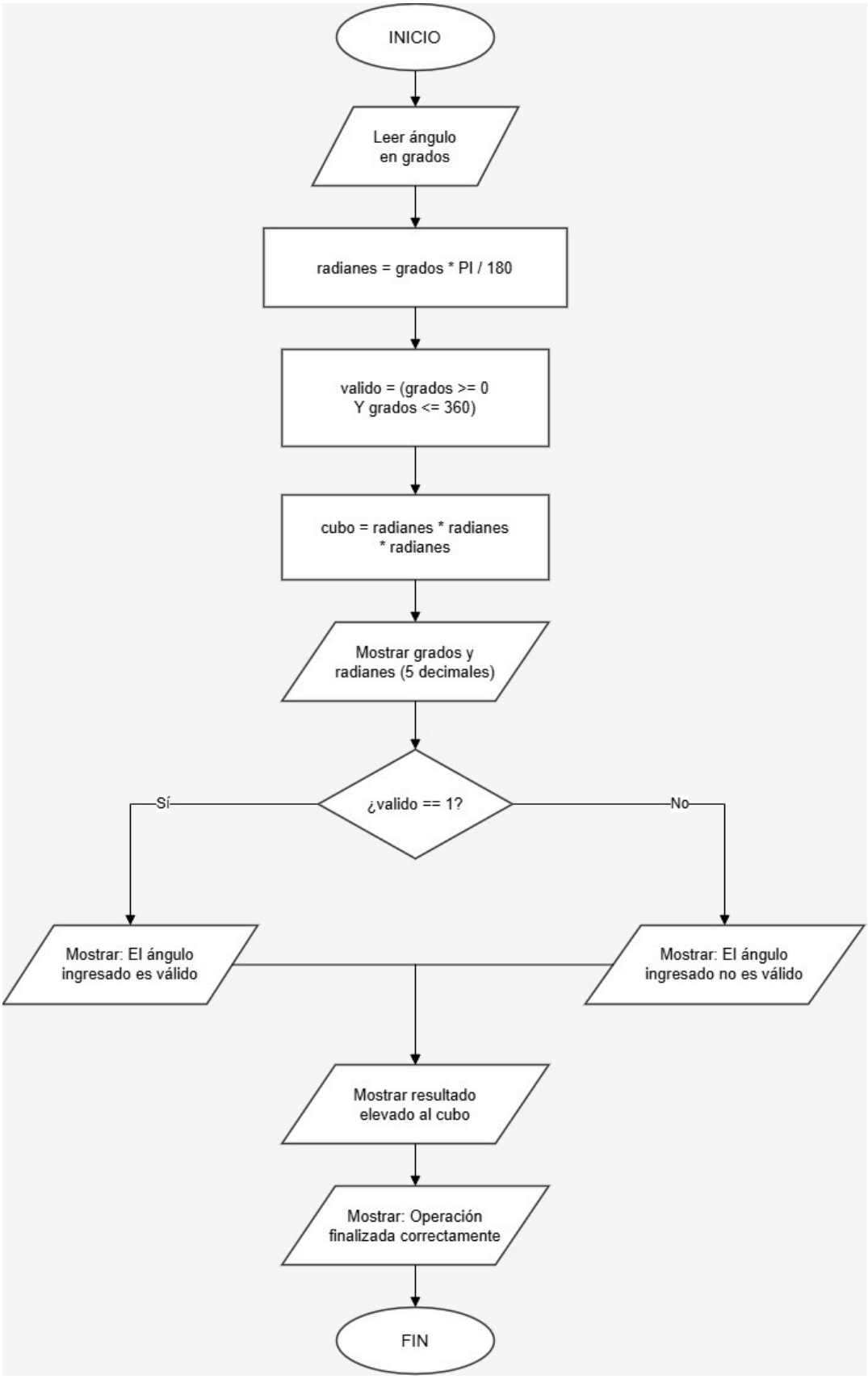
4. Variables a utilizar

Variable	Tipo de dato	Descripción
grados	float	Almacena el ángulo ingresado en grados sexagesimales.
radianes	float	Almacena la equivalencia del ángulo en radianes.
valido	int	Almacena el resultado de la expresión lógica (1 = verdadero, 0 = falso).
cubo	float	Almacena el resultado de los radianes elevados al cubo.

Constante simbólica:

Constante	Valor	Descripción
PI	3.14159	Representa el valor aproximado de π utilizado para la conversión de grados a radianes.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Grados_Radianes

    Definir grados, radianes, cubo Como Real
    Definir valido Como Entero
    Definir PI Como Real

    PI <- 3.14159

    Escribir "Ingrese el angulo en grados: "
    Leer grados

    radianes <- (grados * PI) / 180

    valido <- (grados >= 0 Y grados <= 360)

    cubo <- radianes * radianes * radianes

    Escribir grados, " grados equivalen a ", radianes, " radianes"

    Si valido = 1 Entonces

        Escribir "El angulo ingresado es valido."

    SiNo

        Escribir "El angulo ingresado no es valido."

    FinSi

    Escribir "El resultado elevado al cubo es: ", cubo

    Escribir "Operacion finalizada correctamente."

FinAlgoritmo
```

5.1. Prueba de escritorio

1. Datos de entrada

90 grados

2. Fórmulas utilizadas

Conversión de grados a radianes:

$$Radianes = \frac{Grados \times 3.14159}{180}$$

Cálculo del cubo:

$$Cubo = Radianes \times Radianes \times Radianes$$

Condición de validación:

$$0 \leq Grados \leq 360$$

Paso	Variable / proceso	Resultado
1	grados = 90	90
2	radianes = (90 * 3.14159) / 180	1.57080
3	valido = (90 >= 0 && 90 <= 360)	Verdadero (1)
4	cubo = 1.570795 * 1.570795 * 1.570795	3.87578
5	Evaluación valido == 1	Verdadero
6	Mostrar resultado	Ángulo válido

5.2. Prueba de escritorio

1. Datos de entrada

400 grados

2. Fórmulas utilizadas

Conversión de grados a radianes:

$$Radianes = \frac{Grados \times 3.14159}{180}$$

Cálculo del cubo:

$$Cubo = Radianes \times Radianes \times Radianes$$

Condición de validación:

$$0 \leq Grados \leq 360$$

Paso	Variable / proceso	Resultado
1	grados = 400	400
2	radianes = (400 * 3.14159) / 180	6.98131
3	valido = (400 >= 0 && 400 <= 360)	Falso (0)
4	cubo = 6.98131 * 6.98131 * 6.98131	340.29175
5	Evaluación valido == 1	Falso
6	Mostrar resultado	Ángulo no válido

6. Codificación en lenguaje C

```c
#include <stdio.h>

#define PI 3.14159

int main()
{
    float grados;
    float radianes;
    float cubo;
    int valido;


    printf("Ingrese el angulo en grados: ");
    scanf("%f", &grados);

    radianes = (grados * PI / 180);

    valido = (grados >= 0 && grados <= 360);

    cubo = (radianes * radianes * radianes);

    printf("\n%.2f grados equivalen a %.5f radianes\n", grados, radianes);

    if (valido == 1)
    {
        printf("El angulo ingresado es valido.\n");
    }
    else
    {
        printf("El angulo ingresado no es valido.\n");
    }

    printf("El resultado elevado al cubo es: %.5f\n", cubo);

    printf("\nOperacion finalizada correctamente.\n");

    return 0;
}
```

7. Salida del programa

Prueba 1

```
Ingrese el angulo en grados: 90

90.00 grados equivalen a 1.57080 radianes
El angulo ingresado es valido.
El resultado elevado al cubo es: 3.87578

Operacion finalizada correctamente.
```

Prueba 2

```
Ingrese el angulo en grados: 400

400.00 grados equivalen a 6.98131 radianes
El angulo ingresado no es valido.
El resultado elevado al cubo es: 340.26010

Operacion finalizada correctamente.
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 18

1.Enunciado del problema

En una universidad, cree un algoritmo que lea un monto en soles (real). Use una constante simbolica TASA_CAMBIO = 0.27. Calcule el equivalente en dolares con la expresion aritmetica (soles * TASA_CAMBIO) y asignelo a dolares. Evalúe la expresion relacional (dolares > 0). Muestre por pantalla 'X soles equivalen a Y dolares'. Adicionalmente, redondee el resultado a 2 decimales. Incluya una seccion de conclusiones al final del algoritmo.

2.Analisis del problema

1. Datos de entrada

- soles: monto de dinero ingresado por teclado, expresado en soles (real).

2. Proceso

1. Definir la constante simbólica:
    - TASA_CAMBIO = 0.27.
2. Calcular la equivalencia de soles a dólares mediante la expresión aritmética:

$$dolares = soles \times TASA_CAMBIO$$

3. Evaluar la expresión relacional para determinar si el monto convertido es mayor que cero:

$$positivo = \left(dolares > 0\right)$$

4. Redondear el resultado de la conversión a dos decimales.
5. Mostrar una sección de conclusiones al finalizar el algoritmo.

3. Datos de salida

- Equivalencia del monto ingresado en soles a dólares.
- Resultado de la evaluación relacional (verdadero o falso).
- Resultado convertido y redondeado a dos decimales.
- Conclusiones finales del algoritmo.

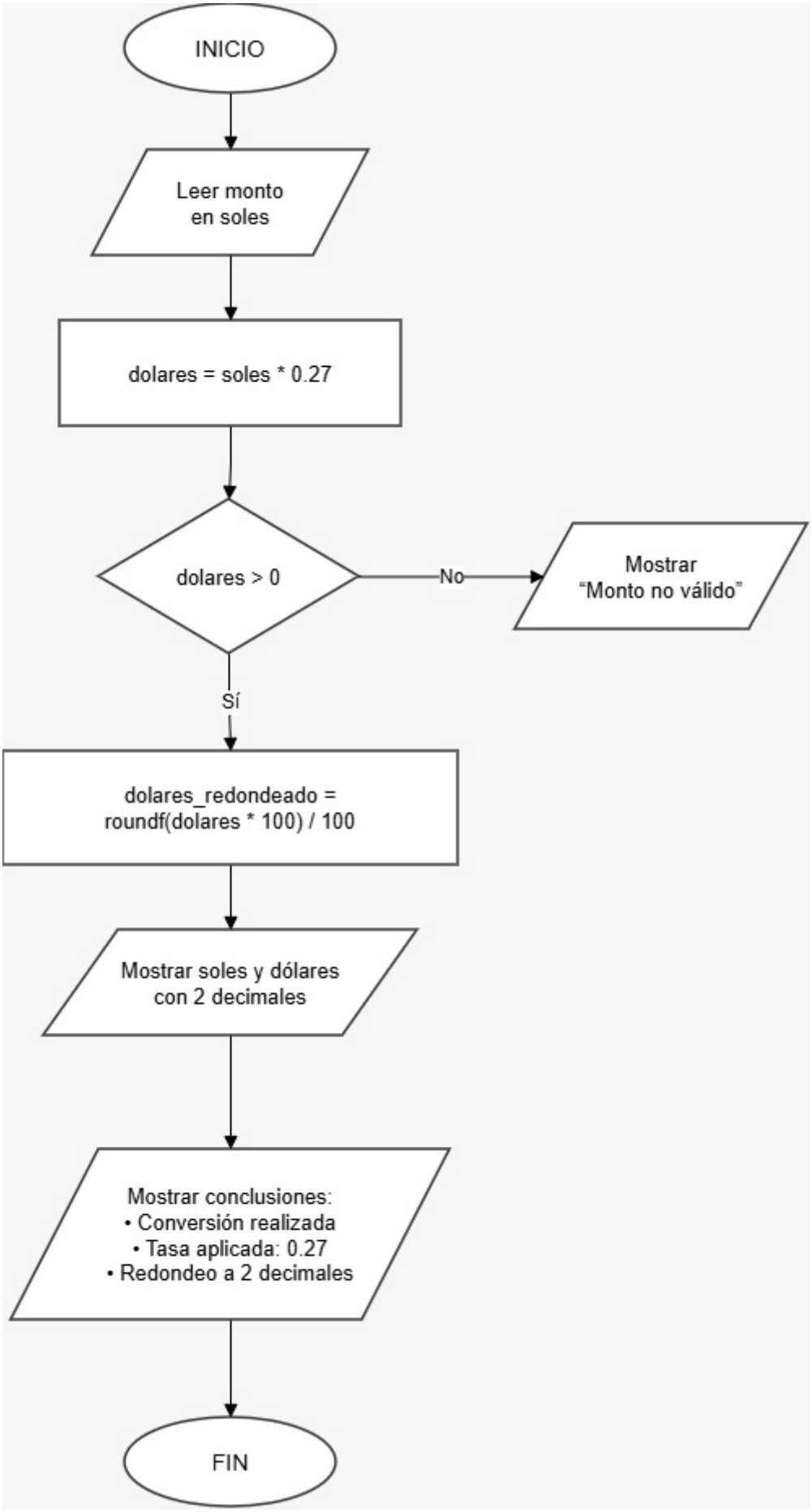
4. Variables a utilizar

Variable	Tipo de dato	Descripción
soles	float	Almacena el monto ingresado en soles.
dolares	float	Almacena el resultado de la conversión a dólares.
positivo	int	Almacena el resultado de la expresión relacional (1 = verdadero, 0 = falso).
dolares_redondeados	float	Almacena el resultado de la conversión redondeado a dos decimales.

Constante simbólica:

Constante	Valor	Descripción
TASA_CAMBIO	0.27	Factor utilizado para convertir soles a dólares.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Soles_Dolares

    Definir soles, dolares, dolares_redondeado Como Real
    Definir TASA_CAMBIO Como Real

    TASA_CAMBIO <- 0.27
    soles <- 0
    dolares <- 0
    dolares_redondeado <- 0

    Escribir "Ingrese el monto en soles: "
    Leer soles

    dolares <- soles * TASA_CAMBIO

    Si dolares > 0 Entonces

        dolares_redondeado <- Redondear(dolares * 100) / 100

        Escribir soles, " soles equivalen a ",
                 dolares_redondeado, " dolares"

    SiNo

        Escribir "Monto no valido"

    FinSi

    Escribir " CONCLUSIONES "
    Escribir "La conversion de soles a dolares se realizo correctamente."
    Escribir "Se aplico la tasa de cambio de 0.27."
    Escribir "El resultado se redondeo a dos decimales."

FinAlgoritmo
```

5.1. Prueba de escritorio N.º 1: Triángulo equilátero

Datos de entrada: soles = 100

N.º de paso	Variables / procesos	Evaluación de condiciones	Salida en pantalla
1	soles = 100	No aplica	Ingrese el monto en soles: 100
2	dolares = 100 * 0.27 = 27	No aplica	—
3	dolares > 0	Verdadero (1)	—
4	dolares_redondeado = roundf(27 * 100) / 100 = 27.00	No aplica	—
5	Se ejecuta el bloque if	Verdadero	100.00 soles equivalen a 27.00 dolares
6	Se muestran las conclusiones	No aplica	Se muestran los tres mensajes finales

5.2. Prueba de escritorio N.º 2: Triángulo no válido

Datos de entrada: soles = 50

N.º de paso	Variables / procesos	Evaluación de condiciones	Salida en pantalla
1	soles = 50	No aplica	Ingrese el monto en soles: 50
2	dolares = 50 * 0.27 = 13.50	No aplica	—
3	dolares > 0	Verdadero (1)	—
4	dolares_redondeado = roundf(13.50 * 100) / 100 = 13.50	No aplica	—
5	Se ejecuta el bloque if	Verdadero	50.00 soles equivalen a 13.50 dolares
6	Se muestran las conclusiones	No aplica	Se muestran los tres mensajes finales

6. Codificación en lenguaje C

```c

#include <stdio.h>
#include <math.h>

#define TASA_CAMBIO 0.27

int main() {
    float soles = 0.0;
    float dolares = 0.0;
    float dolares_redondeado = 0.0;

    printf("Ingrese el monto en soles: ");
    scanf("%f", &soles);

    dolares = soles * TASA_CAMBIO;

    if (dolares > 0)
    {
        dolares_redondeado = roundf(dolares * 100) / 100;

        printf("\n%.2f soles equivalen a %.2f dolares\n",
               soles, dolares_redondeado);
    }
    else
    {
        printf("\nMonto no valido\n");
    }

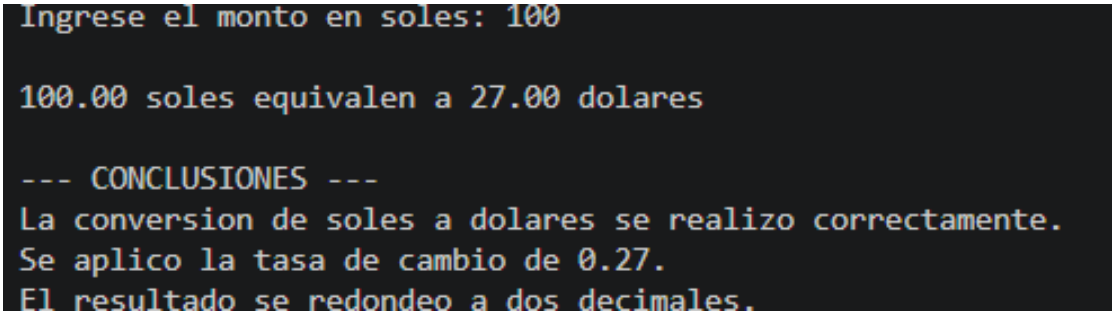
    printf("\n--- CONCLUSIONES ---\n");
    printf("La conversion de soles a dolares se realizo correctamente.\n");
    printf("Se aplico la tasa de cambio de 0.27.\n");
    printf("El resultado se redondeo a dos decimales.\n");


}

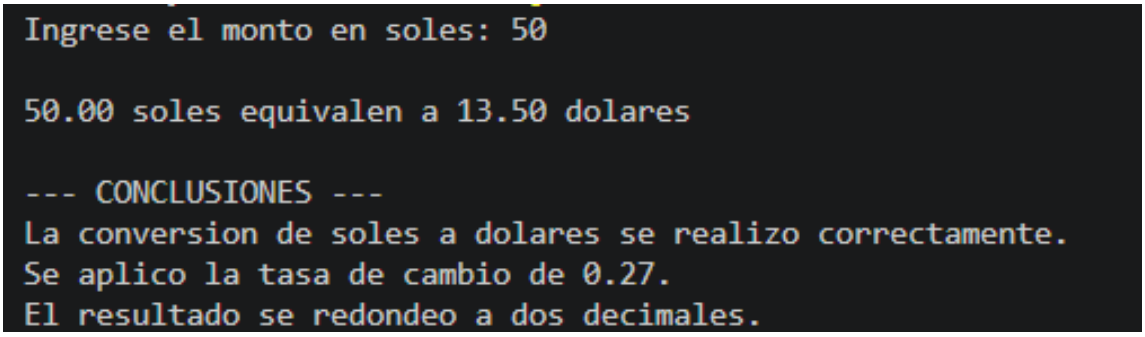
```

7. Salida del programa

Prueba 1



Prueba 2



8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba

EJERCICIO 19

1.Enunciado del problema

En una libreria, construya un algoritmo que lea un peso en libras (real). Use una constante simbolica LIBRA_KG = 0.453592. Calcule el equivalente en kilogramos con la expresion aritmetica (libras * LIBRA_KG) y asignelo a kg. Evalúe la expresion relacional (kg > 0). Muestre por pantalla 'X libras equivalen a Y kg'. Ademas del calculo principal, calcule el logaritmo en base 10 del resultado. Muestre al final los valores de todas las variables auxiliares.

2.Analisis del problema

1. Datos de entrada

Son los datos que el usuario debe ingresar por teclado para realizar los cálculos.

Dato	Tipo de dato	Descripción
libras	Real (float)	Peso ingresado por el usuario en libras.

2. Proceso

1. Declarar la constante simbólica LIBRA_KG = 0.453592.
2. Leer el peso en libras ingresado por el usuario.
3. Calcular la equivalencia en kilogramos mediante la expresión aritmética:

$$\boldsymbol{kg = libras \times LIBRA_KG}$$

4. Evaluar la expresión relacional:

$$\boldsymbol{kg} > 0$$

5. Si el resultado es verdadero, calcular el logaritmo en base 10 del valor obtenido en kilogramos:

$$\boldsymbol{kg_log} = \log_{10}(\boldsymbol{kg})$$

6. Mostrar por pantalla el mensaje: «X libras equivalen a Y kg».
7. Mostrar el resultado del logaritmo en base 10.
8. Mostrar al final los valores de todas las variables auxiliares utilizadas.
9. Si la condición es falsa, mostrar un mensaje indicando que el peso ingresado no es válido.

3. Datos de salida

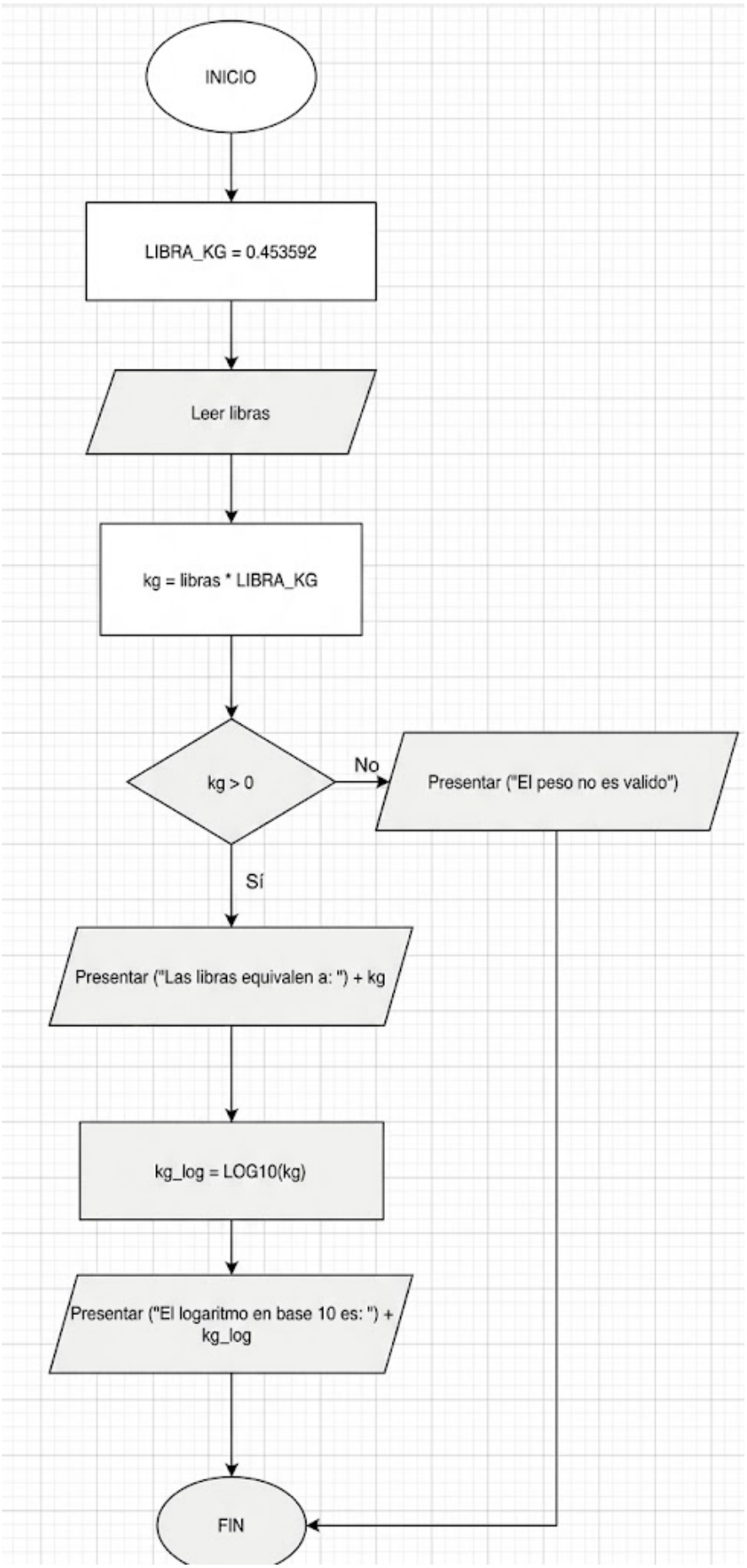
Son los resultados y mensajes que el programa mostrará en pantalla.

Dato de salida	Descripción
kg	Equivalencia del peso ingresado en kilogramos.
validacion	Resultado de comprobar si los kilogramos son mayores que cero (verdadero o falso).
kg_log	Resultado del logaritmo en base 10 de los kilogramos.
Valores auxiliares	Muestra los valores de las variables utilizadas durante el proceso.
Mensaje de validación	Indica si el peso ingresado es válido o no.

4. Variables a utilizar

Variable	Tipo de dato	Descripción
libras	float	Almacena el peso ingresado por el usuario en libras.
kg	float	Almacena el equivalente del peso en kilogramos.
kg_log	float	Almacena el resultado del logaritmo en base 10.
validacion	int	Almacena el resultado de la expresión relacional (1 = verdadero, 0 = falso).
LIBRA_KG	Constante real	Factor de conversión equivalente a 0.453592.

3. Diagrama de flujo



4. Pseudocódigo

```
Conversion_Libras_Kilogramos

    Definir libras, kg, kg_log Como Real
    Constante LIBRA_KG = 0.453592

    libras = 0.0
    kg = 0.0
    kg_log = 0.0

    Escribir "Ingrese el peso en libras: "
    Leer libras

    kg = libras * LIBRA_KG

    Si kg > 0 Entonces

        Escribir libras, " libras equivalen a: ", kg, " kg"

        kg_log = log10(kg)

        Escribir "El logaritmo en base 10: ", kg_log

    SiNo

        Escribir "El peso no es valido"

    FinSi

    Escribir "--- VALORES DE LAS VARIABLES ---"
    Escribir "Libras: ", libras
    Escribir "Kilogramos: ", kg
    Escribir "Logaritmo: ", kg_log
    Escribir "Factor de conversion: ", LIBRA_KG

FinAlgoritmo
```

5.1 Prueba de escritorio N.º 1: Peso válido

Dato de entrada: 10 libras.

Cálculos:

- Conversión a kilogramos:

$$kg = 10 \times 0.453592 = 4.53592$$

- Evaluación de la condición:

$$kg > 0 \Rightarrow 4.53592 > 0 = 1$$

- Logaritmo en base 10:

$$kg_log = \log_{10}(4.53592) \approx 0.6566$$

Tabla de prueba de escritorio:

Paso	libras	kg	kg_log	Condición kg > 0	Salida en pantalla
1	0.00	0.0000	0.0000	—	Ingrese el peso en libras:
2	10.00	0.0000	0.0000	—	Se ingresa 10
3	10.00	4.5359	0.0000	—	Se calcula kg
4	10.00	4.5359	0.0000	Verdadero (1)	Entra al if
5	10.00	4.5359	0.6566	Verdadero (1)	Se calcula el logaritmo
6	10.00	4.5359	0.6566	—	Muestra los valores de las variables

5.2 Prueba de escritorio N.º 2: Peso no válido

Dato de entrada: -5 libras.

Cálculos:

- Conversión a kilogramos:

$$kg = -5 \times 0.453592 = -2.26796$$

- Evaluación de la condición:

$$kg > 0 \Rightarrow -2.26796 > 0 = 0$$

Como la condición es falsa, el programa ejecuta el bloque else y no calcula el logaritmo.

Tabla de prueba de escritorio:

Paso	libras	kg	kg_log	Condición kg > 0	Salida en pantalla
1	0.00	0.0000	0.0000	—	Ingrese el peso en libras:
2	-5.00	0.0000	0.0000	—	Se ingresa -5
3	-5.00	-2.2680	0.0000	—	Se calcula kg
4	-5.00	-2.2680	0.0000	Falso (0)	Entra al else
5	-5.00	-2.2680	0.0000	Falso (0)	El peso no es válido
6	-5.00	-2.2680	0.0000	—	Muestra los valores de las variables

6. Codificación en lenguaje C

```c

#include <stdio.h>
#include <math.h>

#define LIBRA_KG 0.453592

int main(){
    float libras = 0.0;
    float kg = 0.0;
    float kg_log = 0.0;

    printf("Ingrese el peso en libras: ");
    scanf("%f", &libras);

    kg = libras * LIBRA_KG;

    if (kg > 0)
    {
        printf("%.2f libras equivalen a: %.4f kg\n", libras, kg);

        kg_log = log10(kg);

        printf("El logaritmo en base 10: %.4f\n", kg_log);
    }
    else
    {
        printf("El peso no es valido\n");
    }
     printf("\n--- VALORES DE LAS VARIABLES ---\n");
    printf("Libras: %.2f\n", libras);
    printf("Kilogramos: %.4f\n", kg);
    printf("Logaritmo: %.4f\n", kg_log);
    printf("Factor de conversion: %.6f\n", LIBRA_KG);

}
```

7. Salida del programa

Prueba 1

```
Ingrese el peso en libras: 10
10.00 libras equivalen a: 4.5359 kg
El logaritmo en base 10: 0.6567

--- VALORES DE LAS VARIABLES ---
Libras: 10.00
Kilogramos: 4.5359
Logaritmo: 0.6567
Factor de conversion: 0.453592
```

Prueba 2

```
Ingrese el peso en libras: -5
El peso no es valido

--- VALORES DE LAS VARIABLES ---
Libras: -5.00
Kilogramos: -2.2680
Logaritmo: 0.0000
Factor de conversion: 0.453592
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.

EJERCICIO 20

1.Enunciado del problema

En una empresa de telecomunicaciones, diseñe un algoritmo que lea una distancia en yardas (real). Use la constante literal 3. Calcule la equivalencia en pies con la expresion aritmetica (yardas * 3) y asignela a pies. Evalúe la expresion relacional (pies > 0). Muestre por pantalla 'X yardas equivalen a Y pies'. Adicionalmente, calcule el cociente del resultado entre un factor de 1.5. Incluya una seccion de conclusiones al final del algoritmo.

2.Analisis del problema

1. Datos de entrada

- yardas: distancia ingresada por teclado, expresada en yardas (real).

2. Proceso

1. Utilizar la constante literal:
    - 3: factor de conversión de yardas a pies.
2. Calcular la equivalencia de yardas a pies mediante la expresión aritmética:

$$pies = yardas \times 3$$

3. Evaluar la expresión relacional para determinar si el resultado es mayor que cero:

$$positivo = \left(pies > 0\right)$$

4. Calcular el cociente del resultado entre el factor de 1.5:

$$cociente = \frac{pies}{1.5}$$

5. Mostrar una sección de conclusiones al finalizar el algoritmo.

3. Datos de salida

- Equivalencia de la distancia ingresada en yardas a pies.
- Resultado de la evaluación relacional (verdadero o falso).
- Cociente del resultado en pies dividido para 1.5.
- Conclusiones finales del algoritmo.

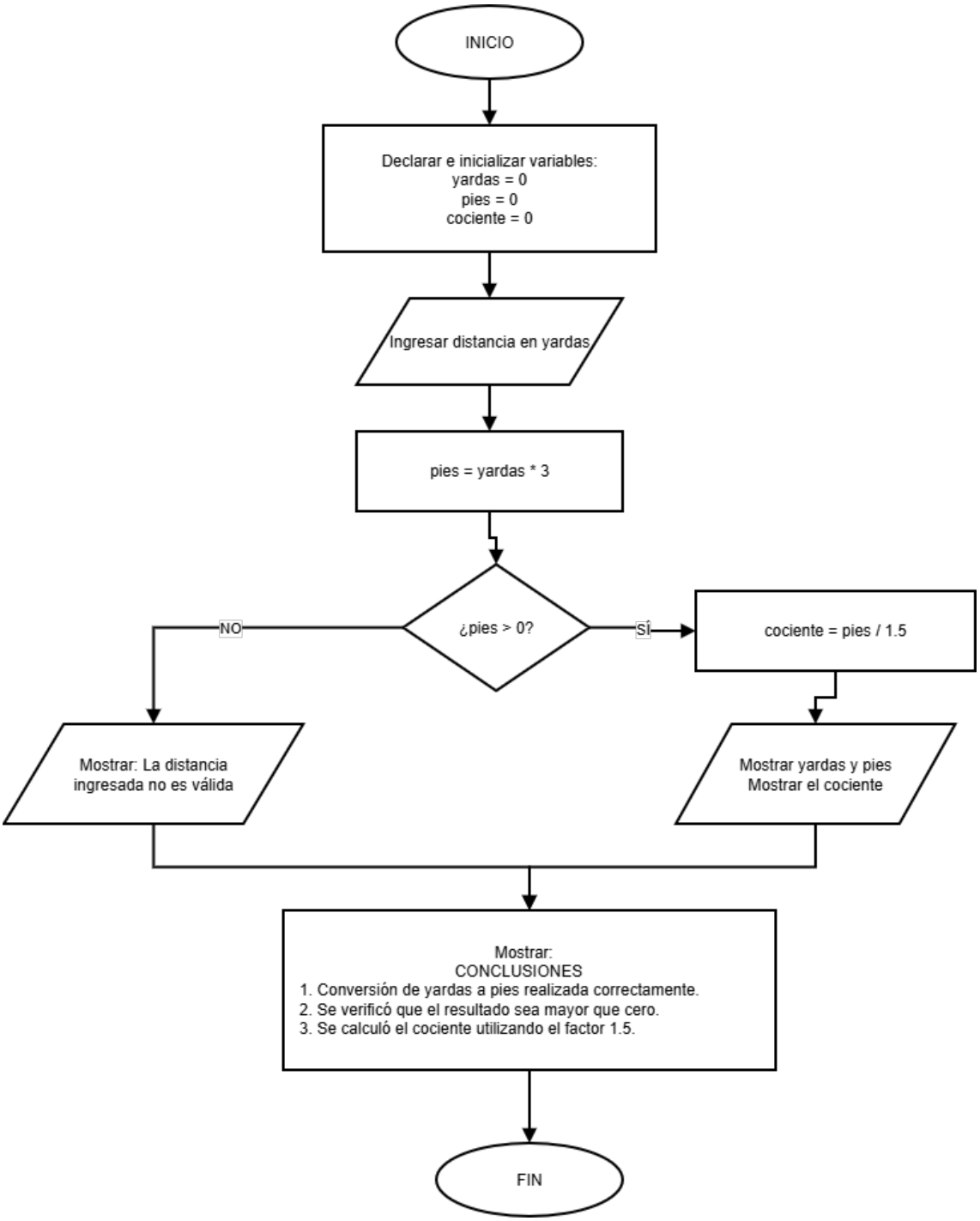
4. Variables a utilizar

Variable	Tipo de dato	Descripción
yardas	float	Almacena la distancia ingresada en yardas.
pies	float	Almacena la equivalencia de yardas a pies.
positivo	int	Almacena el resultado de la expresión relacional (1 = verdadero, 0 = falso).
cociente	float	Almacena el resultado de dividir los pies entre 1.5.

Constantes literales:

Constante	Valor	Descripción
Constante literal	3	Factor de conversión de yardas a pies.
Constante literal	1.5	Factor utilizado para calcular el cociente.

3. Diagrama de flujo



4. Pseudocódigo

```
Algoritmo Conversion_Yardas_Pies

    Definir yardas, pies, cociente Como Real

    yardas <- 0
    pies <- 0
    cociente <- 0

    Escribir "Ingrese la distancia en yardas: "
    Leer yardas

    pies <- yardas * 3

    Si pies > 0 Entonces

        cociente <- pies / 1.5

        Escribir yardas, " yardas equivalen a ", pies, " pies"

        Escribir "El cociente del resultado entre 1.5 es: ", cociente

    SiNo

        Escribir "La distancia ingresada no es valida"

    FinSi

    Escribir "--- CONCLUSIONES ---"

    Escribir "La conversion de yardas a pies se realizo correctamente."

    Escribir "Se verifico que el resultado sea mayor que cero."

    Escribir "Se calculo el cociente utilizando el factor 1.5."

FinAlgoritmo
```

5.1. Prueba de escritorio

Datos de entrada: yardas = 10

N.º de paso	Variables / procesos	Evaluación de condiciones	Salida en pantalla
1	yardas = 10	No aplica	Ingrese la distancia en yardas: 10
2	pies = 10 * 3 = 30	No aplica	—
3	pies > 0	Verdadero (1)	—
4	cociente = 30 / 1.5 = 20	No aplica	—
5	Se ejecuta el bloque if	Verdadero	10.00 yardas equivalen a 30.00 pies
6	Se muestra el cociente	No aplica	El cociente del resultado entre 1.5 es: 20.00
7	Se muestran las conclusiones	No aplica	Se muestran los tres mensajes finales

5.2. Prueba de escritorio

Datos de entrada: yardas = 5

N.º de paso	Variables / procesos	Evaluación de condiciones	Salida en pantalla
1	yardas = 5	No aplica	Ingrese la distancia en yardas: 5
2	pies = 5 * 3 = 15	No aplica	—
3	pies > 0	Verdadero (1)	—
4	cociente = 15 / 1.5 = 10	No aplica	—
5	Se ejecuta el bloque if	Verdadero	5.00 yardas equivalen a 15.00 pies
6	Se muestra el cociente	No aplica	El cociente del resultado entre 1.5 es: 10.00
7	Se muestran las conclusiones	No aplica	Se muestran los tres mensajes finales

6. Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    float yardas = 0.0;
    float pies = 0.0;
    float cociente = 0.0;


    printf("Ingrese la distancia en yardas: ");
    scanf("%f", &yardas);

    pies = yardas * 3;

    if (pies > 0)
    {

        cociente = pies / 1.5;


        printf("\n%.2f yardas equivalen a %.2f pies\n", yardas, pies);

        printf("El cociente del resultado entre 1.5 es: %.2f\n", cociente);
    }
    else
    {
        printf("\nLa distancia ingresada no es valida\n");
    }


    printf("\n--- CONCLUSIONES ---\n");
    printf("La conversion de yardas a pies se realizo correctamente.\n");
    printf("Se verifico que el resultado sea mayor que cero.\n");
    printf("Se calculo el cociente utilizando el factor 1.5.\n");



}
```

7. Salida del programa

Prueba 1

```
Ingrese la distancia en yardas: 10

10.00 yardas equivalen a 30.00 pies
El cociente del resultado entre 1.5 es: 20.00

--- CONCLUSIONES ---
La conversion de yardas a pies se realizo correctamente.
Se verifico que el resultado sea mayor que cero.
Se calculo el cociente utilizando el factor 1.5.
```

Prueba 2

```
Ingrese la distancia en yardas: 5

5.00 yardas equivalen a 15.00 pies
El cociente del resultado entre 1.5 es: 10.00

--- CONCLUSIONES ---
La conversion de yardas a pies se realizo correctamente.
Se verifico que el resultado sea mayor que cero.
Se calculo el cociente utilizando el factor 1.5.
```

8. Conclusiones del algoritmo

1. Con esta práctica he podido ver cómo esque hay que manejar los datos para que todo funcione correctamente. También he aprendido a manejar mejor las variables y las condiciones para que puedan funcionar correctamente. De esta forma he visto cómo hay que manejar un algoritmo para conseguir los resultados deseados del docente.

2. En esta actividad he tenido algunos problemas en cuanto a cómo ordenar las instrucciones. Para ello he ido viendo cómo estaba el código de forma tranquila y he podido corregir los errores que tenía. De esta forma he probado con distintos valores para ver cómo quedaba.